

Carto

(Smart Shopping Cart)

Youssef Saeed El Khouly 20222349
Alaa Waheed 20221117
Abdelrahman Sawan 20223151
Mazen Hesham 20221971

Supervised by:
Dr. Amal Mehanna
T.A Linah Elnaghi

FUTURE UNIVERSITY IN EGYPT
جـــامـعـــة المســتقبـــل

Table of Contents

Chapter 1: Introduction *12*

1.1 Background of the Study 12

1.2 Problem Statement 12

1.3 Aim and Objectives 12

1.4 Scope of the Project 13

1.5 Significance of the Project 13

1.6 Project Methodology (Brief Overview) 14

1.7 Project Organization 14

Chapter 2: Literature Review *16*

2.1 Introduction 16

2.2 Indoor Navigation 16

2.2.1 Mapping With SLAM 16

2.2.1.1 Evaluating the LidarView and MATLAB Algorithmic Approaches 17

2.2.1.2 Scan Matching Algorithms 17

2.2.1.3 Loop Closure Detection 18

2.2.1.4 Map Generation 18

2.2.2 Localization and Navigation With SLAM 19

2.2.2.1 Multi-Modal Navigation With ORB-SLAM 19

2.2.2.2 Indoor Localization via IMU-LiDAR Sensor Fusion 19

2.2.2.3 GraphSLAM, A Batch-Processing Solution to the Navigation Problem 20

2.2.2.4 2D LiDAR for Indoor Localization 20

2.2.2.5 3D LiDAR for Indoor Localization 20

2.2.3 Integrating SLAM with ROS: A Framework for Robotic Mapping and Navigation 21

2.2.3.1 ROS as a Modular Framework for SLAM Development and Integration 21

2.2.3.2 Comparative Strengths of ROS-Based SLAM Algorithms 21

2.2.3.3 Sensor Fusion and Data Processing for Indoor SLAM in ROS 21

2.2.3.4 Performance Evaluation and Algorithm Selection for Specific Use Cases 22

2.2.4 SLAM and Mapping for Smart Shopping Cart Navigation 22

2.2.5 Ways Other Than SLAM for Indoor Navigation 23

2.2.5.1 Localization Using ArUco Markers 23

2.2.5.2 Mapping and Localization Using Video Footages 23

2.3 Product scanning and detection 23
2.3.1 Vision-Based Detection Systems 23
2.3.1.1 Implementations and Performance 24
2.3.1.2 Model Selection and Architecture 24
2.3.1.3 Dataset Curation and Preparation 24
2.3.1.4 Training Procedure 24
2.3.1.5 Camera Configuration and Deployment 25
2.3.2 Sensor-Fusion Approaches 25
2.3.2.1 Vision-Weight Fusion for Product Verification 25
2.3.2.2 Multi-Modal Fusion for Action Verification 26
2.3.2.3 RFID-Weight Fusion for Security 26
2.3.2.4 Decoupled Fusion for Tracking and Navigation 26
2.3.3 Barcode and RFID-Based Scanning Systems 26
2.3.3.1 Standalone Barcode Scanning Architectures 27
2.3.3.2 Augmented Barcode Scanning with Fallback Mechanisms 27
2.3.3.3 Radio Frequency Identification (RFID) Systems 27
2.3.3.4 Integration with Broader AI-Driven Retail Ecosystems 28
2.3.4 RFID-Based Identification Systems 28
2.3.4.1 Core RFID Scanning and Billing Architecture 28
2.3.4.2 Security-Oriented RFID Designs 28
2.3.4.3 Low-Power and Networked Implementations 29
2.3.4.4 Enhanced User-Centric Features 29
2.3.4.5 Limitations and Adoption Barriers 29
2.4 WebApp 30
2.4.1 Navigation Assistance 30
2.4.1.1 Virtual Store Layout 30
2.4.1.2 Route Guidance 30
2.4.2 Real-Time Information 30
2.4.2.1 Product Details Display 31
2.4.2.2 Inventory Status 31
2.4.3 Budget Management 31
2.4.3.1 Live Cost Tracking 31

2.4.3.2 Budget Limit Alerts 31
2.4.4 Personalized Recommendations 32
2.4.4.1 AI-Driven Suggestions 32
2.4.4.2 Adaptive Learning 32
2.4.5 User Interface and Experience 32
2.4.5.1 Real-Time Data Feed 32
2.4.5.2 Enhanced Experience 32
2.4.6 Communication Role of the Application 33
2.4.6.1 Data Exchange with the Cart 33
2.5 Self-Checkout 34
2.5.1 Automated Product Verification and Receipt Generation 34
2.5.1.1 Real-Time Cart Synchronization and Bill Updates 34
2.5.1.2 Multi-Sensor Validation for Accurate Billing 34
2.5.2 Secure and Flexible Payment Processing 34
2.5.2.1 Integration with Digital Payment Gateways 34
2.5.2.2 Multi-Modal Payment Initiation Interfaces 35
2.5.2.3 Facial Recognition Payment (FacePay) 35
2.5.2.4 Final Transaction Verification and Security 35
2.5.3 Customer Journey Completion and Post-Checkout 36
2.5.3.1 Post-Payment Cart Reset and Receipt Finalization 36
2.5.3.2 Autonomous Cart Management for Fleet Efficiency 36
2.6 Hardware Components 37
2.6.1 Processing Units 37
2.6.1.1 Heterogeneous Dual-Processor Architecture for Navigation 37
2.6.1.2 Central Single-Board Computers (SBCs) for General-Purpose Control 37
2.6.1.3 Dedicated Microcontrollers for Subsystem Management 37
2.6.1.4 Advanced Edge AI Processors 38
2.6.1.5 Edge-Cloud Processing Split 38
2.6.2 Sensing & Scanning Modules 38
2.6.2.1 Navigation, Tracking & Obstacle Avoidance Suite 38
2.6.2.2 Product Identification & Billing Sensors 39
2.6.2.3 Radio-Frequency Identification (RFID) Systems 39

2.6.2.4 Payment & Infrastructure Integration 39
2.6.3 Actuation & Mobility 40
2.6.3.1 Manual Control Interface 40
2.6.3.2 Automated Cart Access & Security Mechanism 40
2.6.3.3 Automated Item Retrieval & Conveyor Systems 40
2.6.4 Communication Modules 40
2.6.4.1 Primary Connectivity & Data Exchange 41
2.6.4.2 Communication Protocols & Software Architecture 41
2.6.4.3 Security Hardware & Protocols 41
2.6.4.4 Peripheral & Subsystem Communication Interfaces 42
2.6.4.5 System Integration & Final Verification Points 42
2.6.5 Power Systems 42
2.6.6 Billing & User Interface Modules 42
2.6.6.1 Transaction Processing Hardware 43
2.6.6.2 Visual Display Interfaces 43
2.6.6.3 User Input & Control Hardware 43
2.6.6.4 Feedback & Interaction Hardware 43
2.6.6.5 Payment & Authentication Hardware 43
2.6.7 Store Integration & Enabling Hardware 44
Chapter 3: System Analysis and Design 49
3.1 System Overview 49
3.2 Functional Requirements 49
3.2.1 Log in (User) 49
3.2.2 Logout (User) 49
3.2.3 Create List (User, Guest) 49
3.2.4 Delete List (User) 50
3.2.5 Add Payment Method (User, Guest) 50
3.2.6 Editing Store Map (Admin) 50
3.2.7 Generating QR Code (Smart Shopping Cart) 50
3.2.8 Navigation (Smart Shopping Cart) 50
3.2.9 Recommendation (Smart Shopping Cart) 50
3.2.10 Checkout (Smart Shopping Cart) 50

3.2.11 Product Scanning (AI Model) 51
3.2.12 Adding/Removing Products (AI Model) 51
3.3 Non-Functional Requirements 51
3.3.1 Performance Requirements 51
3.3.2 Reliability Requirements 51
3.3.3 Security Requirements 51
3.3.4 Usability Requirements 51
3.3.5 Compatibility Requirements 51
3.3.6 Maintainability Requirements 51
3.4 System Environment 52
3.5 System Architecture 53
3.6 Class Diagram 54
3.7 Use Case Diagram 55
3.8 Sequence Diagram 56
3.8.1 Log-in 57
3.8.2 User Start Shopping 58
3.8.3 Cart navigation & Route Generation 58
3.8.4 Product Scanning & Virtual Receipt Update 59
3.8.5 Adding Payment Method 60
3.8.6 Finish Shopping & Payment Flow 61
3.9 ERD 62
3.10 Flowchart 63
Chapter 4: System Design 66
4.1 Introduction 66
4.2 Overall Thesis-Level Architecture 66
4.3 Map Generation Design 67
4.3.1 LiDAR Data Model and ROS Topic Design 67
4.3.2 Occupancy Grid Output and PGM Input Format 67
4.3.3 PGM-to-Shopper-Friendly Map Conversion Design 68
4.4 Web Application Design 70
4.4.1 Web Application Pages and User Journey 70
4.4.2 Web Application Screenshots and Functionality 70

4.4.3 Backend API and Data Model Design 73
4.5 Cart Screen, QR Pairing, and Integration Design 74
4.5.1 Cart Screen State Design 74
4.5.2 QR Pairing Contract 76
4.5.3 Device API and Polling Integration 76
4.6 AI Product Recognition Design 77
4.6.1 AI Problem Definition and Design Challenges 77
4.6.2 Dataset and Product Class Design 77
4.6.3 YOLO11m Model Architecture 78
4.6.4 Prediction Workflow and Event Generation 79
4.6.5 AI Backend API Design 80
4.7 Integrated Database and End-to-End Data Flow 80
Chapter 5: System Implementation *82*
5.1 Introduction 82
5.2 Map Generation Implementation 82
5.2.1 Custom LiDAR Decoder Implementation 82
5.2.2 ROS 2 Mapping Runtime 83
5.2.3 PGM Conversion Implementation 83
5.3 Web Application Implementation 84
5.3.1 Repository Structure 84
5.3.2 Authentication, Guest Mode, and List CRUD 84
5.3.3 Session, Receipt, Checkout, and Admin Implementation 84
5.4 Cart Screen and QR Integration Implementation 85
5.4.1 Cart Screen Repository Structure and Modes 85
5.4.2 Device Endpoints and Runtime Synchronisation 85
5.5 AI Product Recognition Implementation 86
5.5.1 Data Collection, Cleaning, and Splitting 86
5.5.2 Augmentation and Training Configuration 86
5.5.3 Tracking, Hand Detection, and Event State Machine 87
5.5.4 AI Service Deployment 88
5.6 End-to-End Integration Implementation 89
Chapter 6: System Testing and Results *91*

6.1 Introduction 91
6.2 Testing Strategy 91
6.3 Map Generation Testing and Results 91
6.3.1 PGM Conversion Test 91
6.3.2 Discussion of Mapping Outcomes 93
6.4 Web Application Testing and Results 93
6.5 Cart Screen, QR Pairing, and Integration Testing 94
6.6 AI Product Recognition Testing and Results 94
6.6.1 Controlled Experiments 94
6.6.2 Final Model Evaluation 97
6.6.3 Curve, Confusion Matrix, and Qualitative Results 97
6.6.4 Performance and Deployment Results 100
6.7 End-to-End System Testing Discussion 101
Chapter 7: Conclusions and Future Work 103
7.1 Introduction 103
7.2 Project Summary 103
7.3 Project Contributions 103
7.3.1 Map Generation Contribution 103
7.3.2 Web Application Contribution 103
7.3.3 Cart Screen and QR Integration Contribution 103
7.3.4 AI Product Recognition Contribution 104
7.4 Current Limitations 104
7.5 Future Work Roadmap 104
7.6 Final Conclusion 105
References: 106

List of Figures

FIGURE 2.1: POINT CLOUD MAP [1]. 18
FIGURE 2.2: OCTREE MAP [1]. 18
FIGURE 2.3: SEMANTIC MAP [1]. 19
FIGURE 2.4: ORB-SLAM3 AND MU-SLAM PERFORMANCE [20]. 22
FIGURE 3.1: SYSTEM ENVIRONMENT. 53
FIGURE 3.2: SYSTEM ARCHITECTURE. 54
FIGURE 3.3: CLASS DIAGRAM. 55
FIGURE 3.4: USE CASE DIAGRAM. 56
FIGURE 3.5: LOG-IN SEQUENCE DIAGRAM. 57
FIGURE 3.6: USER START SHOPPING SEQUENCE DIAGRAM. 58
FIGURE 3.7: CART NAVIGATION AND ROUTE GENERATION SEQUENCE DIAGRAM. 59
FIGURE 3.8: PRODUCT SCANNING AND VIRTUAL RECEIPT UPDATE SEQUENCE DIAGRAM. 60
FIGURE 3.9: ADDING PAYMENT METHOD SEQUENCE DIAGRAM. 61
FIGURE 3.10: FINISH SHOPPING AND PAYMENT FLOW SEQUENCE DIAGRAM. 62
FIGURE 3.11: ERD. 63
FIGURE 3.12: FLOWCHART. 64
FIGURE 4.1: CARTO THESIS-LEVEL SYSTEM DECOMPOSITION. 66
FIGURE 4.2: PGM-TO-SHOPPER-FRIENDLY MAP CONVERSION WORKFLOW. 68
FIGURE 4.3: RAW PGM OCCUPANCY GRID USED AS THE MAP GENERATION TEST INPUT. 69
FIGURE 4.4: SHOPPER-FRIENDLY MAP GENERATED FROM THE PGM TEST MAP. 69
FIGURE 4.5: WEB APPLICATION SIGN-IN AND GUEST ENTRY SCREEN. 70
FIGURE 4.6: WEB APPLICATION DASHBOARD SCREEN. 71
FIGURE 4.7: SHOPPING LIST MANAGEMENT SCREEN. 71
FIGURE 4.8: CART QR PAIRING SCREEN. 72
FIGURE 4.9: ACTIVE SHOPPING SESSION AND VIRTUAL RECEIPT SCREEN. 72
FIGURE 4.10: ADMIN DASHBOARD SCREEN. 73
FIGURE 4.11: CART SCREEN WAITING AND QR STATE. 74
FIGURE 4.12: BUNDLED CART SCREEN STATIC STORE MAP. 75
FIGURE 4.13: QR PAIRING AND CART-WEBAPP INTEGRATION FLOW. 76
FIGURE 4.14: YOLO11M-BASED AI MODEL ARCHITECTURE AND EVENT LAYER. 78
FIGURE 4.15: AI INPUT/OUTPUT AND PREDICTION WORKFLOW. 79
FIGURE 5.1: LIDAR DECODING AND ROS MAPPING RUNTIME FLOW. 83
FIGURE 5.2: CART SCREEN INTERFACE WITH SHOPPING LIST, MAP, AND RECEIPT PANELS. 85
FIGURE 5.3: AI BACKEND LAYERED ARCHITECTURE. 88
FIGURE 5.4: END-TO-END INTEGRATED SHOPPING FLOW. 89
FIGURE 6.1: MAP GENERATION TEST - RAW PGM OCCUPANCY GRID. 92
FIGURE 6.2: MAP GENERATION TEST - SHOPPER-FRIENDLY OUTPUT. 92
FIGURE 6.3: CLASS DISTRIBUTION ACROSS THE CONSOLIDATED AI PRODUCT DATASET. 95
FIGURE 6.4: YOLOV8M BASELINE TRAINING CURVES. 95
FIGURE 6.5: YOLO11M TRAINING AND VALIDATION CURVES. 96
FIGURE 6.6: TRAINING BATCH MOSAICS WITH GROUND-TRUTH BOXES. 96
FIGURE 6.7: PRECISION-RECALL AND F1-CONFIDENCE CURVES. 97
FIGURE 6.8: PRECISION AND RECALL CONFIDENCE CURVES. 98
FIGURE 6.9: AI PRODUCT RECOGNITION CONFUSION MATRIX. 98
FIGURE 6.10: GROUND TRUTH AND YOLO11M PREDICTIONS ON VALIDATION BATCH 0. 99
FIGURE 6.11: GROUND TRUTH AND YOLO11M PREDICTIONS ON VALIDATION BATCH 1. 99
FIGURE 6.12: GROUND TRUTH AND YOLO11M PREDICTIONS ON VALIDATION BATCH 2. 100

List of Tables

TABLE 2.1: FEATURES OF SCAN MATCHING METHODS [1]. 17
TABLE 2.2: PROCESSING UNITS SYNTHESIS. 44
TABLE 2.3: SENSING AND SCANNING SYNTHESIS. 45
TABLE 2.4: ACTUATION AND MOBILITY SYNTHESIS. 45
TABLE 2.5: BILLING AND USER INTERFACE SYNTHESIS. 46
TABLE 2.6: COMMUNICATION SYNTHESIS. 47
TABLE 2.7: POWER AND INFRASTRUCTURE SYNTHESIS. 47
TABLE 2.8: MECHANICAL AND SECURITY SYNTHESIS. 47
TABLE 4.1: TECHNOLOGY STACK BY SYSTEM COMPONENT. 67
TABLE 4.2: MAIN DATABASE ENTITIES AND RESPONSIBILITIES. 73
TABLE 4.3: QR PAIRING CONTRACT AND SECURITY RULES. 76
TABLE 4.4: AI PRODUCT RECOGNITION OBJECTIVES AND SUCCESS CRITERIA. 77
TABLE 4.5: COMPLETE LIST OF PRODUCT CLASSES USED BY THE AI MODEL. 77
TABLE 4.6: YOLO11M MODEL STRUCTURE SUMMARY. 78
TABLE 4.7: AI BACKEND ENDPOINT REFERENCE. 80
TABLE 5.1: SOURCE REPOSITORY STRUCTURE BY COMPONENT. 82
TABLE 5.2: ROS MAPPING IMPLEMENTATION FILES AND CONFIGURATION. 83
TABLE 5.3: WEB APPLICATION PAGES, APIS, AND RESPONSIBILITIES. 84
TABLE 5.4: CART SCREEN RUNTIME MODES AND DEVICE ENDPOINTS. 85
TABLE 5.5: DATASET PARTITION STATISTICS AFTER CONSOLIDATION AND CLEANING. 86
TABLE 5.6: DATA AUGMENTATION CONFIGURATION AND TECHNICAL RATIONALE. 86
TABLE 5.7: YOLO11M TRAINING CONFIGURATION. 87
TABLE 5.8: MODEL EXPORT FORMAT COMPARISON. 88
TABLE 6.1: MAP CONVERSION TEST CASE AND OUTCOME. 91
TABLE 6.2: WEB APPLICATION TEST CASES. 93
TABLE 6.3: CART SCREEN AND QR INTEGRATION TEST CASES. 94
TABLE 6.4: COMPLETE AI EXPERIMENT LOG. 94
TABLE 6.5: FINAL YOLO11M EVALUATION METRICS ON THE HELD-OUT TEST SET. 97
TABLE 6.6: AI RESOURCE UTILISATION UNDER SUSTAINED 30-FPS INPUT LOAD. 100
TABLE 6.7: END-TO-END SYSTEM INTEGRATION TEST RESULTS. 101
TABLE 7.1: YOLOV8M AND YOLO11M COMPARISON. 104
TABLE 7.2: CURRENT LIMITATIONS AND FUTURE WORK ROADMAP. 104

Carto

CHAPTER 1

Introduction

Chapter 1: Introduction

1.1 Background of the Study

Traditional shopping carts are widely used in supermarkets, but they still depend mainly on manual interaction from shoppers. Customers usually need to prepare their shopping lists separately, search for products inside the store, manually track what they have placed in the cart, and wait at the checkout counter to complete payment. This traditional process can be inefficient, especially in large supermarkets where products are distributed across many aisles and sections.

Recent developments in Internet of Things (IoT), computer vision, artificial intelligence, and indoor mapping technologies have created new opportunities for improving the retail shopping experience. These technologies allow physical shopping carts to become smarter and more interactive by connecting software systems, sensors, cameras, and user interfaces. Based on this idea, this project proposes an AI-powered smart shopping cart called Carto, which aims to improve user convenience, reduce shopping time, support product recognition, and increase supermarket operational efficiency.

1.2 Problem Statement

Shoppers often struggle with locating items, managing lists, and calculating expenses in real-time. Manual shopping is time-consuming and prone to errors. Existing smart cart solutions lack integrated navigation, real-time product recognition, and personalized recommendations.

1.3 Aim and Objectives

The main aim of this project is to design and develop an intelligent shopping cart system that automates item tracking, supports navigation, and improves the checkout process. The system is intended to provide a more efficient and interactive shopping experience by connecting the user's prepared shopping list with the smart cart used inside the supermarket.

To achieve this aim, the project focuses on developing a web application that allows users to create and manage shopping lists before visiting the supermarket. The project also implements QR-based pairing so that the user's saved shopping list can be transferred from the web application to the smart cart screen. In addition, the system integrates SLAM-based map generation and shortest-path planning to help guide the shopper inside the store. Another important objective is to develop an AI-based product recognition system using onboard cameras to identify products placed inside the cart. The project also aims to update the receipt in real time, support product recommendations based on the shopping list and nearby product locations, and provide a smoother checkout and payment experience.

1.4 Scope of the Project

The scope of this project covers the design and development of a smart shopping cart prototype that combines hardware, software, mapping, artificial intelligence, and user-interface components to improve the supermarket shopping experience. The hardware scope includes a Raspberry Pi as the main processing unit responsible for running and coordinating the cart-side software, handling communication with the web application, managing the cart screen interface, and supporting integration with the connected modules. The system also includes a LiDAR sensor used for scanning the surrounding environment and generating a map of the supermarket layout. In addition, the smart cart includes a touchscreen display for user interaction, QR code pairing, shopping-list viewing, map display, receipt tracking, and checkout guidance. Cameras are also included to support AI-based product recognition inside the cart.

The software scope includes the development of a web application that allows users to create and manage shopping lists before using the smart cart. It also includes the cart screen application, which receives the user's shopping list through QR-based pairing and displays the active shopping session, supermarket map, selected products, recommendations, and real-time receipt. The mapping part of the project focuses on using LiDAR data to generate a store map and convert the raw map output into a shopper-friendly representation that can be displayed clearly on the cart screen. The AI component focuses on recognizing products placed inside the cart using camera input and updating the receipt automatically based on the detected items.

The project also covers the integration between the web application, backend system, cart screen, AI product recognition service, and map generation module. This integration allows the user's shopping list to be transferred to the cart, products to be recognized and added to the receipt, and the cart interface to update during the shopping process. However, the project does not include full supermarket inventory management, stock control, supplier management, or large-scale commercial payment deployment. The system depends on the accuracy of the supermarket product database, the quality of the generated map, and the reliability of the hardware components such as the Raspberry Pi, LiDAR sensor, cameras, and touchscreen.

1.5 Significance of the Project

This project is significant because it demonstrates how automation and artificial intelligence can be applied to improve the traditional supermarket shopping experience. By allowing users to create shopping lists, pair them with a cart, view a store map, recognize products automatically, and track the receipt in real time, the system reduces the effort required from shoppers and makes the shopping journey more organized.

The project also helps reduce checkout time and minimizes human error in product tracking and receipt calculation. From the supermarket's perspective, such a system can provide useful insights about shopping behavior, product selection, and customer movement patterns. In addition, the project promotes the use of AI, IoT, and computer vision in the retail field, showing how these technologies can be combined to create practical smart retail solutions.

1.6 Project Methodology (Brief Overview)

The project methodology followed a structured development process beginning with requirements gathering, where the needs of shoppers and supermarkets were studied to define the main system features. After that, the system design phase focused on planning the architecture of the hardware and software components, including the web application, cart screen, AI model, mapping system, and integration workflow.

The development phase involved building the main software components, including the web application for shopping-list management, the AI product recognition model, and the navigation and map generation module. The integration phase then connected these components together so that the web application, QR pairing process, cart screen, product recognition system, and receipt update workflow could operate as one system. Finally, the system was tested through functional and integration testing to evaluate whether the main features worked correctly. A pilot deployment in a supermarket environment is considered as a future step for validating the system in real-world conditions.

1.7 Project Organization

This report is organized into seven chapters. Chapter 1 introduces the project by presenting the background, problem statement, aim, objectives, scope, significance, methodology, and report structure. Chapter 2 reviews related works and previous research in indoor navigation, SLAM mapping, product detection, web applications, self-checkout systems, and smart cart technologies. Chapter 3 presents the system analysis and design, including functional requirements, non-functional requirements, system environment, architecture, diagrams, and data model.

Chapter 4 explains the detailed system design, including map generation, web application design, cart screen and QR pairing design, AI product recognition design, and the integrated database and data flow. Chapter 5 discusses the implementation of the main system components, including the mapping module, web application, cart screen, QR integration, AI model, and end-to-end system integration. Chapter 6 presents the testing process and results for the main features of the project, including map generation, web application testing, cart screen and QR integration testing, AI recognition testing, and overall system testing. Finally, Chapter 7 concludes the thesis by summarizing the project contributions, discussing the current limitations, and presenting possible future work.

CHAPTER 2

Literature Review

Chapter 2: Literature Review

2.1 Introduction

The development of intelligent shopping systems represents a significant frontier in retail technology innovation. Previous research has explored various approaches to modernize the traditional shopping experience, focusing primarily on three key areas: automated product identification, indoor navigation systems, and user-centric design frameworks.

Several studies have established foundational methodologies in these domains. Research in automated billing systems has demonstrated the superiority of Radio Frequency Identification (RFID) technology over traditional barcode scanning, highlighting advantages in scanning range, simultaneous item detection, and environmental robustness. Concurrently, computer vision approaches utilizing YOLO (You Only Look Once) object detection architectures have shown promise in visual product recognition, offering real-time identification capabilities. Advancements in indoor positioning systems have utilized Simultaneous Localization and Mapping (SLAM) techniques with LiDAR sensors to achieve sub-meter accuracy in complex retail environments. In the realm of path planning, classical algorithms like A* have been validated as optimal solutions for grid-based navigation in structured settings like supermarkets.

User experience research has further informed the design of interactive shopping interfaces, emphasizing the importance of mobile application integration, real-time feedback mechanisms, and seamless payment processing. These collective works provide a comprehensive technological foundation and validate key design principles that directly inform the development of our integrated smart shopping cart system.

The following review synthesizes these critical contributions, examining their methodologies, findings, and limitations to establish the theoretical and practical framework for our proposed solution.

2.2 Indoor Navigation

Research in indoor positioning has established various methods for real-time location tracking and user guidance within enclosed spaces. These approaches focus on solving the challenges of accurate localization and optimal path planning in complex, dynamic environments like supermarkets, providing reliable foundations for automated navigation systems.

2.2.1 Mapping With SLAM

Simultaneous Localization and Mapping (SLAM) enables mobile robots to navigate unfamiliar environments by constructing a map while tracking their own position. LiDAR (Light Detection and Ranging) is a key sensor in SLAM systems, using laser beams to measure distances and generate precise 3D point clouds of surroundings. Unlike cameras, LiDAR works reliably in varying lighting and texture-poor spaces, making it ideal for indoor applications like smart shopping carts, where real-time mapping and obstacle detection are essential for autonomous navigation [1].

2.2.1.1 Evaluating the LidarView and MATLAB Algorithmic Approaches

LidarView SLAM is an open-source algorithm integrated into Kitware's LidarView software, inspired by the LOAM method and designed to process raw 3D LiDAR data by allowing parameter adaptation for different environments. In contrast, MATLAB's 3D LiDAR SLAM implementation utilizes functions from its Navigation and Computer Vision Toolboxes, following a workflow that includes point cloud preprocessing, registration using methods like the Normal Distributions Transform (NDT), loop closure detection, and pose graph optimization to correct drift and build a consistent map. Both approaches were tested in the study for indoor trajectory estimation, with LidarView producing smoother point clouds in certain sections and MATLAB showing slightly better accuracy in longer straight corridors, highlighting the trade-offs between algorithmic design and software-specific processing capabilities [8].

2.2.1.2 Scan Matching Algorithms

The performance of the scan matching algorithm is crucial, as it directly determines the accuracy of both trajectory estimation and map construction in SLAM.

Scan matching algorithms can be broadly categorized according to their methodology, falling into three main groups: iterative closest point (ICP)-based algorithms, geometric feature-based algorithms, and mathematical feature-based algorithms [1].

This table summarizes the features of the scan matching methods and their differences

Features	ICP-Type Methods	Feature-Based Methods	NDT-Type Methods
Iteration	Need	Optional	Need
Initial value	Need	No need	Need
Domain of convergence	Small	Large	Medium
Operating speed	Slow	Fast	Medium
Robustness	Poor	Medium	Good
Precision	High, Affected by outliers and noise	Low, Related to feature extraction accuracy	Medium, Related to voxel size
Scope of application	Wide	Structured scene	Wide

Table 2.1: Features of Scan Matching Methods [1].

2.2.1.3 Loop Closure Detection

SLAM systems contend with computational errors arising from two primary sources: local errors from front-end scan matching and global drift from back-end map optimization, both stemming from inherent sensor noise and environmental deviations. Consequently, the effective handling of loop closures becomes critical for long-term operation [1]. These closures occur when a robot revisits a known location, requiring the system to detect the event and correct the accumulated global pose and map errors. This presents a significant challenge, as detection must remain reliable across changes in viewpoint, lighting, and dynamic scene elements [2].

2.2.1.4 Map Generation

The map generated by a LiDAR SLAM algorithm models the robot's environment and serves as the essential foundation for localization and autonomous navigation. Within SLAM, common map representations primarily include point clouds, octrees, and semantic maps [1]. The point cloud map is a collection of three-dimensional space points obtained by sensors such as lidar. An octree map is an efficient spatial data structure that recursively subdivides 3D space into cubic voxels. Each node represents a region and stores its state, typically whether it is occupied or free. While this hierarchical approach compresses map storage significantly, it involves a more complex initial construction process. The semantic map contains not only the structural information of the environment but also the semantic information of the objects in the scene (such as object category, function, state, etc.). Semantic maps can provide rich environmental information, but data acquisition is complex and computing resources are consumed [1].

These are map examples:

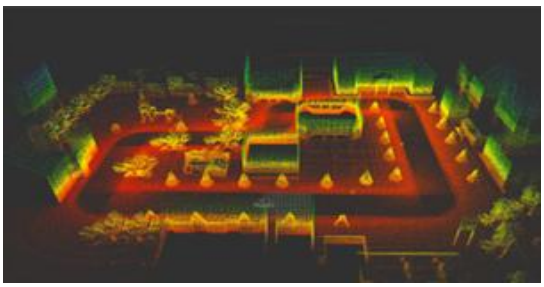


Figure 2.1: Point Cloud Map [1].

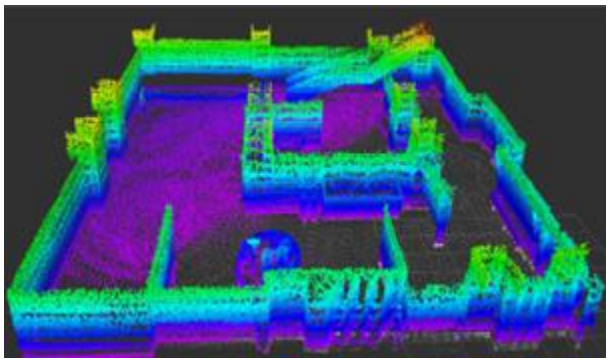


Figure 2.2: Octree Map [1].

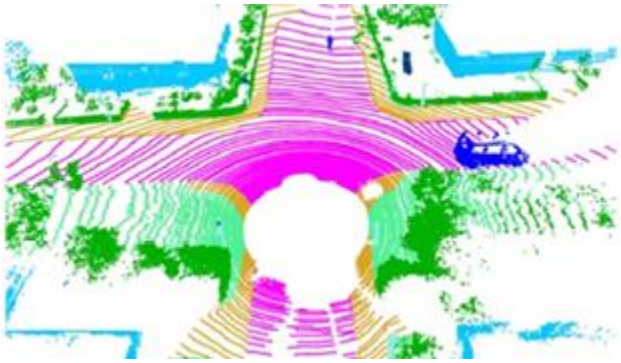


Figure 2.3: Semantic Map [1].

2.2.2 Localization and Navigation With SLAM

After a map is constructed, the core challenges shift to real-time localization (determining the robot's precise pose within the map) and navigation (planning and executing safe paths to a goal). The solutions to these challenges vary significantly based on a robot's primary sensor suite—whether it relies on LiDAR, cameras, or other sensing modalities—each offering distinct trade-offs in robustness, precision, and computational cost.

2.2.2.1 Multi-Modal Navigation With ORB-SLAM

ORB-SLAM provides a flexible SLAM framework that supports monocular, stereo, and RGB-D camera modalities. The inclusion of RGB-D sensing, in particular, allows robots to navigate reliably in environments with challenging or variable lighting. This versatility enables the system to be deployed across a wide range of scales and settings, from small indoor scenes to large-scale outdoor environments [2].

2.2.2.2 Indoor Localization via IMU-LiDAR Sensor Fusion

In indoor environments where GPS is unavailable, IMU and LiDAR are tightly coupled using an Error-State Kalman Filter (ESKF) for real-time localization: the IMU provides high-frequency (1000 Hz) motion predictions (acceleration and rotation) between LiDAR scans, while the LiDAR (10 Hz) captures 3D point clouds of the indoor structure (walls, floors, corridors). The LiDAR data is used to correct IMU drift by matching current scans to a pre-built or concurrently built map (scan-to-map matching), extracting plane features (e.g., walls) to constrain motion estimates. This fusion allows the system to maintain high accuracy (≤0.05 m error) and real-time performance even in feature-sparse or repetitive indoor spaces like tunnels, warehouses, or long hallways, effectively eliminating cumulative errors and enabling reliable indoor navigation [3].

2.2.2.3 GraphSLAM, A Batch-Processing Solution to the Navigation Problem

GraphSLAM is a leading algorithm for resolving the full navigation and mapping problem. It addresses the offline nature of the SLAM problem by accumulating all sensor data and robot poses during operation, then formulating them as constraints in a graph once the robot's activity is complete. GraphSLAM first constructs a dense graph of these constraints, which is then linearized (e.g., using a Taylor expansion) to obtain a manageable mathematical description. The algorithm employs variable elimination—marginalizing out the robot's pose parameters—to simplify the resulting information structure, thereby converting the problem into an optimization over the map features. Finally, a standard optimization technique, such as the conjugate gradient method, is used to solve this problem. GraphSLAM reconstructs the map by solving a series of smaller optimization sub-problems, one for each feature, using the estimated robot trajectory [2].

2.2.2.4 2D LiDAR for Indoor Localization

2D LiDAR-based indoor localization primarily relies on planar scanning to capture cross-sectional contours of the environment, which is particularly effective in structured indoor spaces such as corridors and rooms [6]. These systems often employ scan-matching algorithms like Iterative Closest Point (ICP) or Normal Distributions Transform (NDT) to align successive LiDAR scans and estimate robot motion [6]. Popular SLAM frameworks such as GMapping and Cartographer use probabilistic filtering or graph optimization to build consistent 2D occupancy grid maps in real-time, enabling robots to localize themselves within previously mapped areas [6]. For enhanced robustness, some systems integrate 2D LiDAR with inertial measurement units (IMUs) to mitigate drift in featureless environments [7]. Additionally, spinning-actuated 2D LiDAR configurations can be used to create 3D-like maps by rotating the sensor, expanding the field of view and improving localization in multi-level indoor environments [7]. However, 2D LiDAR systems are limited in vertical perception and may struggle in highly dynamic or cluttered spaces [6].

2.2.2.5 3D LiDAR for Indoor Localization

3D LiDAR-based indoor localization captures rich, three-dimensional point cloud data of the environment, enabling precise six-degree-of-freedom pose estimation in complex indoor spaces such as multi-story buildings, atriums, and warehouses [6]. These systems extract geometric features like edges and planes from point clouds and use registration methods such as LOAM (Lidar Odometry and Mapping) to perform real-time odometry and mapping [6]. Tightly-coupled fusion with IMU data through filters like the Error-State Kalman Filter (ESKF) or graph optimization frameworks significantly reduces drift and improves accuracy in feature-sparse or dynamic indoor areas [7]. Advanced 3D LiDAR SLAM systems, such as LIO-SAM and FAST-LIO, support both mechanical and solid-state LiDARs, offering robustness in varying lighting conditions and unstructured environments [6]. Graph optimization back-ends further refine pose estimates by incorporating loop closure detection and multi-sensor constraints, ensuring globally consistent 3D maps [6]. However, challenges remain in degraded environments—like long, repetitive corridors—where geometric features are scarce, necessitating additional sensor fusion or semantic augmentation for reliable localization [6].

2.2.3 Integrating SLAM with ROS: A Framework for Robotic Mapping and Navigation

The Robot Operating System (ROS) provides a modular and efficient platform for developing SLAM systems, enabling seamless integration of sensors, algorithms, and simulation tools. By supporting widely-used packages like GMapping, HectorSLAM, and Cartographer, ROS allows for rapid prototyping and evaluation of mapping solutions—especially in GPS-denied indoor environments. This flexibility makes ROS essential for implementing robust and adaptable navigation systems across various robotic applications.

2.2.3.1 ROS as a Modular Framework for SLAM Development and Integration

ROS provides a flexible and modular framework for implementing and testing SLAM algorithms, significantly reducing development time and enabling rapid prototyping in both simulated and real-world robotic applications [28]. The ROS ecosystem includes essential tools like Gazebo for 3D simulation, Rviz for data visualization, and pre-built packages such as GMapping, HectorSLAM, and Cartographer, which allow researchers to focus on algorithm integration and performance evaluation rather than low-level coding [28]. This modularity supports the seamless integration of various sensors—such as LiDAR, RGB-D cameras (e.g., Asus Xtion Pro), and IMUs—into the SLAM pipeline, facilitating sensor fusion and robust indoor mapping [5].

2.2.3.2 Comparative Strengths of ROS-Based SLAM Algorithms

Several widely-used SLAM algorithms are available as ROS packages, each with distinct strengths and suitability for different operational conditions [27]. GMapping relies on Rao-Blackwellized particle filters and performs best with reliable odometry data, making it ideal for wheeled robots but less effective in odometry-free handheld systems [27]. HectorSLAM uses scan matching and an Extended Kalman Filter, functioning without odometry but requiring high-quality, low-noise LiDAR data [27]. Cartographer, a graph-based SLAM system, excels in odometry-free and handheld applications due to its submap-based approach and robust loop closure detection, consistently producing accurate 2D maps even under unpredictable human motion [27].

2.2.3.3 Sensor Fusion and Data Processing for Indoor SLAM in ROS

Sensor integration and data processing within ROS are critical for achieving accurate localization and mapping, particularly in indoor environments where GPS is unavailable [5]. ROS nodes can process depth images from RGB-D cameras into 3D point clouds and subsequently into 2D laser scans using packages like depth_image_proc and pointcloud_to_laserscan, enabling devices like the Kinect or Asus Xtion Pro to function as pseudo-LiDAR sensors for cost-effective SLAM [28]. Additionally, odometry data from wheel encoders can be fused with visual or LiDAR data in packages like RTAB-Map to improve pose estimation and map consistency, supporting both 2D and 3D mapping workflows [5].

2.2.3.4 Performance Evaluation and Algorithm Selection for Specific Use Cases

Performance evaluation of ROS-based SLAM systems highlights the importance of selecting the appropriate algorithm based on sensor availability, environment type, and platform constraints [27]. In comparative studies, Cartographer demonstrated superior performance in handheld, odometry-free scenarios, achieving mapping errors as low as 0.017m in structured environments like offices and corridors, while maintaining robustness to sudden movements [27]. In contrast, GMapping often failed without reliable odometry, and HectorSLAM struggled with low-cost LiDARs and sharp turns [27]. These findings underscore the need for careful algorithm selection and parameter tuning within ROS to meet specific application requirements, such as those for first responders, service robots, or autonomous indoor vehicles [27, 28].

2.2.4 SLAM and Mapping for Smart Shopping Cart Navigation

The smart shopping cart's autonomous navigation is powered by MU-SLAM, a sensor-fusion system combining Mask R-CNN for dynamic object removal, UWB for absolute positioning, and ORB-SLAM3 for visual odometry. This integration allows the cart to filter out moving people in real-time, build a static map of the supermarket layout, and localize itself within it with high accuracy, enabling reliable path planning and obstacle avoidance [20].

In comparative tests, MU-SLAM demonstrated a 92% improvement in positioning accuracy over standard ORB-SLAM3 in dynamic indoor environments. Trajectory error was reduced from 0.88–1.28 meters to just 0.03–0.13 meters. The system effectively eliminated dynamic objects from the map, ensuring stable and precise localization even in crowded aisles, which is critical for tasks such as product fetching and customer following [20].

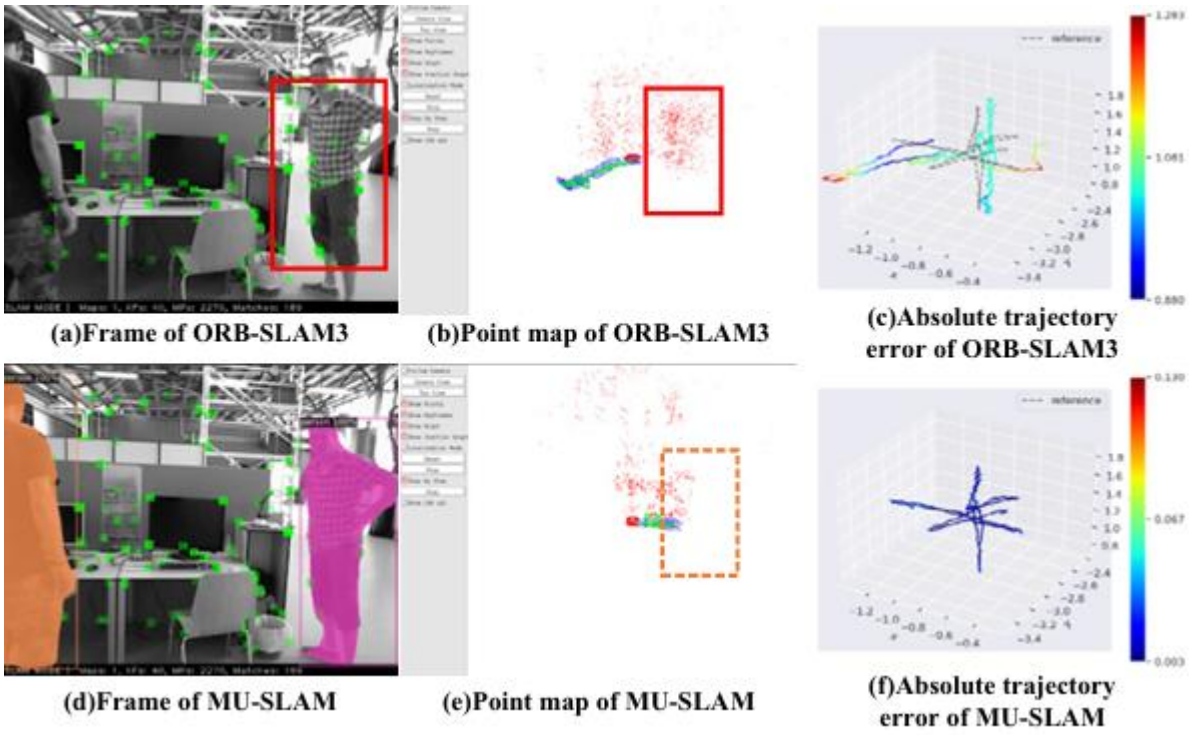


Figure 2.4: ORB-SLAM3 and MU-SLAM Performance [20].

The MU-SLAM module was deployed on a cart equipped with a Velodyne LiDAR and onboard computer. During field tests, the cart successfully constructed a real-time 3D map of the store and navigated accurately to product locations sent from the back-end system. This mapping and localization capability directly supported the cart's autonomous guidance function, contributing to a 33.15% reduction in average shopping time by streamlining in-store navigation [20].

2.2.5 Ways Other Than SLAM for Indoor Navigation

Some previous researches introduced ways other than SLAM for indoor navigation.

2.2.5.1 Localization Using ArUco Markers

The navigation module provides optimized guidance based on the user's precise location, which is continuously identified by detecting ArUco markers placed throughout the environment. Using the system's stereo cameras to recognize these fiducial markers determines the user's position on the supermarket map, while the specific camera that detects the marker (left or right) establishes their initial movement orientation [9].

2.2.5.2 Mapping and Localization Using Video Footages

video footages are collected from both real and simulated grocery store environments. A wide-angle camera, mounted at cart height, captured comprehensive views of aisles and product displays. This video data was then processed using computer vision methodologies to analyze critical aspects of the store's configuration, including aisle boundaries, product arrangements, and directional signage.

Using Image processing algorithms they transformed the recorded footage into a detailed environmental map. The mapping process involved three phases: image stabilization, feature extraction, and layout reconstruction. Techniques such as edge detection and contour mapping were employed to ensure the cart could accurately perceive its surroundings and determine optimal navigation paths. Furthermore, this navigation system was integrated with a pre-existing object detection module, allowing for real-time adjustments as the cart moves [10].

2.3 Product scanning and detection

2.3.1 Vision-Based Detection Systems

Vision-based systems for smart carts predominantly utilize deep learning object detection models to identify products directly from camera feeds. The following sections detail the key implementations, architectural choices, and methodologies found in the literature.

2.3.1.1 Implementations and Performance

Multiple YOLO (You Only Look Once) variants have been applied to product scanning in smart cart contexts with varying performance outcomes:

- YOLOv2 for Produce Detection: One implementation used YOLOv2 to detect unpackaged vegetables (e.g., potato, tomato, carrot), achieving 70% accuracy. This camera-only pipeline demonstrated feasibility for fresh produce recognition but was limited to static item placement and a small product catalog. A custom dataset annotated with bounding boxes was used for training [19].
- YOLOv3 for General Groceries: A more detailed implementation employed YOLOv3 trained on 2,800 annotated images across five grocery product classes. It achieved a high mAP of 82.28%, with per-class accuracy ranging from 70.59% (ramen) to 89.38% (Downy). The model was trained over 16,200 epochs to a final loss of 0.0616 and uses a confidence threshold of 0.6 during real-time inference on video streams [29].
- Mentioned Advanced Architectures: Another smart cart system cited the use of CBAM-YOLOv7 for product verification during checkout, though specific details on its performance, training, or camera setup were not provided [20].

2.3.1.2 Model Selection and Architecture

The YOLO family is favored for its balance of speed and accuracy, which is critical for real-time applications in dynamic retail environments.

- Rationale for YOLOv3: TechCart, a representative system, selected YOLOv3 over alternatives like Faster R-CNN due to its superior inference speed while maintaining strong accuracy[10].
- Detection Pipeline: The core architecture frames detection as a grid-based regression problem. An input image is divided into an S × S grid. Each grid cell predicts bounding boxes, confidence scores, and class probabilities, formulated as a tensor: S × S × (B×5 + C) [10, 19].

2.3.1.3 Dataset Curation and Preparation

Training robust models requires carefully prepared, annotated datasets.

- Dataset Used: TechCart's model was trained on the Freiburg Grocery Dataset, which contains 4,947 annotated images across 25 common grocery classes (e.g., beans, milk, pasta), with each class having at least 97 images [10].
- Data Preprocessing: The dataset, originally in Pascal VOC format, was converted to YOLOv3 format using the Roboflow platform and split into 80/20 training/validation sets. A custom Bash script (reorganization.sh) was used to restructure the data for compatibility with the Darknet implementation of YOLOv3 [10].

2.3.1.4 Training Procedure

Standard deep learning practices, including transfer learning, are employed to train the detection models efficiently.

- Transfer Learning: Models are typically initialized with pre-trained weights from ImageNet to enhance generalization and reduce required training time [10].
- Process & Monitoring: Training is conducted over thousands of iterations/epochs (e.g., 6,000 iterations in approximately 13 hours [10], or 16,200 epochs [29]) on GPU platforms like Google Colab. Hyperparameters are tuned per established guidelines, and performance is monitored on the validation set using metrics like precision, recall, and mean Average Precision (mAP) [10, 29].

2.3.1.5 Camera Configuration and Deployment

Hardware setup is designed to integrate detection with other cart functions.

- **Camera Setup: Systems often use a single wide-angle camera mounted at cart height, oriented to capture forward-facing views of store aisles and shelves. This camera serves a dual purpose: providing input for product detection and aiding in store layout mapping for navigation[10].**
- **Real-Time Processing: Camera feeds are processed in real-time by an onboard microcontroller unit (MCU). The vision system is frequently supplemented by ultrasonic and infrared sensors for obstacle avoidance and enhanced environmental awareness [10].**

2.3.2 Sensor-Fusion Approaches

To enhance reliability beyond the limitations of vision-only systems, various sensor-fusion architectures have been explored. These methods combine visual data with other sensory inputs—such as weight, motion, or RFID—to create more robust verification, tracking, and security systems for smart carts.

2.3.2.1 Vision-Weight Fusion for Product Verification

The fusion of camera-based recognition with weight sensors is a prominent approach for validating product identity, especially for variable-weight items.

- **YOLOv2 and Load Cell System: One implementation combines YOLOv2 object detection with a load cell (calibrated to 5 kg) for verification [19]. The process involves:**
  1. **The camera identifies the product class (e.g., tomato).**
  2. **An Arduino-triggered load cell measures the total weight of items from that class placed in the cart.**
  3. **The total cost is calculated by multiplying the measured weight by a predefined unit price.**
- **Advantages: This approach provides a tangible validation mechanism, reduces dependency on barcodes/RFID for perishables, and enables per-weight pricing for fruits and vegetables [19].**
- **Redundant Verification System: Another hybrid design uses a continuously monitoring load cell (10 kg capacity) alongside a primary identification camera [15]. When an item is added or removed, the weight change is cross-referenced with the visual scan to confirm the action. If the primary barcode scan fails, a machine learning vision model works in conjunction with weight data for product identification, enhancing reliability in dynamic environments [15].**

2.3.2.2 Multi-Modal Fusion for Action Verification

More comprehensive architectures integrate vision with motion-triggering sensors to create a complete verification pipeline for product addition/removal.

- YOLOv3 with Ultrasonic Trigger and Load Cell: A detailed system combines YOLOv3 object detection with an ultrasonic sensor array and a load cell [29]. The operational flow is:
    1. Ultrasonic sensors detect product movement near the cart.
    2. This triggers a 7-second video stream from a Pi Camera.
    3. YOLOv3 processes the video for product recognition.
    4. The load cell simultaneously measures weight change to cross-check and resolve ambiguous detection cases.
- System Architecture: This multi-modal pipeline operates via TCP/IP communication between the cart, a central server, and an Android app, demonstrating how augmenting vision with simple sensors improves reliability without requiring expensive RFID infrastructure [29].

2.3.2.3 RFID-Weight Fusion for Security

Some systems prioritize security and error prevention by fusing Radio-Frequency Identification (RFID) with weight verification.

- RFID and Load Cell with Camera Surveillance: One design integrates RFID identification with load cell weight verification [12]. The process is:
    1. An RFID reader scans pre-tagged items.
    2. The load cell measures the product's weight and cross-references it against a database entry.
    3. If a weight discrepancy is detected, a Pi Camera is activated for live surveillance (serving a security, not recognition, role).
- Limitations: This approach aims to prevent scanning errors or theft but requires every product to be pre-tagged and does not provide quantitative detection performance metrics [12].

2.3.2.4 Decoupled Fusion for Tracking and Navigation

Sensor fusion is also applied not for product recognition itself, but for complementary tasks like customer tracking and cart navigation.

- Task-Specific Sensor Assignment: Some architectures decouple the functions, using specific sensor suites for specific tasks [11]. For example:
    - RGB-D cameras, ultrasonic sensors, and infrared detectors are fused (using Kalman filters and re-identification models) to maintain robust customer tracking.
    - Barcode scanners handle product identification separately.
- Rationale: This approach assigns the right sensor to the right task rather than fusing all modalities directly for product recognition [11].

2.3.3 Barcode and RFID-Based Scanning Systems

Barcode and RFID scanning form the backbone of many smart cart systems, offering high-accuracy product identification by relying on pre-encoded labels or tags. These approaches are often enhanced with supplementary features like security, automated billing, and integration with broader retail architectures.

2.3.3.1 Standalone Barcode Scanning Architectures

These systems use dedicated hardware to scan barcodes, connecting to databases for product information and billing.

- **Core Implementation: A typical system uses a GM-73 barcode scanner and Raspberry Pi 4 to enable real-time product addition, expiry detection, and automated billing [22]. Product identification relies on encoded barcodes, ensuring high accuracy but requiring all items to be pre-tagged.**
- **User Interface and Security: These systems often feature a user-facing touchscreen LCD for displaying the running bill and product details [15, 23]. Some incorporate a security layer, such as making purchase histories visible to staff for theft prevention [22] or including a buzzer-based deterrent that triggers if an unscanned item is removed [18].**

2.3.3.2 Augmented Barcode Scanning with Fallback Mechanisms

To handle edge cases, some barcode systems are augmented with auxiliary sensors or fallback recognition methods.

- **Vision-Augmented Scanning: One system employs a Raspberry Pi Camera Module (8 MP) for real-time barcode scanning [15]. A buzzer provides audible confirmation upon a successful scan. Crucially, this design includes a fallback to a machine learning model for visual recognition when barcodes are unreadable, bridging the gap between traditional and vision-based methods.**
- **Physical Security Augmentation: A novel design uses a two-stage cart with an ultrasonic sensor to physically prevent theft [23]. The process involves:**
    1. **The sensor opens the cart's lid upon detecting a product.**
    2. **The item is scanned with a barcode reader in the first stage.**
    3. **After scanning, the item drops into a locked second-stage compartment, eliminating post-scan removal.**

2.3.3.3 Radio Frequency Identification (RFID) Systems

RFID offers a fully automated, contactless alternative to barcode scanning, though with higher infrastructure requirements.

- **Automated Scanning: In RFID-based carts, each item is tagged with an RFID chip. A cart-mounted EM-18 RFID reader automatically detects items placed inside, retrieving prices from a database and displaying a running total on an LCD [14].**
- **Advantages and Limitations: The key advantages over barcodes are a longer scanning range (up to 300 ft) and the ability to read multiple tags simultaneously without line-of-sight [14]. The primary limitation is the significant upfront investment required to pre-tag all products.**

2.3.3.4 Integration with Broader AI-Driven Retail Ecosystems

Barcode and RFID scanners are frequently embedded within larger, multi-sensor smart cart architectures designed for comprehensive retail management.

- **Decoupled, Multi-Task Systems: In advanced implementations, a dedicated barcode scanner or RFID reader handles product identification, while separate sensor suites (e.g., RGB-D cameras, ultrasonic sensors) are used for customer tracking and navigation [11].**
- **Closed-Loop Retail Operations: These systems create an integrated ecosystem where a simple scan triggers multiple backend processes: real-time inventory updates, automated POS billing, and the generation of personalized recommendations using AI models like LSTM networks for inventory forecasting [11]. This demonstrates how simple identification technologies can serve as the trigger for a complex, data-driven retail workflow.**

2.3.4 RFID-Based Identification Systems

RFID-based smart carts utilize radio frequency identification for fully automated, contactless product detection. These systems prioritize automation, security, and user convenience but are contingent on the universal pre-tagging of store products, which presents scalability and cost challenges.

2.3.4.1 Core RFID Scanning and Billing Architecture

The fundamental architecture of an RFID smart cart involves an onboard reader, a microcontroller for processing, and a user interface for billing.

- **Automated Detection Process: A typical system employs an Arduino-controlled RFID reader mounted inside the cart. It continuously scans tagged items placed within its range, automatically updating a running bill displayed on an integrated LCD [14].**
- **Key Technological Advantage: Unlike barcodes, RFID does not require line-of-sight and can read multiple items simultaneously, significantly accelerating the scanning process and improving user convenience [14].**

2.3.4.2 Security-Oriented RFID Designs

Several implementations focus on using RFID technology to enhance theft prevention and transaction security.

- **Focused Signal and Multi-Layer Verification: One advanced system uses a UHF RFID reader with a 10-meter range inside a metallic cart enclosure that blocks signals from other carts to prevent cross-reads [21]. Its security protocol includes:**
  1. **A theft alert if an unscanned item is placed in the cart, blocking bill generation.**
  2. **A final verification layer at the store exit, where a separate RFID reader at a smart checkout point re-scans the entire cart contents.**
- **Communication and Notification: Data is transmitted via low-power XBee modules to a central SQL database, and the final bill is sent to the customer's smartphone via a Telegram application, creating a closed-loop, secure transaction system [21].**

2.3.4.3 Low-Power and Networked Implementations

Early and cost-sensitive RFID cart designs emphasize energy efficiency and reliable wireless data transmission.

- **ZigBee-Based Communication: A representative design uses an AT89S52 microcontroller with a 125 kHz RFID reader [30]. Scanned tag IDs are temporarily stored in an EEPROM and then transmitted wirelessly to a central billing server via a ZigBee CC2500 module, forming a low-power mesh network suitable for battery-operated carts [30].**

2.3.4.4 Enhanced User-Centric Features

Beyond basic scanning, some RFID systems integrate additional functionalities focused on shopper assistance and accessibility.

- **Budget Control and Motorized Assistance: One implementation uses dual Arduino microcontrollers to manage an RFID reader, keypad, LCD, and buzzer [31]. It allows users to set a purchase limit via a keypad, with a buzzer alerting them if the limit is exceeded. Furthermore, the cart is motor-driven and remotely controlled via an RF module, reducing physical effort for shoppers with mobility challenges [31].**

2.3.4.5 Limitations and Adoption Barriers

Despite their advantages, RFID-based systems face consistent challenges that affect widespread deployment.

- **Universal Pre-Tagging Requirement: All implementations require every product to be pre-tagged with an RFID chip, representing a significant upfront infrastructure investment for retailers [14, 21, 30].**
- **Metrics and Cost: These systems rarely provide quantitative detection accuracy metrics. Furthermore, the added complexity of features like motorization leads to a higher unit cost compared to conventional carts, creating an adoption barrier [31].**

2.4 WebApp

The web application functions as the central digital interface connecting the user, the smart cart, and the store's backend system. Its primary role is to enable shoppers to prepare their shopping list prior to entering the store, and selecting desired products. Once the user arrives at the store, the web app facilitates seamless authentication and cart pairing through a built-in QR code scanner that links the user's account and shopping list to an available smart cart. Throughout the shopping journey, the web app synchronizes with the cart's onboard system to display real-time updates, including item detection results, cart status, and expenditure tracking. After the user completes their shopping, the web application also provides the interface for secure digital payment by reading the checkout QR code generated by the cart. In this way, the web application serves as the primary control and communication platform, supporting list creation, cart activation, live monitoring, and payment processing to ensure a smooth and integrated shopping experience.

2.4.1 Navigation Assistance

2.4.1.1 Virtual Store Layout

The app displays a simplified digital representation of the store layout, helping users understand aisle arrangements [10]. The system uses real-time store layout data to optimize navigation. The Reinforcement Learning (RL) model dynamically adjusts paths based on the store's physical layout and obstacles, which implies that a WebApp could display an interactive, virtual map of the store to guide users [11]. The system uses a touchscreen interface at the vending box to display selectable items. There is no interactive store map or navigation; items are chosen from a digital catalog [17].

2.4.1.2 Route Guidance

It communicates navigation instructions received from the cart and central server, allowing dynamic updates as the shopping route changes [10]. The cart's RL-based navigation system provides optimized, real-time route guidance to avoid obstacles and reduce travel time. A WebApp could visualize this route, offering step-by-step directions to products or aisles [11]. AI-powered navigation apps offer real-time route optimization to help customers find products quickly. These apps reduce time spent wandering aisles and improve shopping efficiency [13]. The system provides guided navigation indirectly. When a user confirms a product selection, the Back-End Management Platform sends product location data to the Smart Shopping Cart, which then autonomously guides the customer to that location. The app acts as the initiating interface for this process [20].

2.4.2 Real-Time Information

Data from the cart (detected items, total price) is transmitted to the application instantly via wireless communication, ensuring accurate and up-to-date information.

The application retrieves product data, pricing, and promotional details directly from the server and displays this information to the user while shopping [10].

2.4.2.1 Product Details Display

When the cart's camera identifies an item, the app instantly shows product name, price, availability, and category [10]. The cart automates product identification via barcode scanners or RFID, allowing real-time display of product details (price, description, promotions) through a customer interface (mobile app or in-store display) [11]. The WebApp displays real-time product details such as product name, price, weight, manufacturer, and expiry date after scanning via RFID. This information is fetched from a database and updated in the app [12]. A core function of the system is providing personalized product recommendations via an improved KNN collaborative filtering algorithm. The Client App is the endpoint that receives and displays these AI-driven suggestion lists to the user.

2.4.2.2 Inventory Status

The system retrieves stock data from the server, ensuring the user knows whether an item is available [10]. The system uses LSTM networks for real-time demand forecasting and inventory tracking. A WebApp could show live inventory status ("in stock," "low stock") for each product [11]. Inventory status is implicitly managed; when a product is scanned, the system can update stock at the retailer's end. However, real-time inventory display to the customer is not explicitly detailed [12]. AI-driven systems provide real-time stock alerts and inventory updates. Apps can notify customers if an item is out of stock and suggest alternatives. Smart shelves with RFID help maintain accurate inventory levels visible to both store and customer [13].

2.4.3 Budget Management

2.4.3.1 Live Cost Tracking

As items are added to the cart, the app automatically calculates total spending [10]. The cart integrates with POS systems for automated billing and real-time product scanning. A WebApp could display a running total of the customer's cart, updating as items are added or removed [11]. The system calculates and displays the total bill in real time on the touchscreen as items are selected [17]. The WebApp continuously updates the bill as items are scanned. The total cost is displayed in real time, allowing customers to track spending [12].

2.4.3.2 Budget Limit Alerts

Users can set a budget; the app notifies them when they are close to exceeding it [10]. While not explicitly mentioned, the system's real-time tracking and POS integration could support budget alerts if a user-defined spending limit is approached [11].

2.4.4 Personalized Recommendations

2.4.4.1 AI-Driven Suggestions

The app generates item recommendations based on past shopping habits, current cart contents, and user preferences [10]. The system uses a CNN-based deep learning model to analyze customer behavior and generate personalized product recommendations, which can be displayed via the WebApp [11]. A core function of the system is providing personalized product recommendations via an improved KNN collaborative filtering algorithm The Client App is the endpoint that receives and displays these AI-driven suggestion lists to the user.

2.4.4.2 Adaptive Learning

The system refines suggestions over time as user behavior data accumulates [10]. The recommendation model continuously learns from shopping history and real-time interactions, improving suggestions over time (adaptive learning) [11]. AI systems continuously learn from customer behavior to refine suggestions, enhancing relevance over time (adaptive learning) [13]. The recommendation system is adaptive. It constructs a customer-product rating matrix from historical data and calculates customer similarities to predict preferences for new items. This allows the system to learn and adapt its suggestions based on user behavior patterns [20].

2.4.5 User Interface and Experience

The application provides a structured interface through which users access product details, shopping history, and personalized promotions [10].

2.4.5.1 Real-Time Data Feed

The app continuously receives updates from the cart and displays them visually [10]. The WebApp (customer interface) provides a real-time feed of cart contents, recommendations, navigation prompts, and inventory updates, powered by edge and cloud processing [11]. The WebApp provides a real-time feed of scanned items, prices, and weight data. It also supports live surveillance via Pi Camera in case of weight mismatch [12]. Apps provide a live feed of scanned items, prices, promotions, and navigation cues. Features like AR overlays and digital receipts enhance interactivity [13]. The Client App maintains a secure, real-time communication channel (via enhanced HTTPS) with the Back-End Management Platform [20].

2.4.5.2 Enhanced Experience

The interface is designed to simplify decision-making and improve comfort during shopping [10]. Features like automated billing, contactless payment, and personalized navigation contribute to a seamless, enhanced shopping experience through the WebApp [11]. The app enhances experience by enabling contactless, queue-free shopping, automated billing, and multiple payment options (UPI, cards, wallets) [12]. The integration of contactless payment, automated checkout, and personalized assistance creates a seamless, engaging shopping experience, improving overall satisfaction [13]. The system enhances experience through automation, cashless QR payment, and reduced waiting time. It is designed for convenience at mini-marts [17]. The app is designed with a modular architecture (User, Device Connect, Product Display, Checkout, Personal Center modules) to provide a structured and seamless workflow from login to payment. Features like personalized recommendations, guided shopping and integrated payment directly contribute to an enhanced user experience [20].

2.4.6 Communication Role of the Application

2.4.6.1 Data Exchange with the Cart

The application communicates over wireless protocols.

- **It receives navigation, item recognition, and cost updates from the smart cart. The cart communicates with a cloud-based central server via MQTT, HTTP/HTTPS, and Wi-Fi/Bluetooth. The WebApp acts as a client, exchanging data such as customer profiles, cart updates, and inventory information in real time [11]. The WebApp communicates with a central server/database via the Raspberry Pi. Product data, billing info, and inventory updates are exchanged in real time using Bluetooth (for cart control) and likely Wi-Fi/HTTP for data sync [12]. Mobile apps connect to cloud-based central servers via Wi-Fi/cellular networks to sync shopping data, update inventory, process payments, and fetch personalized content in real time [13].**

2.5 Self-Checkout

The self-checkout process in a smart shopping cart system represents the culmination of the automated shopping experience, integrating real-time product detection, secure transaction processing, and seamless payment to eliminate traditional checkout queues. This section reviews the technologies and methodologies enabling automated checkout, from product verification and receipt generation to secure payment processing and transaction finalization.

2.5.1 Automated Product Verification and Receipt Generation

The cornerstone of self-checkout is the accurate and continuous tracking of cart contents, ensuring that the final bill reflects all items added and removed during the shopping session.

2.5.1.1 Real-Time Cart Synchronization and Bill Updates

In smart cart systems, the receipt is dynamically updated as items are added or removed [15]. This relies on continuous communication between the cart's identification system (e.g., cameras, RFID) and the central billing logic. As products are visually identified and confirmed via sensor fusion (e.g., weight change), they are instantly added to a digital cart within the system. The running total is calculated in real-time and displayed to the user, providing transparency and allowing for budget monitoring. This real-time synchronization is fundamental to the "just walk out" or seamless checkout experience, as the final transaction is essentially pre-verified throughout the journey [11, 20].

2.5.1.2 Multi-Sensor Validation for Accurate Billing

To ensure billing accuracy and prevent errors, advanced systems employ multi-modal verification. A common approach fuses visual identification with weight data [19, 29]:

1. Vision-Weight Correlation: When a product is placed in the cart, a vision system (e.g., YOLO) identifies it, while a load cell measures the weight change. The system cross-references the identified product type with the expected weight range. A significant discrepancy can trigger a re-scan or alert, preventing misidentification or accidental double-counting [19, 29].
2. Action-Triggered Verification: Some systems use ultrasonic sensors to detect a "pick" or "place" event near the cart basket [29]. This event triggers the camera and weight sensor to capture data simultaneously, creating a tightly correlated verification instance for each item interaction.

This layered validation is critical for building user trust in the automated checkout process, as it moves beyond a single point of failure [15, 19].

2.5.2 Secure and Flexible Payment Processing

Once shopping is complete, the system must facilitate a secure, convenient, and flexible payment process without requiring the user to unload items or interact with a traditional cashier.

2.5.2.1 Integration with Digital Payment Gateways

Smart cart systems typically offload payment processing to established, secure Payment Service Providers (PSPs) [17]. The cart's main processor communicates with PSP APIs (e.g., for credit cards, digital wallets like MoMo or VNPay) over encrypted HTTPS connections. The user's payment credentials are handled by the PSP, not stored on the cart, enhancing security and compliance.

2.5.2.2 Multi-Modal Payment Initiation Interfaces

To cater to different user preferences, systems offer multiple ways to initiate and complete payment:

- QR Code Payment: Upon reaching a checkout zone or pressing a "checkout" button on the cart's interface, the system generates a dynamic, transaction-specific QR code on the cart's display [17]. The user scans this code with their smartphone's banking or wallet app to authorize payment. This method is popular due to its simplicity and widespread adoption.
- In-App Payment: The companion WebApp can display the final bill and integrate a secure payment portal [20]. After reviewing the receipt, the user can pay directly within the app using saved payment methods.
- Cart-Based Payment Interface: Some cart designs include a touchscreen that allows users to select a payment method and enter a PIN or confirm a contactless card payment directly at the cart, functioning as a mobile point-of-sale terminal [13].

2.5.2.3 Facial Recognition Payment (FacePay)

For maximum convenience and contactless experience, some advanced smart retail systems implement facial recognition technology for payment authentication [13]. This biometric approach, often referred to as "FacePay," allows customers to complete transactions simply by having their face scanned at the checkout point. The system typically works as follows:

1. User Enrollment: Customers pre-register their facial biometrics and link them to a payment method (e.g., credit card, digital wallet) through the store's mobile app or at a kiosk.
2. Checkout Authentication: At the time of checkout, a dedicated camera on the smart cart or at a checkout terminal captures the customer's facial features.
3. Secure Verification: The system performs a secure, real-time match against the enrolled biometric template [13]. Upon successful verification, the transaction is automatically authorized using the linked payment method.

This technology represents the pinnacle of contactless checkout, offering a frictionless "walk-out" experience while maintaining high security through biometric authentication.

2.5.2.4 Final Transaction Verification and Security

To prevent theft and ensure transaction integrity, a final verification step is often employed before the store exit:

- Checkpoint Validation: As the cart passes through a designated "smart gate" at the store exit, a fixed RFID reader may perform a final scan of all items in the cart, cross-checking them against the generated bill [21]. Any discrepancy can alert staff or prevent the gate from opening.
- Digital Receipt and Audit Trail: A detailed digital receipt is sent to the user's WebApp or email, listing all items, prices, and the transaction ID [11, 20]. This receipt also serves as an audit trail. The system maintains a secure log of all sensor data and identification events associated with the transaction, which can be reviewed in case of disputes.

2.5.3 Customer Journey Completion and Post-Checkout

The checkout process extends beyond payment to ensure a smooth conclusion to the shopping trip and prepare the system for the next user.

2.5.3.1 Post-Payment Cart Reset and Receipt Finalization

After successful payment authorization, the system performs several automated tasks:

1. **Cart Reset: The cart's digital bill is cleared, and its item log is reset [11, 17]. The weight sensor is tared to zero in preparation for the next customer.**
2. **Inventory Update: The sale is finalized in the store's central inventory management system, decrementing stock levels in real-time [11, 13].**
3. **User Session End: The pairing between the user's WebApp and the cart is terminated [20]. The cart's screen may display a "Thank You" message and then revert to showing a login QR code for the next user.**

2.5.3.2 Autonomous Cart Management for Fleet Efficiency

In fully autonomous systems, the cart can manage its own operational status post-checkout:

- **Self-Docking for Charging: Once empty and idle, the cart can autonomously navigate to a designated charging dock, ensuring it is powered and ready for the next customer, contributing to a fully autonomous retail ecosystem [20].**

2.6 Hardware Components

The hardware architecture of a smart cart is defined by its processing hierarchy, which balances real-time responsiveness with computational power. This is achieved through a layered approach, separating high-level intelligence, low-level control, and specialized subsystem management.

2.6.1 Processing Units

Smart cart designs implement a tiered processing architecture to ensure both powerful computation for complex tasks and deterministic, low-latency control for safety-critical operations.

2.6.1.1 Heterogeneous Dual-Processor Architecture for Navigation

High-end, autonomous carts employ a split architecture to separate high-level planning from real-time control.

- High-Level CPU: A powerful central unit, such as a Raspberry Pi or NVIDIA Jetson module, runs computationally intensive algorithms like Simultaneous Localization and Mapping (SLAM), visual odometry, and path planning [20].
- Real-Time MCU: A dedicated microcontroller unit (MCU) like an STM32 handles all time-critical, low-level operations. This includes motor control, PWM generation for actuators, and direct interfacing with proximity sensors and encoders. This separation ensures that high-level processing does not introduce latency that could compromise the cart's immediate stability and safety [20].

2.6.1.2 Central Single-Board Computers (SBCs) for General-Purpose Control

For many smart carts, a versatile SBC acts as the main computational hub.

- Raspberry Pi as Primary Hub: The Raspberry Pi 4 Model B (with quad-core processor, 2-8GB RAM) is a common choice. It serves as the central board, handling tasks like barcode reading, machine vision-based object recognition, data processing, and cloud communication [15, 17].
- Application-Specific Use: In stationary automated checkout or vending systems, the Raspberry Pi also manages the user interface, payment APIs, and control of conveyor motors and actuators [17].

2.6.1.3 Dedicated Microcontrollers for Subsystem Management

To isolate specific functions, dedicated microcontrollers are often employed for subsystems like billing, security, or peripheral management.

- Billing Subsystem: A separate microcontroller (e.g., an Arduino) can be dedicated to managing the billing subsystem, interfacing with weight sensors, and processing product identification signals, isolating financial transactions from navigation tasks [19].
- Multi-MCU Coordination: Some architectures use a primary Arduino Mega 2560 (chosen for its high I/O pin count) to manage the main user interface and sensor logic, paired with a secondary Arduino Uno to handle specific peripheral communications, distributing the processing load [14, 18].
- Sequential Security Logic: For carts with automated physical security, a microcontroller runs a strict sequential logic: it processes ultrasonic sensor input, activates a servo to open a lid, awaits a valid barcode scan, and only then triggers a lock mechanism for the storage compartment [23].

2.6.1.4 Advanced Edge AI Processors

Carts with demanding, multi-modal AI requirements utilize specialized hardware for on-device processing.

- **High-Performance AI Platforms: Systems integrating multiple cameras and real-time conversational AI may use platforms like the NVIDIA Jetson AGX Orin. This allows the entire software stack—including computer vision, automatic speech recognition, and natural language processing—to run directly on the cart [9].**

2.6.1.5 Edge-Cloud Processing Split

A critical design principle is the strategic allocation of tasks between onboard and remote processors.

- **Edge Processing (Onboard): Handles all latency-critical tasks requiring immediate response, including real-time sensor fusion, obstacle avoidance, person tracking, and motor control. This ensures safe operation independent of network connectivity [11].**
- **Cloud Processing (Remote): Reserved for non-real-time, computationally intensive tasks such as training machine learning models, analyzing long-term shopping patterns for recommendations, and store-wide inventory forecasting [11].**

2.6.2 Sensing & Scanning Modules

Robust perception in smart carts is achieved through a sensor fusion approach, combining multiple sensing modalities for redundancy, accuracy, and comprehensive environmental awareness. These modules handle tasks from user tracking and navigation to precise product identification for automated billing.

2.6.2.1 Navigation, Tracking & Obstacle Avoidance Suite

This core sensor suite enables the cart to follow the user, map its environment, and avoid collisions.

- **Visual Tracking & Navigation:**
  - **An RGB camera is used for customer tracking (e.g., via skeleton tracking) and detecting visual navigation cues [20].**
  - **More advanced systems use RGB-D cameras, which provide both color and depth data, for more robust person tracking and obstacle recognition [11].**
- **Proximity & Ranging Sensors:**
  - **Infrared (IR) or ultrasonic sensors on the cart's perimeter maintain a safe following distance from the user and detect immediate obstacles for collision avoidance [20].**
  - **Ultrasonic sensors are also used for cart access control; a sensor at the cart's opening can detect a product in a user's hand to trigger an automated lid system [23].**
- **Absolute Positioning Systems:**
  - **For precise indoor localization, many systems integrate Ultra-Wideband (UWB) positioning. A typical setup uses three fixed UWB base stations in the store and a UWB tag on the cart to achieve centimeter-level accuracy in the cart's absolute coordinates [20].**
- **Enhanced Perception Configurations:**
  - **For 360-degree awareness, some systems employ a multi-camera rig (e.g., three 120° FOV cameras) for continuous localization and generating auditory descriptions of the surroundings [24].**

- Infrastructure Augmentation: The cart's onboard perception is often supplemented by the store's infrastructure, such as fixed overhead cameras that can relocate a customer if they move out of the cart's line-of-sight, ensuring seamless tracking [20].

2.6.2.2 Product Identification & Billing Sensors

This category includes the dedicated hardware for recognizing items added to the cart and enabling automated checkout.

- Vision-Based Identification:
  - A dedicated high-resolution camera (e.g., an 8MP Raspberry Pi Camera) is configured for object detection using models like YOLO to identify produce or packaged goods [19, 22].
- Weight Verification:
  - A precision weighing system using strain-gauge load cells interfaced via a 24-bit ADC amplifier (HX711) measures the mass of items placed in the cart [19].
- Automatic Label Scanning:
  - Dedicated barcode scanners (e.g., GM-73) or RFID readers are integrated for automatic item registration [11, 22].
- Sensor Fusion Logic: These modules work together through sensor fusion. The system correlates a sudden weight increase from the load cell with a successful barcode scan or visual identification to definitively register an item as 'added' to the cart [22].
-

2.6.2.3 Radio-Frequency Identification (RFID) Systems

RFID provides a foundational, contactless method for product identification and inventory management in automated stores.

- Core Technology: RFID readers in the store environment, paired with RFID tags on every product, enable contactless identification and real-time inventory tracking without line-of-sight [13].
- On-Cart Implementation: Some systems use a dedicated RFID reader attached to the cart to scan product tags and fetch details from a database, eliminating visual scanning [12].
- Hardware Specifications & Design:
  - Ultra-High Frequency (UHF) RFID readers (865-928 MHz) are preferred for their longer read range (up to 10 meters) and ability to rapidly scan multiple items [21].
  - Dual-Antenna Systems: To prevent misreads, a low-power internal antenna (~30 cm range) reads tags inside the basket, while a directional external antenna (~4 m range) is used for store localization [18].
  - Signal Isolation: The cart's structure may incorporate RF shielding materials (e.g., metallic mesh) to contain the reader's signal within the basket and prevent interference from nearby shelves or other carts [18].

2.6.2.4 Payment & Infrastructure Integration

Sensing modules also extend to completing transactions and integrating with the broader store ecosystem.

- QR Code Payments: The scanning hardware is effectively offloaded to the user's smartphone. The cart's role is to generate and display a dynamic, transaction-specific QR code on its screen for the user to scan [17].
- Smart Store Integration: Onboard perception is augmented by store infrastructure like smart shelves equipped with weight or RFID sensors for real-time inventory monitoring [13].

2.6.3 Actuation & Mobility

Advanced mobility is fundamental for a smooth user-following experience. To navigate crowded aisles and perform precise maneuvers, smart carts utilize sophisticated drive systems. A prominent solution is the use of Mecanum wheels, which consist of four independently controlled wheels with rollers set at 45-degree angles. This configuration allows for omnidirectional movement, enabling the cart to strafe laterally, rotate on the spot, and move in any direction without changing its orientation. The wheel motors are governed by a differential drive control algorithm executed on the dedicated STM32 motion controller. This system processes commands from the high-level planner to accurately follow a customer at a preset distance (e.g., 0.5 m) or navigate autonomously to a specified product location on a store map [20].

The motion controller adjusts speed and direction in real-time based on processed sensor data, enabling smooth navigation and precise customer following [11].

2.6.3.1 Manual Control Interface

To provide direct user control over cart movement, some systems integrate a manual control interface via a smartphone application (e.g., Blue Dot app). This app communicates with the cart's onboard processor via Bluetooth, allowing the user to remotely drive the cart in any direction without physically pushing it, combining manual convenience with powered mobility [12].

2.6.3.2 Automated Cart Access & Security Mechanism

A distinct mechanical design employs an automated lid system for theft prevention and item verification. An ultrasonic sensor mounted on the cart detects when a user approaches with a product and automatically triggers a servo motor to open the cart's lid. The cart interior is divided into two locked compartments. The first is a scanning stage where the item's barcode is read. Upon successful scan verification against a database, a second servo or solenoid lock releases, allowing the item to fall into the main storage compartment. This two-stage physical design ensures only scanned items are stored, physically preventing theft [23].

2.6.3.3 Automated Item Retrieval & Conveyor Systems

For stationary smart retail systems, actuation hardware focuses on automated item handling. The core components are: Auxiliary Conveyor Belts: Multiple small, motorized belts, each dedicated to a column of products on a shelf. They run a short distance (e.g., 7 cm) to push a single selected item off the shelf. Main Conveyor Belt: A larger, continuously running belt that receives items from the auxiliary conveyors and transports them to a customer collection basket or chute. DC Motors & Motor Drivers: These motors power the conveyor belts. Their operation is sequenced by the main controller to deliver specific items upon successful payment verification [17].

2.6.4 Communication Modules

Reliable and secure data exchange is essential for integrating the smart cart into a larger IoT ecosystem. Communication hardware and protocols are multi-layered, handling everything from high-bandwidth cloud syncing to low-power, machine-to-machine messaging.

2.6.4.1 Primary Connectivity & Data Exchange

Smart carts utilize a combination of wireless technologies to meet different bandwidth, range, and power requirements.

- **High-Bandwidth Communication (Wi-Fi): Wi-Fi (802.11 a/b/g/n/ac) serves as the primary channel for uploading sensor logs, downloading map/product updates from the cloud, and syncing cart contents with a companion mobile application [20].**
- **Precise Positioning & Short-Range Comms (UWB): Ultra-Wideband (UWB) technology serves a dual purpose: providing centimeter-level indoor positioning and enabling short-range, low-power communication between devices [20].**
- **Low-Power Mesh Networking (Zigbee/XBee): For creating efficient, low-power wireless networks within a store, Zigbee protocol modules like XBee series radios are used. They enable reliable machine-to-machine (M2M) communication between carts, central servers, and fixed infrastructure like smart shelves, with lower power consumption than Wi-Fi [21].**
- **Simple Dedicated Links (433 MHz RF): For basic wireless data transfer between components within the cart itself (e.g., from a scanner to a display), simple 433 MHz RF transmitter and receiver modules are sometimes employed as a dedicated communication link [18].**

2.6.4.2 Communication Protocols & Software Architecture

The choice of communication protocols and software design is critical for efficient, secure, and responsive system operation.

- **Lightweight IoT Protocol (MQTT): For efficient, low-latency machine-to-machine communication within the store ecosystem, the lightweight MQTT (Message Queuing Telemetry Transport) protocol is often implemented in the hardware's network stack [13].**
- **Modular Software Design: The onboard software is typically architected as multi-threaded and modular, where functional blocks (e.g., image capture, audio processing) run as independent threads. This requires a multi-core processor and influences the hardware's need for robust inter-process communication support [9].**

2.6.4.3 Security Hardware & Protocols

Protecting sensitive user and transaction data is a hardware-enabled feature, requiring specific capabilities and protocols.

- **Encrypted Communication Channels: All sensitive communications are secured using encrypted HTTPS/TLS channels. Additional security layers, such as RSA digital signatures, can be implemented to authenticate messages between the cart, mobile app, and cloud backend, preventing spoofing and ensuring data integrity [20].**
- **Hardware-Enabled Security: Onboard processors must have the capability to enforce role-based access control and handle AES-256 encryption to protect customer data directly on the device [11].**
- **Secure Payment Gateways: Payment processing relies on Wi-Fi connectivity to communicate securely with external Payment Service Provider (PSP) APIs (e.g., MoMo, VNPay) for transaction authorization [17].**

2.6.4.4 Peripheral & Subsystem Communication Interfaces

Hardware components within the cart communicate with the main processors via standard electronic interfaces.

- **Motor Control Interface: DC motors are interfaced via a motor driver IC (e.g., L298N dual H-bridge), which allows low-voltage control signals from the MCU to drive higher-current motors using Pulse-Width Modulation (PWM) for precise speed and direction control [12].**
- **RFID Reader Interface: Common RFID reader modules like the EM-18 operate at 125 kHz and interface with the microcontroller via serial communication (UART) to transmit scanned tag data [14].**

2.6.4.5 System Integration & Final Verification Points

Communication culminates at dedicated infrastructure points that integrate the cart into the final retail workflow.

- **Smart Checkout Gateway: A dedicated smart checkout point acts as a final verification station. Equipped with its own RFID reader/UHF antenna and connected to the store's POS system, it reads all items in the cart as it passes through to validate the purchase and prevent theft [21].**
- **Sensor Fusion for Verification: Ultrasonic distance sensors mounted inside the cart frame measure the distance to items, helping to verify that a scanned item has been physically placed inside the basket, fusing communication data with physical sensing [14].**

2.6.5 Power Systems

As mobile robotic platforms, smart carts are battery-operated, requiring sophisticated power management systems to ensure continuous, autonomous operation.

- **Battery Technology: Carts are typically powered by high-capacity Lithium-ion or Lithium-polymer battery packs.**
- **Power Management Features:**
  - **Remote Battery Monitoring: The system can report real-time battery levels to a central monitoring station for operational planning [20].**
  - **Autonomous Charging: To ensure operational readiness, the cart can be programmed to enter an automatic charging mode upon completing a checkout cycle. It would then navigate autonomously to a designated docking station for inductive or conductive charging, enabling fully autonomous energy management [20].**

2.6.6 Billing & User Interface Modules

This hardware category encompasses the components that enable customer interaction, transaction processing, and payment finalization, transforming the smart cart into a self-service point-of-sale.

2.6.6.1 Transaction Processing Hardware

The core of the automated checkout system relies on sensors and modules that verify and quantify items.

- Precision Weighing System: A precision load cell, interfaced with a high-resolution analog-to-digital converter (ADC) like the HX711 module, provides accurate measurement of the weight of items added to the cart [19].
- Product Identification: This is typically handled by the main CPU's vision system (e.g., YOLO models) or by dedicated scanners (barcode/RFID) as described in section 2.6.2.
- Transaction Logic: A dedicated subsystem (often an MCU) manages the transaction logic, calculating cost based on identified product type and measured weight [19].

2.6.6.2 Visual Display Interfaces

Displays provide users with real-time feedback on their shopping activity and bill status.

- Basic Displays: Simple LCD displays are used to show itemized bills and total costs [19].
- Advanced Interactive Displays: More advanced interfaces employ a 7-inch TFT touchscreen LCD, which not only displays the bill but also allows for interactive input, such as browsing items or initiating checkout directly from the cart [15, 17].

2.6.6.3 User Input & Control Hardware

A variety of input methods are provided to ensure accessibility and cater to different interaction preferences.

- Physical Buttons: Dedicated physical buttons (e.g., 'Reset', 'Next Item', 'Checkout', 'Voice Command') provide tangible, reliable input, which is crucial for accessibility and usability in a moving cart [9, 19].
- Numeric Keypads: For direct numeric input like setting a budget or entering a PIN, some carts integrate a physical membrane keypad (e.g., 4x4 matrix) connected directly to the microcontroller [14].
- Dedicated Payment Trigger: A dedicated physical payment button on the cart's interface can be pressed to trigger the final bill generation and payment sequence [21].

2.6.6.4 Feedback & Interaction Hardware

These components confirm actions and facilitate multi-modal communication with the user.

- Auditory & Visual Feedback: A piezoelectric buzzer provides an audible confirmation beep upon a successful item scan, while LED indicator lights offer visual status cues [15].
- Audio Subsystem: For voice interaction and accessibility, an onboard microphone captures voice commands, and a headphone output delivers private, clear audio feedback (synthesized speech, auditory cues) directly to the user, which is essential in noisy environments [9].

2.6.6.5 Payment & Authentication Hardware

Finalizing the transaction involves secure methods to authorize payment and verify identity.

- Biometric Systems: Advanced systems integrate facial recognition hardware (dedicated cameras & processors) to verify a customer's identity for completely contactless 'FacePay' transactions [13].
- Payment Integration: The billing process is often completed via the user's smartphone. The system may send the final bill and a payment link through messaging services (e.g., Telegram) or display a dynamic QR code on its screen for the user to scan with their phone to pay [17, 21].

2.6.7 Store Integration & Enabling Hardware

The smart cart does not operate in isolation; its full functionality is enabled by complementary hardware deployed as part of the store's digital infrastructure.

- **Indoor Positioning Infrastructure: A network of UWB anchors, Bluetooth Low Energy (BLE) beacons, or visual fiducial markers (ArUco) placed on walls and shelves provides the absolute location data required for precise cart navigation and product location services [13].**
- **Store Sensing Grid: Fixed surveillance and overhead tracking cameras supplement the cart's onboard perception. This infrastructure aids in customer re-identification, store analytics, and seamless tracking if a customer moves out of the cart's line-of-sight [13].**
- **Checkpoint Terminals: Designated payment kiosks or smart gates act as final verification points. These terminals, equipped with their own touchscreens and payment hardware, are where carts can finalize transactions or where users can resolve issues [13].**
- **Charging Infrastructure: Designated docking stations with physical or inductive connectors are installed to enable autonomous cart charging, ensuring the fleet remains operational [13].**

Synthesis of Hardware Components in Smart Retail Systems

Hardware Category	Specific Component / Technology	Primary Function	Key References
Processing Units	Raspberry Pi 4 (SBC)	Central hub for OS, applications, database comms, and UI. Used in both mobile carts and stationary kiosks.	[15], [12], [17]
	NVIDIA Jetson AGX Orin (SoM)	High-performance edge AI processing for multi-camera vision, sensor fusion, and NLP.	[9]
	STM32 Family (MCU)	Real-time control for mobile systems: motor PWM, sensor polling, low-level actuation.	[20]
	Arduino Uno / Mega 2560 (MCU)	Microcontroller for sensor interfacing, billing logic, peripheral control, and RFID handling. Often used in dual-MCU setups.	[19], [14], [18]

Table 2.2: Processing Units Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Sensing & Scanning	RGB / RGB-D Camera	Customer tracking, obstacle detection, and environmental awareness.	[20], [11], [9]
	Pi Camera / USB Camera	Barcode scanning, object detection (YOLO), and live surveillance.	[19], [15], [12]
	UWB Positioning System	Centimeter-accurate absolute indoor localization for mobile carts.	[20]
	IR / Ultrasonic Sensors	Proximity detection, obstacle avoidance, item presence verification, and cart access triggering.	[20], [11], [14], [23]
	RFID Reader (EM-18, UHF) & Tags	Contactless product identification. Dual-antenna systems isolate in-cart items.	[13], [12], [14], [21], [18]
	Load Cell + HX711 ADC	Precise weight measurement of items for billing verification.	[19], [15]
	Smartphone Camera (User)	Offloaded sensor for scanning system-generated QR codes for payment.	[17]

Table 2.3: Sensing and Scanning Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Mechanical & Security	RF Shielding (in cart body)	Metallic mesh/coating to contain RFID signals and prevent cross-read errors.	[18]
	Dual-Compartment Chassis	Physical cart design with separate scanning and storage areas to prevent theft.	[23]
	Vending Machine / Kiosk Enclosure	Protective housing for stationary automated retail systems.	[17]

Table 2.4: Mechanical and Security Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Actuation & Mobility	For Mobile Carts:		
	DC Geared Motors	Provides primary wheel torque for movement.	[20], [12]
	Mecanum Wheels	Enables omnidirectional movement.	[20]
	L298N Motor Driver IC	Interfaces low-voltage MCU signals to higher-current motors for direction and speed control (PWM).	[12], [14]
	Servo Motors / Solenoid Locks	Controls automated cart lids and internal compartment locks for security.	[23]
	For Stationary Systems:		
	Conveyor Belt Systems (Auxiliary/Main)	Automated item retrieval and transport from shelves to a customer pickup point.	[17]
	Conveyor DC Motors	Powers the conveyor belts for item handling.	[17]

Table 2.5: Actuation and Mobility Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Billing & User Interface	LCD / TFT Touchscreen Display	Shows itemized bill, totals, product info, and payment QR codes.	[19], [15], [14], [17]
	Physical Buttons & Keypad	User input for commands, checkout, budget entry, and system reset.	[19], [9], [14], [21]
	Piezoelectric Buzzer	Provides audible feedback (beeps) for scans, errors, and alerts.	[15], [14], [18]
	Onboard Microphone & Headphone Output	Captures voice commands and delivers private audio feedback/TTS.	[9]
	Dedicated Payment Button	Physical button to initiate the final checkout and bill generation process.	[21]

Table 2.6: Billing and User Interface Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Communication	Wi-Fi (802.11 a/b/g/n/ac)	High-bandwidth data transfer with cloud servers, databases, and payment gateways.	[20], [15], [17]
	Bluetooth (BT/BLE)	Pairs with user smartphones for control apps (e.g., Blue Dot) and peripheral connectivity.	[15], [12]
	XBee (Zigbee) Modules	Low-power, reliable Machine-to-Machine (M2M) communication within a store network.	[21]
	RF Tx/Rx Modules (433 MHz)	Simple wireless link between subsystems within a cart or to a central unit.	[14], [18]
	UWB Communication	Used for both precise positioning and short-range data communication.	[20]

Table 2.7: Communication Synthesis.

Hardware Category	Specific Component / Technology	Primary Function	Key References
Power & Infrastructure	Li-ion / Li-Po Battery Pack	Primary mobile power source for carts.	[20]
	Smart Charging Dock	Autonomous docking station for charging mobile carts.	[20], [13]
	Store Infrastructure:		
	Smart Checkout Point / Gate	Fixed terminal with RFID reader for final purchase validation and theft prevention.	[21]
	UWB Anchors / BLE Beacons	External infrastructure for indoor positioning.	[20], [13]
	Fixed Overhead Cameras	Store-wide tracking and surveillance, supplementing cart perception.	[20], [13]
	Central Server / Cloud Database	Backend for inventory, pricing, user profiles, and transaction processing.	[15], [13], [17]

Table 2.8: Power and Infrastructure Synthesis.

Carto

CHAPTER 3

System Analysis and Design

Chapter 3: System Analysis and Design

3.1 System Overview

The system model of the Carto Smart Shopping Cart presents an integrated software-driven architecture designed to enhance in-store shopping efficiency through automation and intelligent interaction. The proposed system combines a web-based application, embedded cart software, AI-driven product recognition, and indoor navigation services into a unified platform. The web application acts as the central interface through which users create and manage shopping lists, authenticate and link with a physical cart via QR code scanning, and track shopping progress in real time. Once connected, the cart retrieves user data from the backend system and utilizes a digital store map to compute optimal navigation paths based on the selected items. Throughout the shopping process, scanned or AI-detected products are synchronized with the backend to generate a dynamic virtual receipt, allowing users to review and modify their selections before checkout. Upon completion, the system securely transitions the user to the payment phase and guides the cart to the designated checkout zone. This system model emphasizes seamless data flow, real-time synchronization, and modular software interaction to provide an intelligent, user-centric shopping experience while minimizing checkout delays and operational inefficiencies.

3.2 Functional Requirements

Actors:

User
Guest
Store Manager (admin)
Smart Shopping Cart (Hardware System)
AI Model (Product Detection, Navigation, Recommendation)

3.2.1 Log in (User)

The system should allow users to log in with valid credentials, unauthorized users should be denied access, and an appropriate error message should be displayed.

3.2.2 Logout (User)

Logged-in users should be able to log out securelyonce logged out, the session should be invalidated to prevent unauthorized access.

3.2.3 Create List (User, Guest)

Users shall be able to create a new shopping list by adding items via search or category browsing, the system shall save shopping lists to the authenticated user's profile for future access and modification, upon store arrival, users can select a saved list to load to their smart cart via QR code scan.

3.2.4 Delete List (User)

Authenticated users shall be able to delete any saved shopping list from their profile, the system shall require confirmation before permanently deleting a list to prevent accidental removal, deleted lists shall be permanently removed from the system and cannot be recovered.

3.2.5 Add Payment Method (User, Guest)

The system shall allow users to securely save and manage multiple payment options (credit/debit cards, digital wallets) in their profile, users shall be able to select a preferred payment method for automatic checkout upon completing their shopping, all payment transactions shall be encrypted and processed through a secure, PCI-compliant payment gateway.

3.2.6 Editing Store Map (Admin)

The system shall provide an admin interface for store managers to edit the store layout, including adding, moving, or removing aisles, Admins shall be able to assign and update product locations within the store map for accurate route planning, All map changes shall be reflected in real-time for users during active shopping sessions.

3.2.7 Generating QR Code (Smart Shopping Cart)

Each smart cart shall automatically generate and display a unique QR code on its integrated screen, the QR code shall remain active and displayed until a user successfully scans it and begins a shopping session.

3.2.8 Navigation (Smart Shopping Cart)

The system shall calculate and display the optimal in-store route based on the user's shopping list, the smart cart shall guide the user autonomously along the calculated route using onboard sensors and mapping, real-time navigation updates shall be provided to the cart screen

3.2.9 Recommendation (Smart Shopping Cart)

The system shall analyze items in the user's shopping list and suggest complementary products based on purchase patterns, recommendations shall be displayed on the cart screen when the user is physically near the suggested product's location, users shall be able to accept or dismiss recommendations directly from the cart interface.

3.2.10 Checkout (Smart Shopping Cart)

Once the user confirms their shopping is complete, the system shall calculate and display the shortest route to the designated checkout/packing zone, the smart cart shall guide the user autonomously to the checkout zone using the same navigation system, at the checkout zone, the system shall automatically finalize the payment process using the user's saved payment method.

3.2.11 Product Scanning (AI Model)

The cart's integrated dual-camera system shall continuously scan items placed into the cart using a trained AI recognition model, the AI model shall be trained on an extensive dataset of supermarket products to accurately identify items without manual barcode scanning.

3.2.12 Adding/Removing Products (AI Model)

The AI camera system shall detect when an item is placed into or removed from the cart, updating the virtual cart accordingly in real time, a weight sensor layer shall validate item changes by detecting weight variations, ensuring accuracy in cart updates, both systems shall work together to minimize errors and provide reliable, multi-layered product tracking.

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

The system shall respond to user interactions within 2 seconds, the AI object detection shall identify items in real-time with a latency of less than 500ms, the navigation system shall recalculate routes within 3 seconds of any map or list changes.

3.3.2 Reliability Requirements

The system shall maintain 99.5% uptime during store operating hours, the shopping cart hardware shall operate continuously for 8 hours on a single battery charge, the AI model shall achieve a minimum accuracy of 95% in product identification.

3.3.3 Security Requirements

All user data shall be encrypted using AES-256 encryption, payment transactions shall comply with PCI DSS standards, user sessions shall automatically expire after 30 minutes of inactivity.

3.3.4 Usability Requirements

First-time users shall be able to create a shopping list within 5 minutes, the cart interface shall be readable in various store lighting conditions.

3.3.5 Compatibility Requirements

The web application shall be compatible with Chrome, Safari, and Firefox browsers, the mobile interface shall be responsive on both iOS and Android devices, the cart system shall integrate with existing store inventory management systems.

3.3.6 Maintainability Requirements

The system shall allow remote software updates without hardware intervention, store administrators shall be able to update product databases without developer assistance, the AI model shall be retrainable with new products without system downtime.

3.4 System Environment

1. The system environment design describes the operational context in which the Carto Smart Shopping Cart system functions. It defines the hardware components, software platforms, intelligent services, and external systems that interact together to deliver an automated and user-centric shopping experience. The environment consists of multiple integrated layers. Users and guests interact with the system through a web-based application that allows them to create shopping lists, manage payment methods, and monitor their shopping progress. Store managers access a dedicated administrative interface to update the store layout, manage product locations, and maintain the product database. Each smart shopping cart operates as an embedded hardware system equipped with cameras, weight sensors, and a display unit. The cart communicates continuously with the backend server to retrieve shopping lists, display navigation instructions, and synchronize detected products in real time.

2. AI services form the intelligence layer of the system. These services analyze camera feeds and weight sensor data to detect when products are added or removed from the cart. The AI layer also optimizes in-store navigation routes and generates product recommendations based on user behavior and shopping patterns. The backend server acts as the central controller of the system. It handles authentication, shopping list management, cart synchronization, checkout processing, and secure communication between all components. Persistent data is stored in dedicated databases, including user accounts, shopping lists, product information, store maps, and transaction records. External systems, such as secure payment gateways and store inventory management systems, are integrated into the environment to support automated checkout and real-time inventory updates. All communication between system components is secured to ensure data integrity, privacy, and reliability. This system environment design ensures scalability, real-time operation, and maintainability, enabling the Carto Smart Shopping Cart system to function efficiently within a modern smart retail environment.

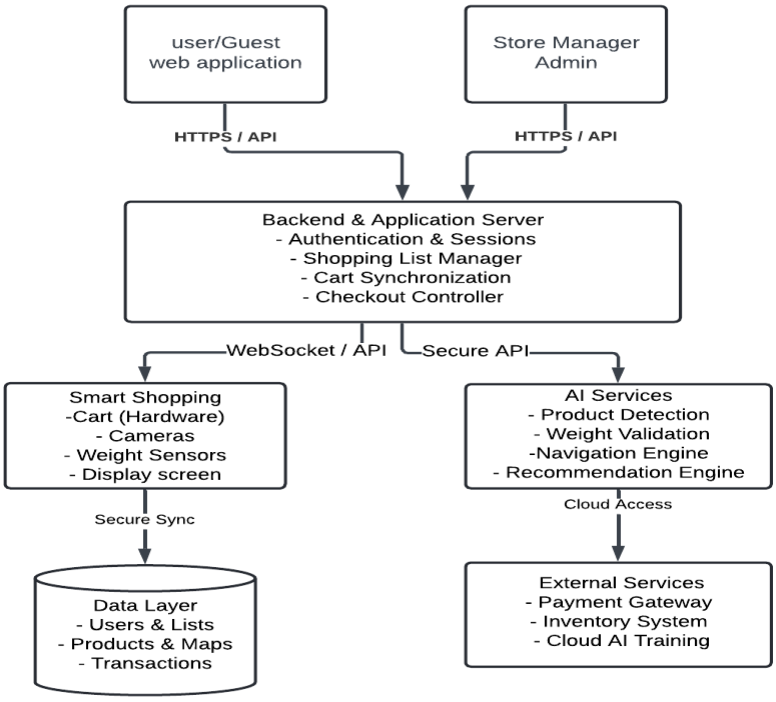


Figure 3.1: System Environment.

3.5 System Architecture

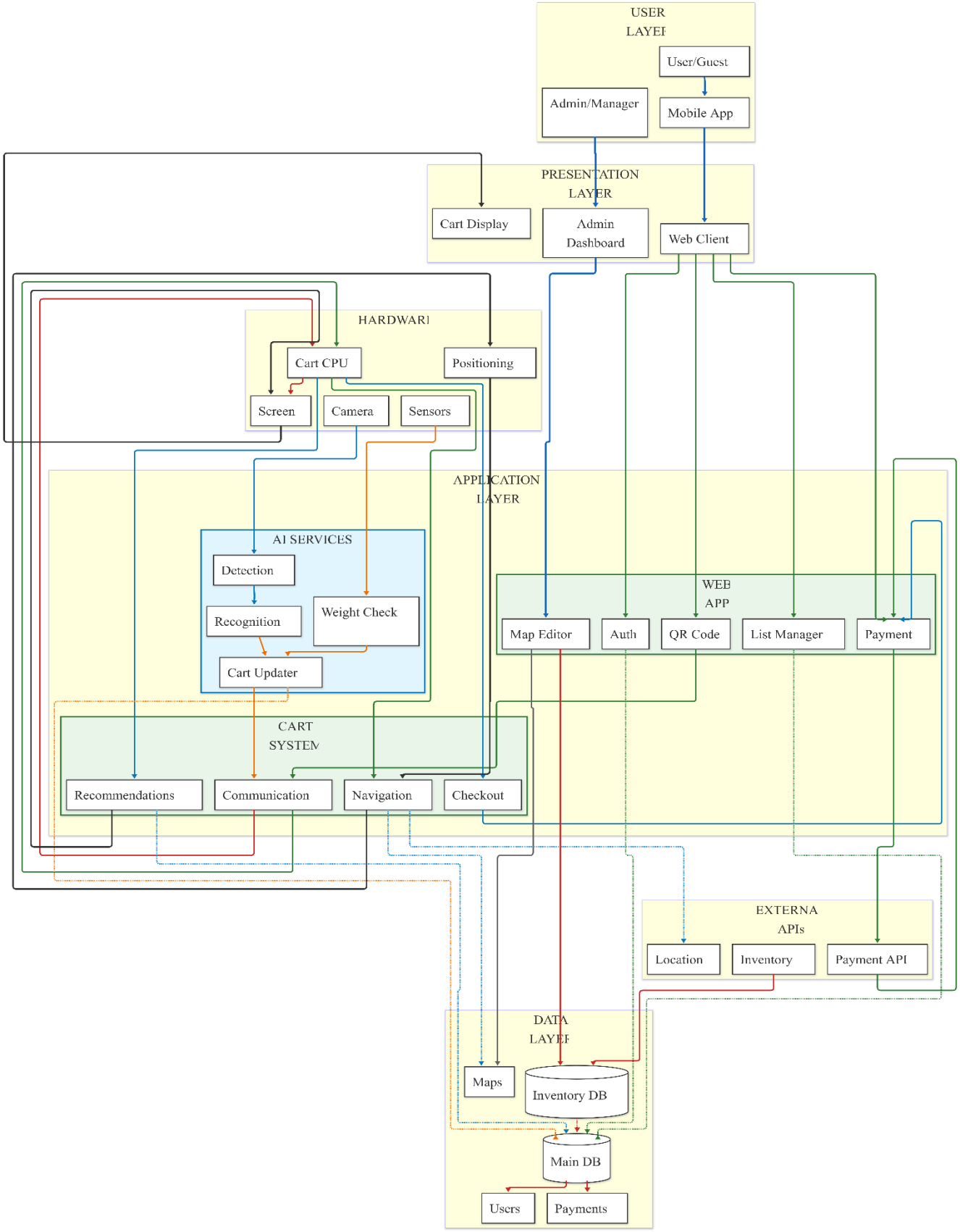


Figure 3.2: System Architecture.

3.6 Class Diagram

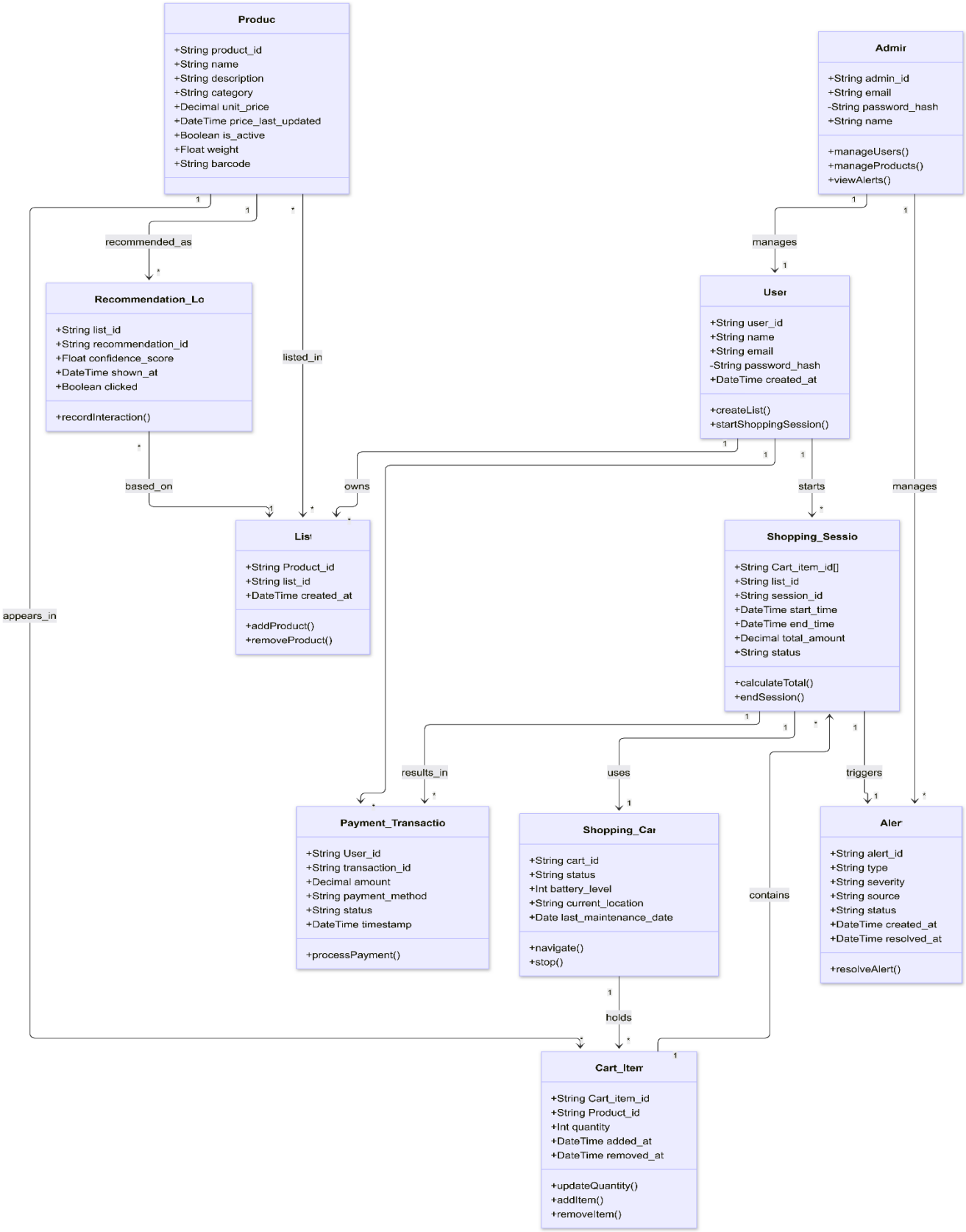


Figure 3.3: Class Diagram.

3.7 Use Case Diagram

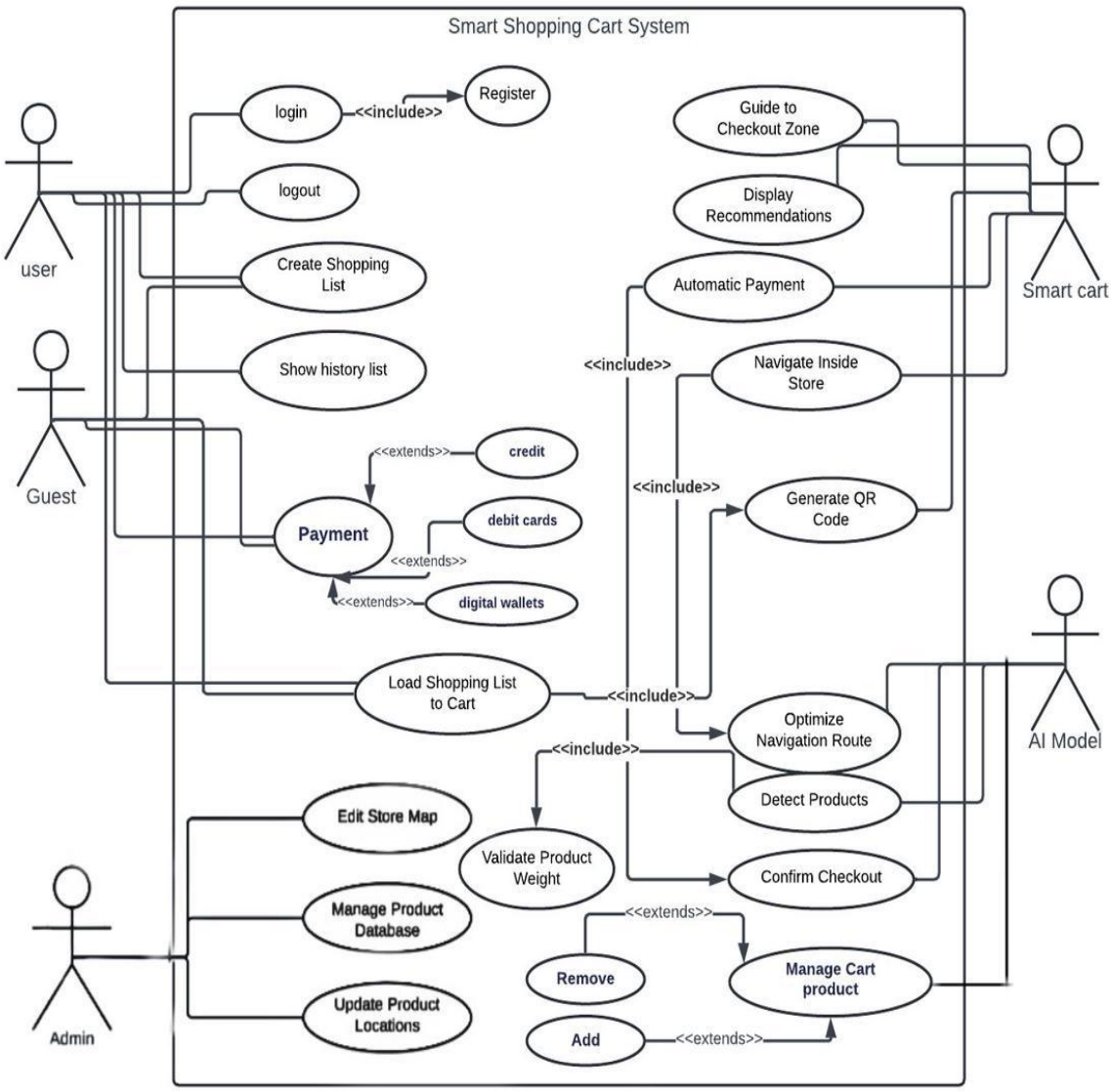


Figure 3.4: Use Case Diagram.

3.8 Sequence Diagram

3.8.1 Log-in

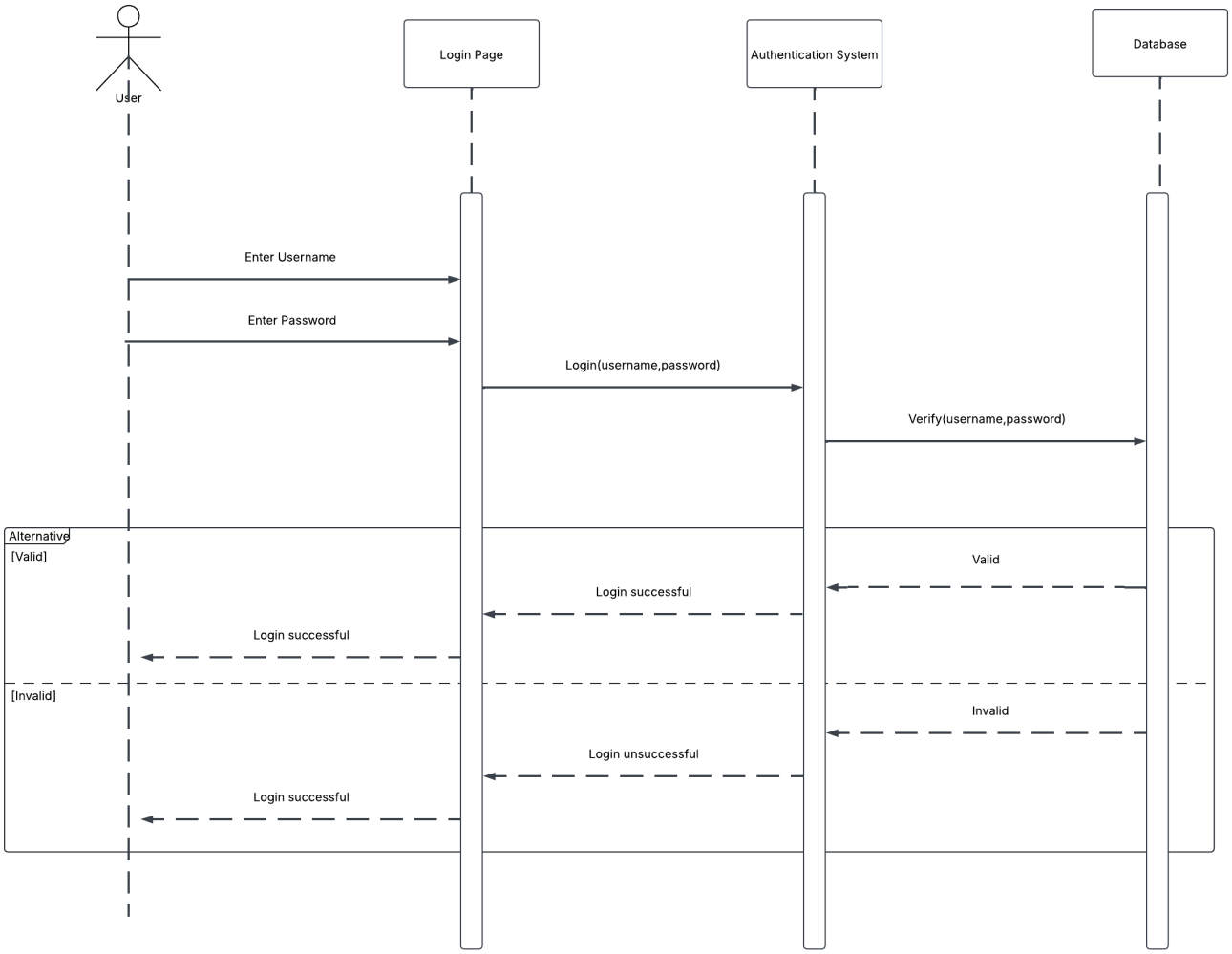


Figure 3.5: Log-in Sequence Diagram.

3.8.2 User Start Shopping

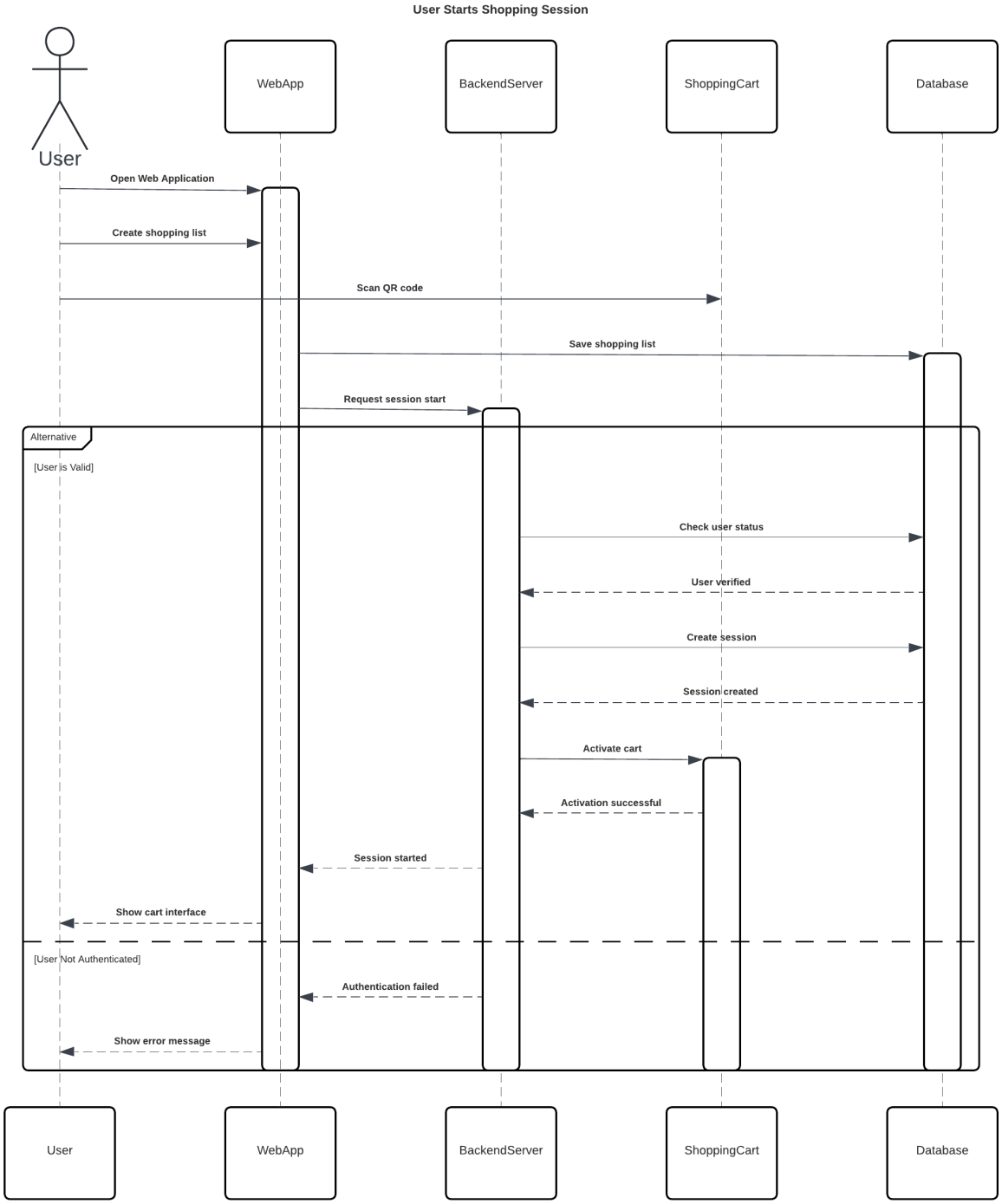


Figure 3.6: User Start Shopping Sequence Diagram.

3.8.3 Cart navigation & Route Generation

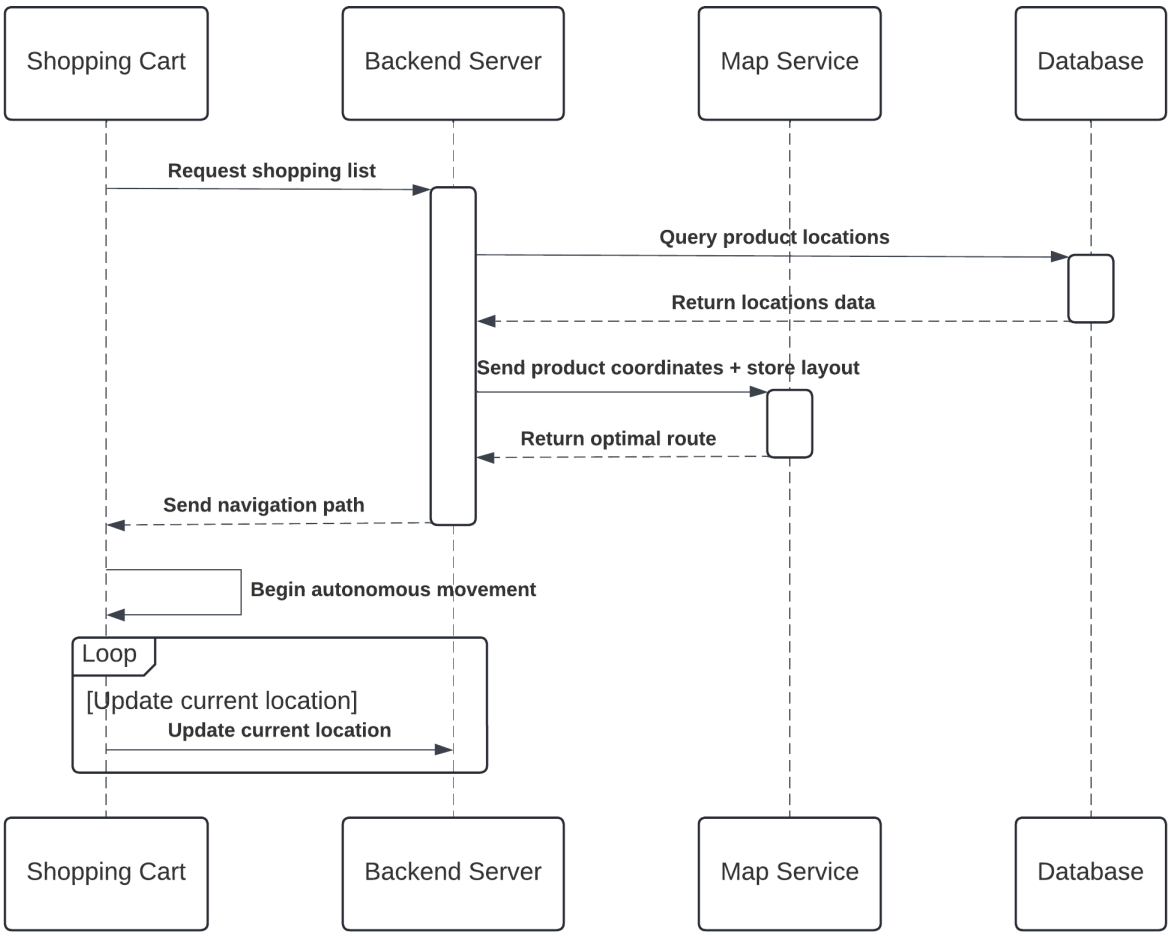


Figure 3.7: Cart Navigation and Route Generation Sequence Diagram.

3.8.4 Product Scanning & Virtual Receipt Update

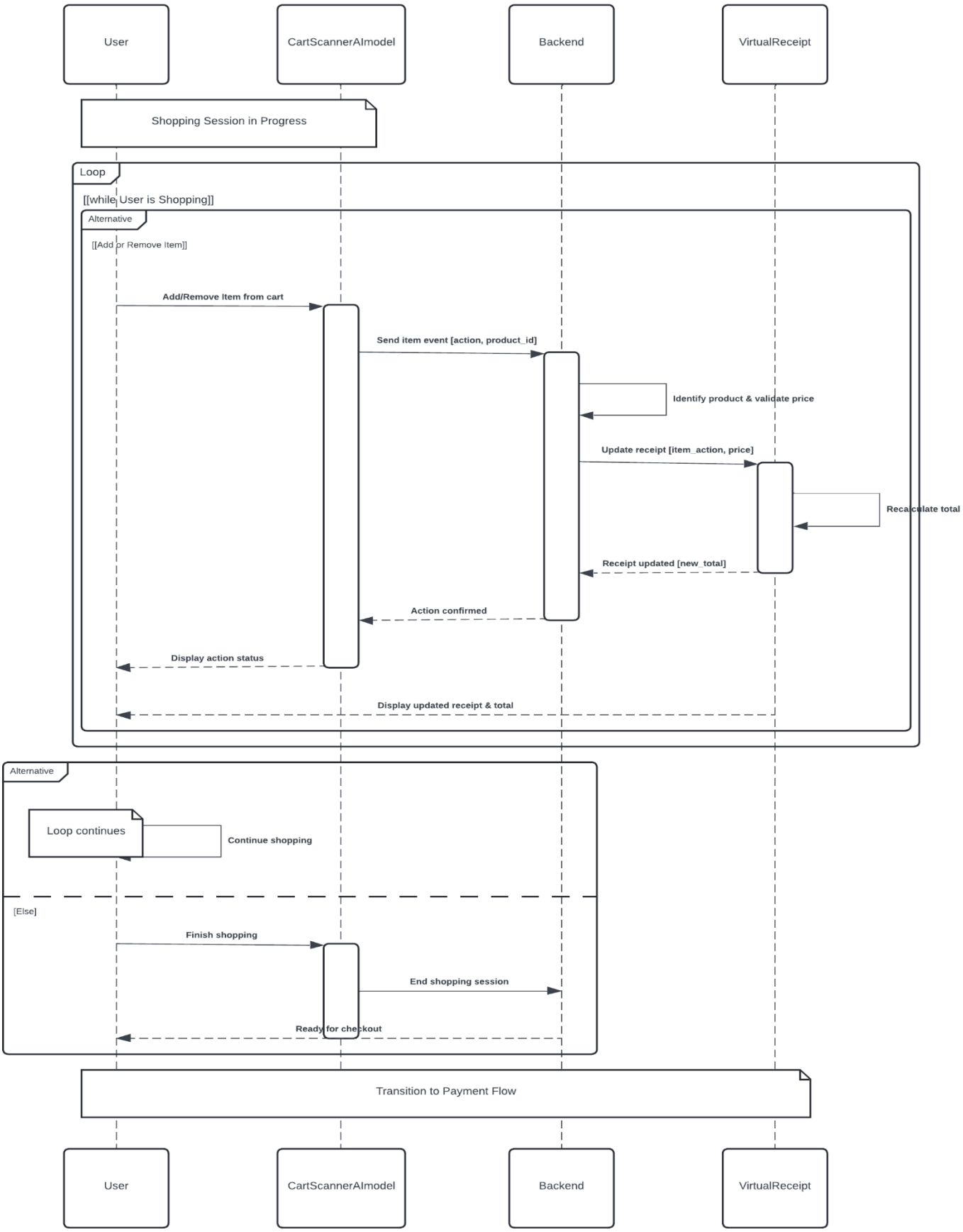


Figure 3.8: Product Scanning and Virtual Receipt Update Sequence Diagram.

3.8.5 Adding Payment Method

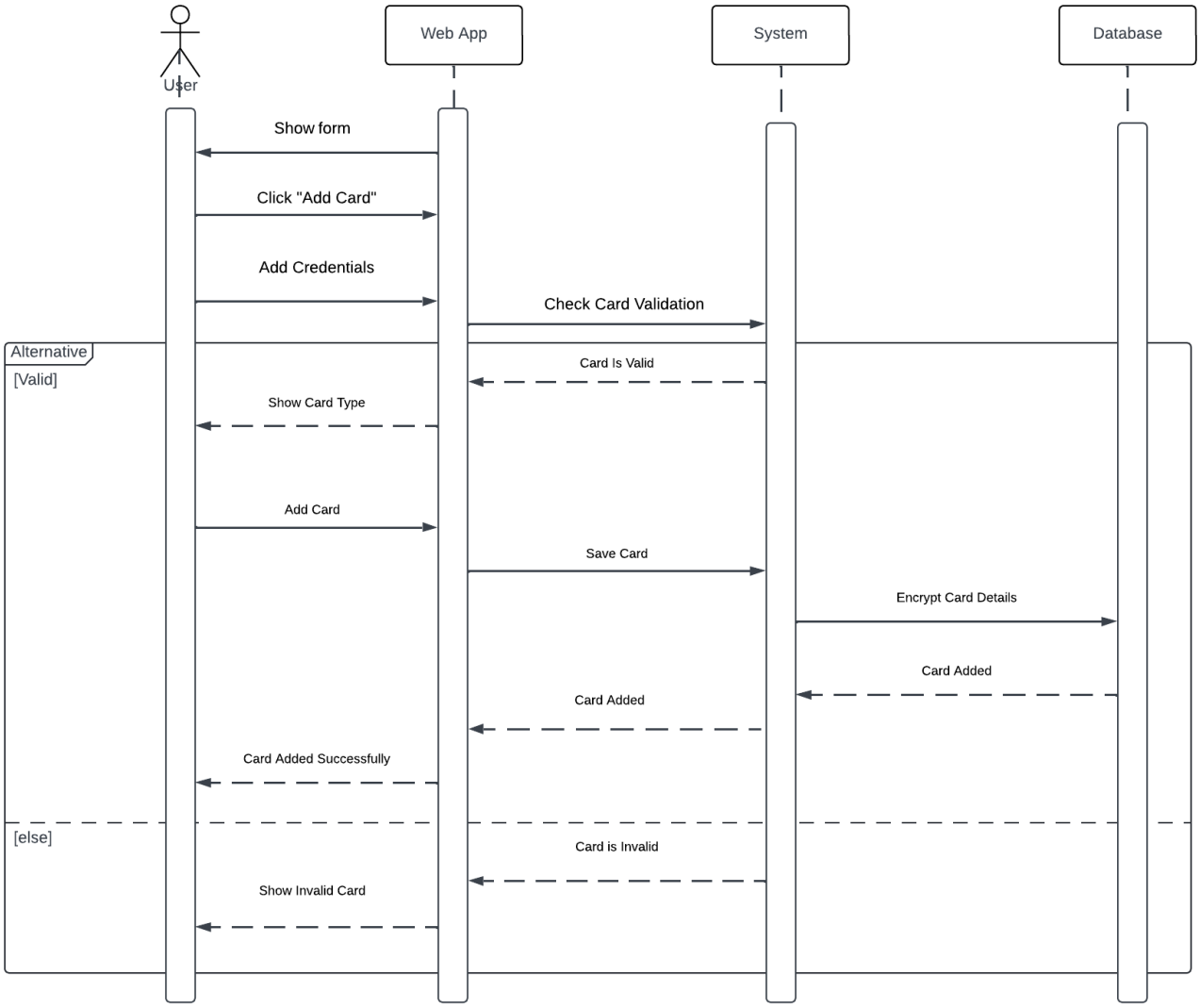


Figure 3.9: Adding Payment Method Sequence Diagram.

3.8.6 Finish Shopping & Payment Flow

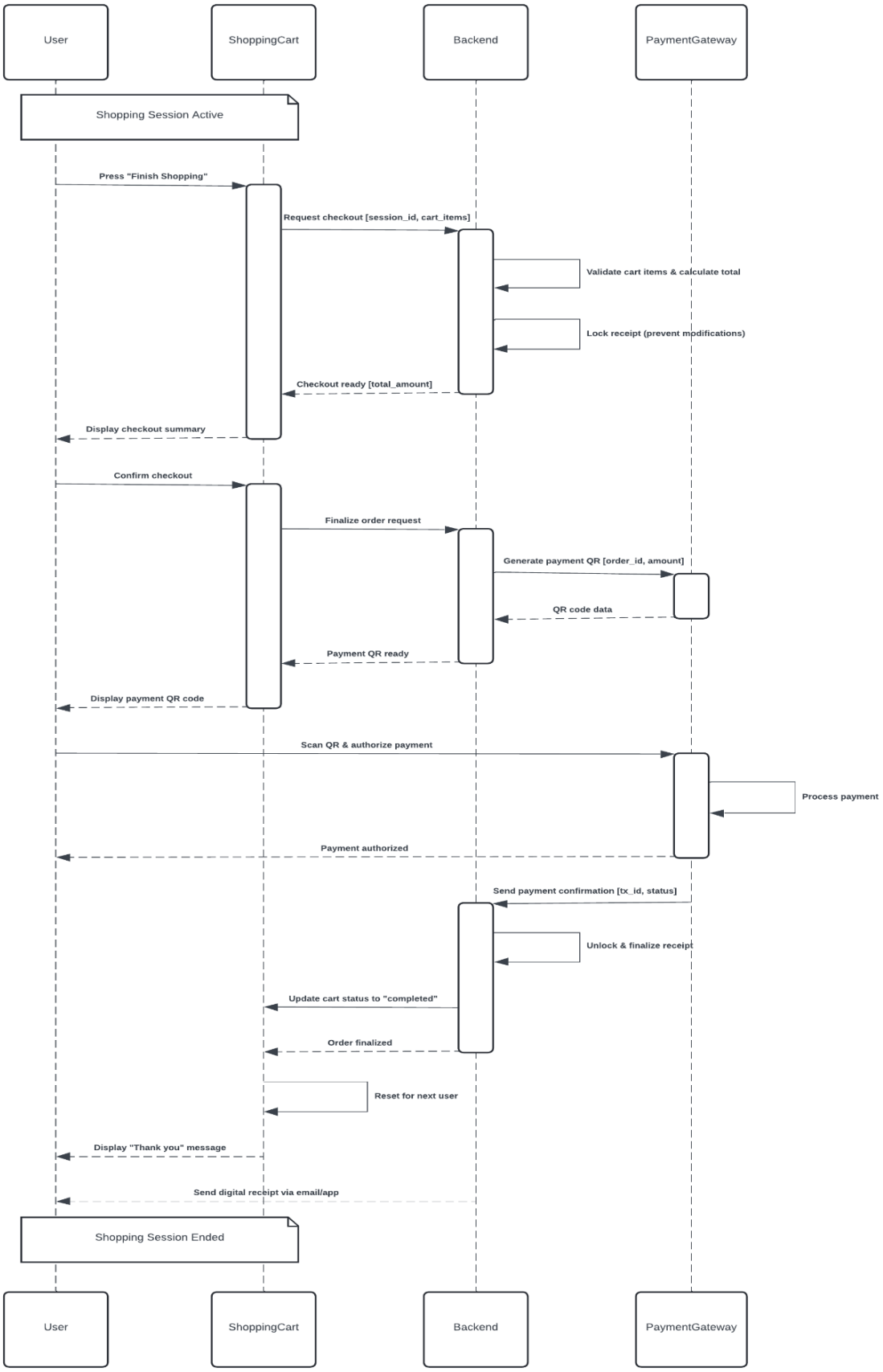


Figure 3.10: Finish Shopping and Payment Flow Sequence Diagram.

3.9 ERD

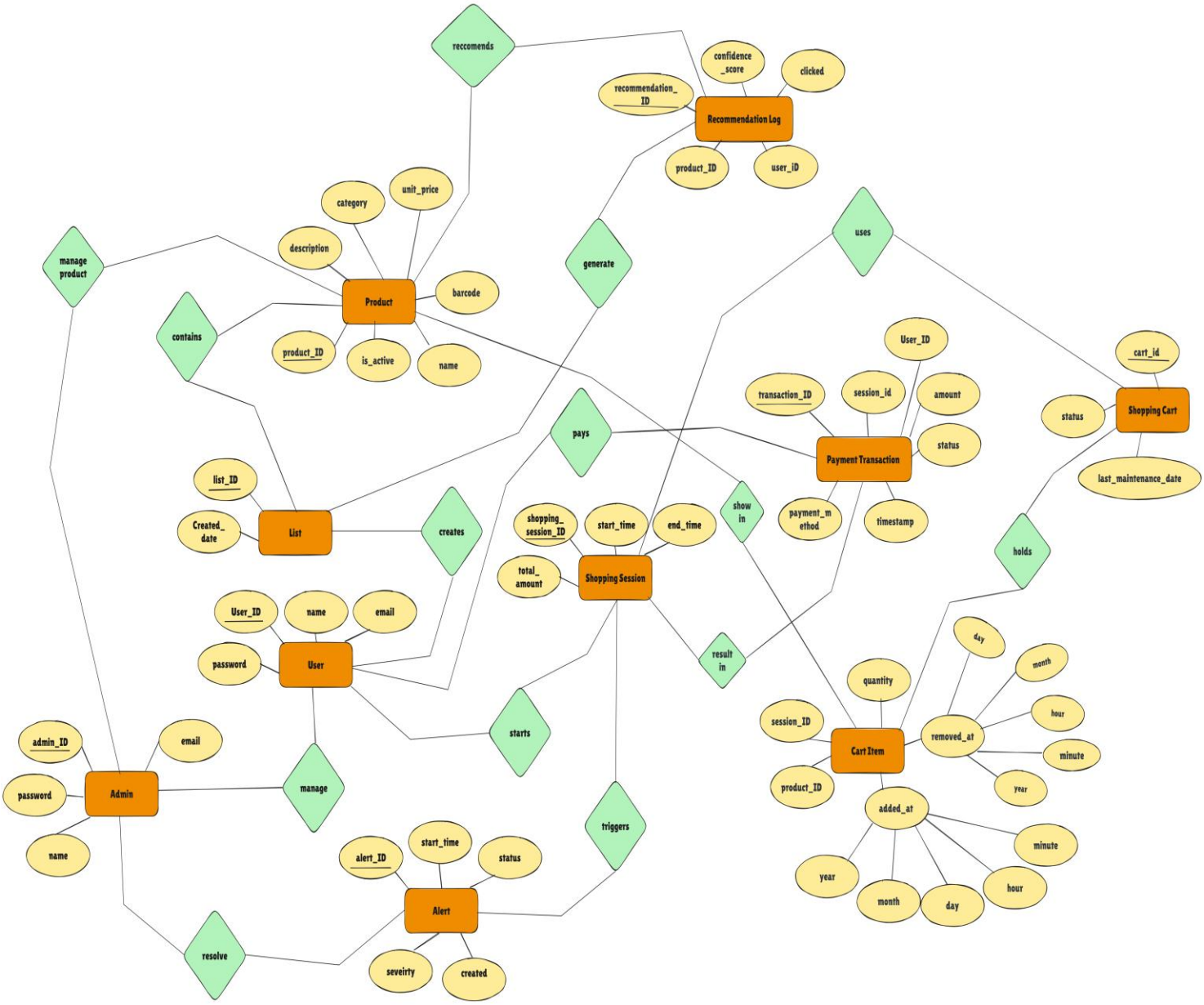


Figure 3.11: ERD.

3.10 Flowchart

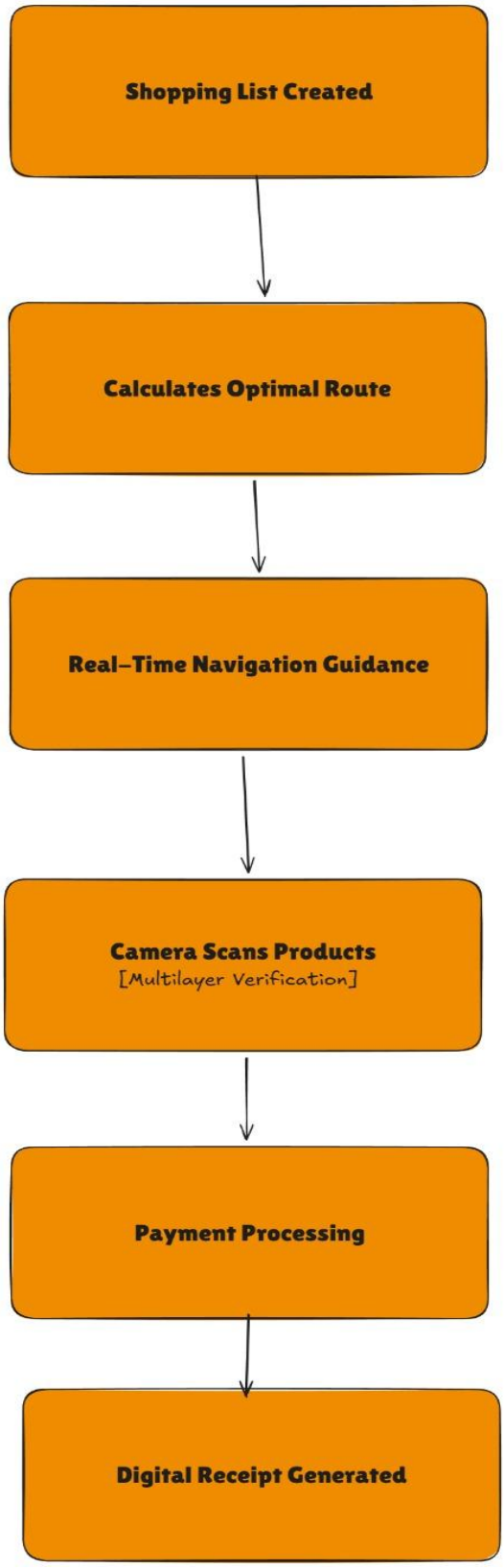


Figure 3.12: Flowchart.

Carto

CHAPTER 4

System Analysis and Design

Chapter 4: System Design

4.1 Introduction

This chapter presents the detailed system design of Carto as a thesis-level integration of four cooperating subsystems: map generation, the shopper web application, the cart screen with QR pairing and backend integration, and the AI product-recognition system. The design follows the same project scope defined in the earlier chapters but expands it into the concrete structures implemented in the three repositories and the AI documentation provided by the team. The project no longer depends on a localization module; therefore, the mapping subsystem is treated as a map-generation and map-preparation component that provides a store layout for route planning, product-zone representation, and cart-screen visualisation.

The design objective is to let a shopper prepare a list before visiting the supermarket, pair that list with a physical smart cart by scanning the cart QR code, view a readable store map and shopping progress on the cart screen, and allow camera-based product recognition to update the receipt when products are added or removed. In this architecture, the backend and database act as the source of truth. The QR code carries pairing identity only, while shopping-list, receipt, session, and product data move through authenticated APIs.

4.2 Overall Thesis-Level Architecture

Carto is designed as a layered system rather than a single application. The shopper web application is responsible for list creation, list editing, QR scanning, session start, checkout, and history. The cart screen is responsible for the physical-cart user interface: it displays a waiting QR state, receives active session data by polling the backend, shows the shopping list and map, and reflects receipt updates. The mapping subsystem uses ROS 2, LiDAR scan decoding, odometry, and SLAM Toolbox to produce a two-dimensional occupancy grid that can be transformed into a shopper-friendly map. The AI subsystem receives camera frames, detects retail products, tracks them over time, filters events using hand proximity, and emits structured add/remove events that can be mapped to receipt updates.

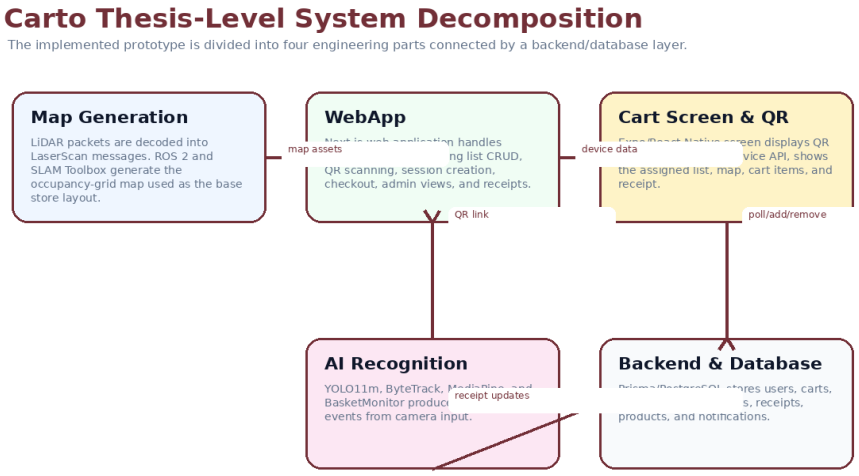


Figure 4.1: Carto Thesis-Level System Decomposition.

Table 4.1: Technology Stack by System Component.

Component	Main Technologies	Design Role
Map generation	Python, pyserial, ROS 2 Humble, sensor_msgs/LaserScan, tf2_ros, RF2O laser odometry, SLAM Toolbox, Nav2 map saver	Decodes TurtleBot3/LD06-style LiDAR data, publishes scans, builds a map, and exports the PGM/YAML occupancy grid.
Web application	Next.js 14 App Router, React 18, TypeScript, Tailwind CSS, NextAuth.js, Prisma, PostgreSQL, Zod, Zustand, QR libraries, Stripe libraries	Provides authentication, guest mode, shopping-list CRUD, QR scanning, session creation, virtual receipt, checkout, and admin screens.
Cart screen	React Native/Expo monorepo, TypeScript, cart-screen, cart-edge, shared protocol package, device API polling	Runs on the mounted smart-cart display, shows QR and shopping state, and communicates with backend device endpoints using a device secret.
AI product recognition	YOLO11m, Ultralytics, OpenCV, ByteTrack, MediaPipe Hands, FastAPI, Uvicorn, Docker	Detects products, tracks object identity, validates interaction using hands, and emits basket add/remove events.
Backend and database	Next.js API routes, Prisma ORM, PostgreSQL models, device-authenticated endpoints	Stores users, guest sessions, carts, cart sessions, shopping lists, receipt items, products, wishlist data, notifications, and admin state.

4.3 Map Generation Design

4.3.1 LiDAR Data Model and ROS Topic Design

The map-generation design begins with the raw bytes produced by the LiDAR. The custom Python node in the mapping repository is designed as a ROS 2 node named LD06Decoder. It opens a serial connection to the LiDAR, searches for the packet header, validates the packet format, extracts twelve distance-intensity measurements per packet, converts distances from millimetres to metres, assigns each measurement to a one-degree angular index, and publishes the resulting 360-value scan as a sensor_msgs/LaserScan message on the /scan topic. This design makes the custom LiDAR compatible with standard ROS mapping packages, because downstream packages receive a normal LaserScan message rather than a device-specific packet stream.

The LaserScan message uses angle_min = -pi, angle_max = pi, angle_increment = one degree, range_min = 0.12 m, and range_max = 4.0 m. Invalid or out-of-range distances are published as infinity. This representation is appropriate for indoor supermarket mapping because it gives a two-dimensional obstacle outline of shelves, walls, corridors, and large static structures. A static transform connects base_link to the laser frame with the sensor mounted approximately 0.18 m above the cart base.

4.3.2 Occupancy Grid Output and PGM Input Format

After the /scan stream is available, RF2O laser odometry and SLAM Toolbox are launched to build the occupancy grid. The SLAM configuration uses map_frame, odom_frame, base_frame, and scan_topic parameters, with a map resolution of 0.05 m per cell and loop-closing enabled. The generated map is saved using nav2_map_server map_saver_cli, producing a .pgm image and a .yaml metadata file. The YAML file defines the image path, trinary mode, resolution, origin, negate flag, occupied threshold, and free threshold. The uploaded test map follows this standard occupancy-grid format: black pixels represent occupied cells, light pixels represent free space, and grey pixels represent unknown space.

The PGM map is technically correct for robotics, but it is not directly suitable for a supermarket shopper. A raw occupancy grid is tilted according to the cart trajectory and sensor frame, uses grey-scale occupancy values, and does not describe departments or product zones. Therefore, Carto separates map generation from shopper map presentation. The robotics output remains the accurate source map, while a conversion layer transforms it into a readable interface asset for the cart screen.

4.3.3 PGM-to-Shopper-Friendly Map Conversion Design

The conversion process treats the raw PGM as an input image plus metadata. First, the pixel values are interpreted according to occupancy-grid conventions: 0 identifies occupied walls and shelf edges, 205 identifies unknown space, and 254/255 identifies free walkable space. Second, the valid occupied/free region is cropped to remove unnecessary unknown margins. Third, the main map orientation is estimated using the principal direction of occupied pixels and rotated to make the floor layout easier to read. Fourth, the map is cleaned with wall thickening and colour replacement: occupied regions are drawn in the Carto burgundy colour, free space is drawn in a light floor colour, and unknown space is kept neutral. Finally, product-zone overlays are added to represent departments such as beverages, snacks, dairy, personal care, checkout, and entrance corridor.

The conversion methodology does not change the underlying structure of the generated map. It only converts a robot-oriented occupancy grid into a user-oriented representation. Product locations can then be stored as semantic zones or aisle coordinates linked to supermarket database products. When a user list contains products assigned to these zones, the cart screen can display the route target and shopping progress without exposing the raw PGM image to the customer.

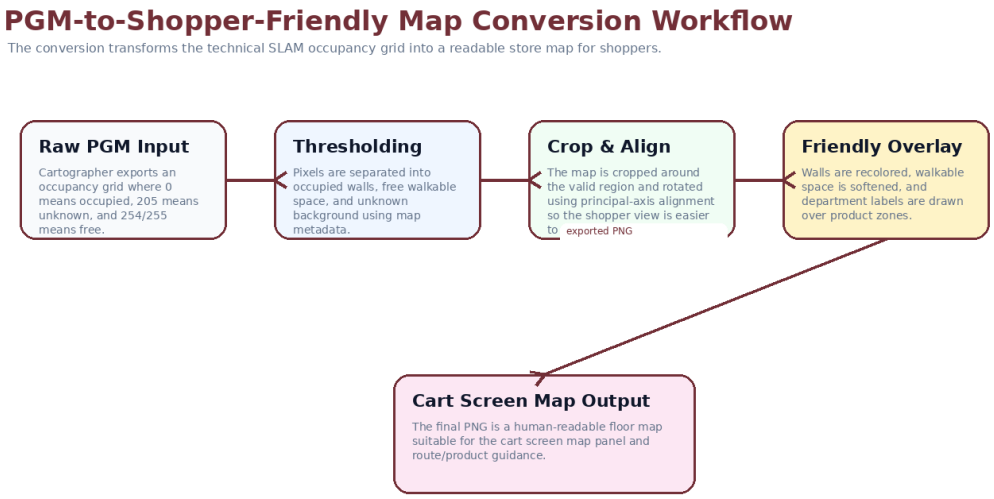


Figure 4.2: PGM-to-Shopper-Friendly Map Conversion Workflow.

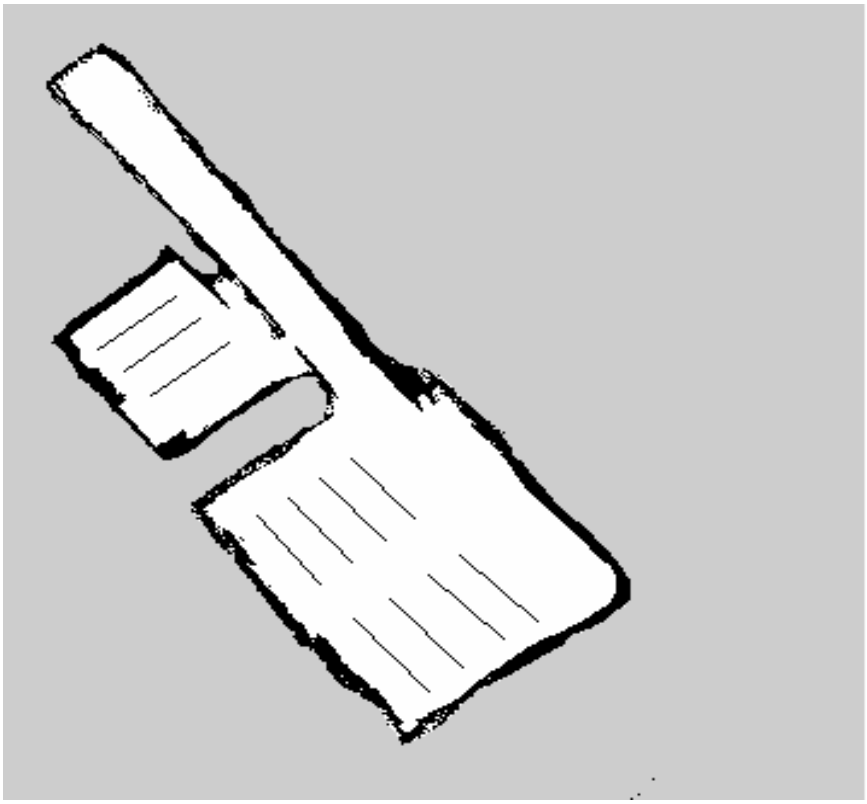


Figure 4.3: Raw PGM Occupancy Grid Used as the Map Generation Test Input.

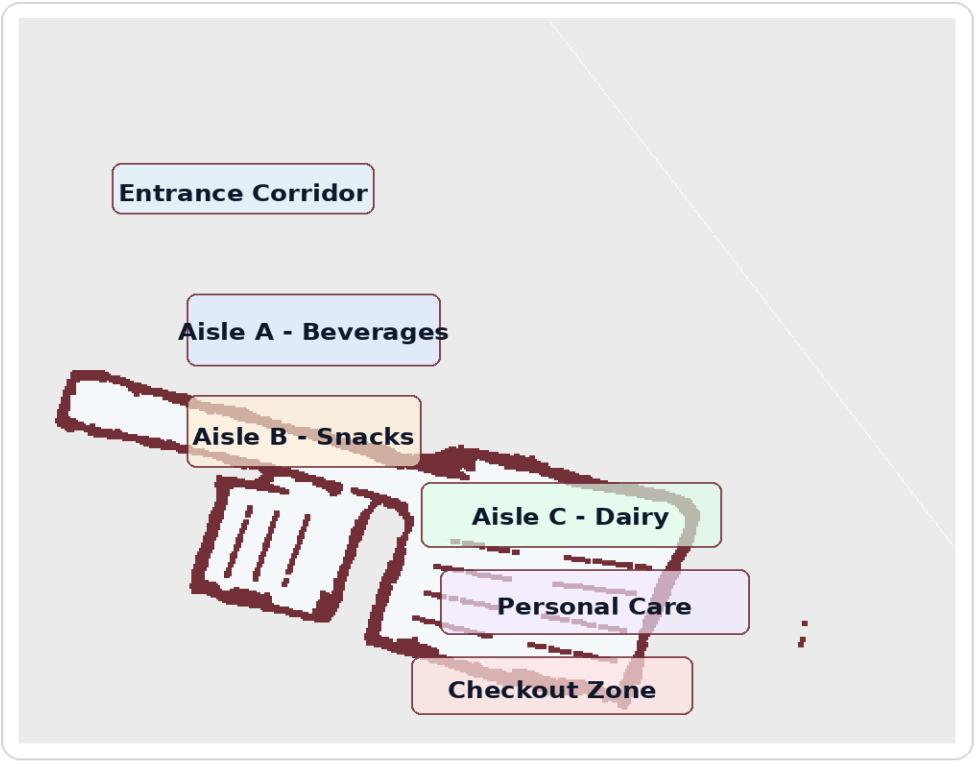


Figure 4.4: Shopper-Friendly Map Generated from the PGM Test Map.

4.4 Web Application Design

4.4.1 Web Application Pages and User Journey

The web application is the shopper-facing planning and checkout interface. The repository is a Next.js 14 application using the App Router pattern, so each folder under app/ represents a route or route group. The main journey begins at the authentication screens, where the user can sign in, sign up, use Google or phone authentication when configured, or continue as a guest. After entering the system, the dashboard displays recent lists, total spending, and a start-shopping action. The list pages implement shopping-list CRUD through /lists, /lists/new, and /lists/[id], supported by list-item management components and product search. The cart connection flow is opened through /session/start?listId=<listId>, where the QR scanner reads the cart pairing payload. After pairing succeeds, /session displays progress, cart connection state, and the virtual receipt. Checkout is completed through /checkout and /checkout/success, while /history and /admin provide receipt history and operational monitoring.

4.4.2 Web Application Screenshots and Functionality

The following web application screens document the implemented interface responsibilities. The screenshots are structured from the implemented pages and components: the sign-in screen supports normal and guest entry; the dashboard encourages the user to activate a list; the shopping-list screen supports product search and item editing; the QR pairing screen connects the selected list to a physical cart; the active-session screen shows list progress, cart status, recommendations, and receipt totals; and the admin screen summarises carts, sessions, receipts, users, guest sessions, and cart fleet status.

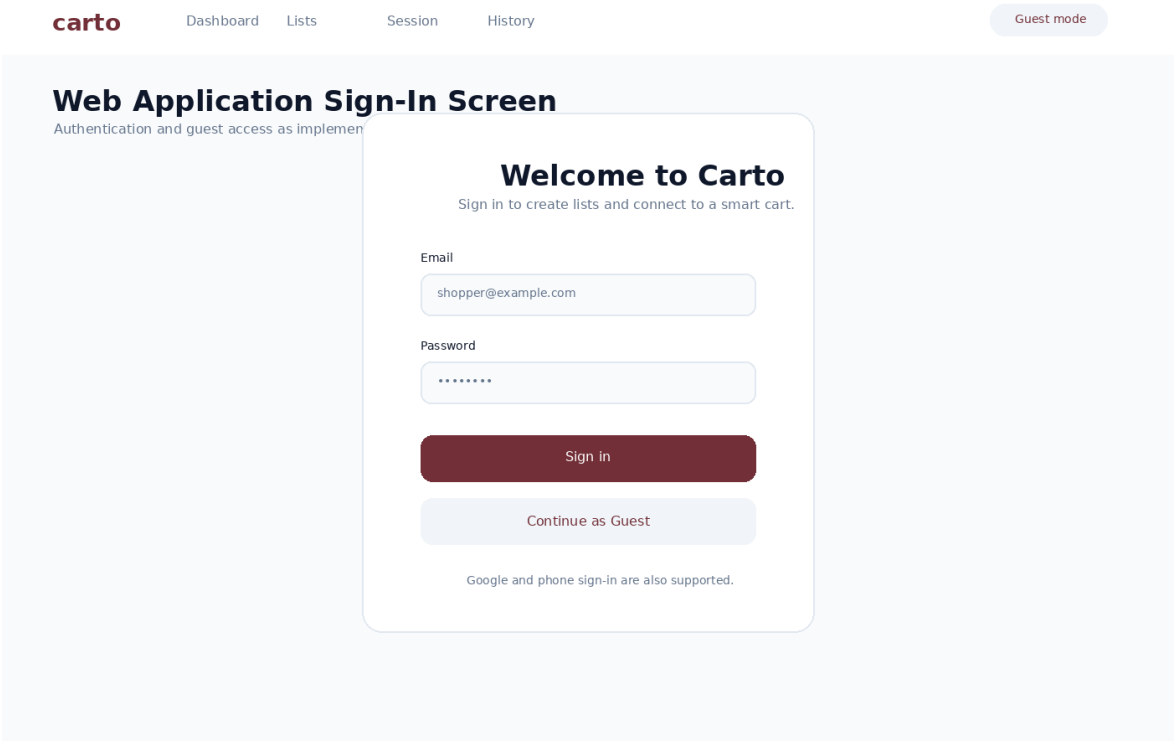


Figure 4.5: Web Application Sign-In and Guest Entry Screen.

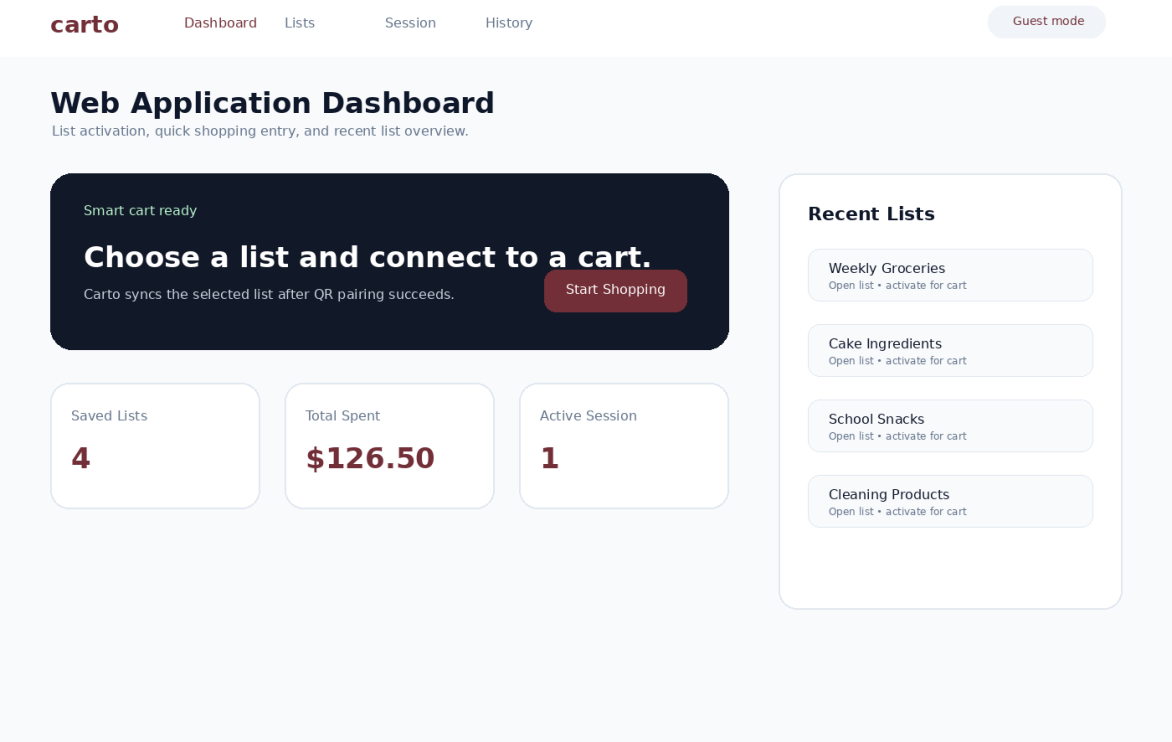


Figure 4.6: Web Application Dashboard Screen.

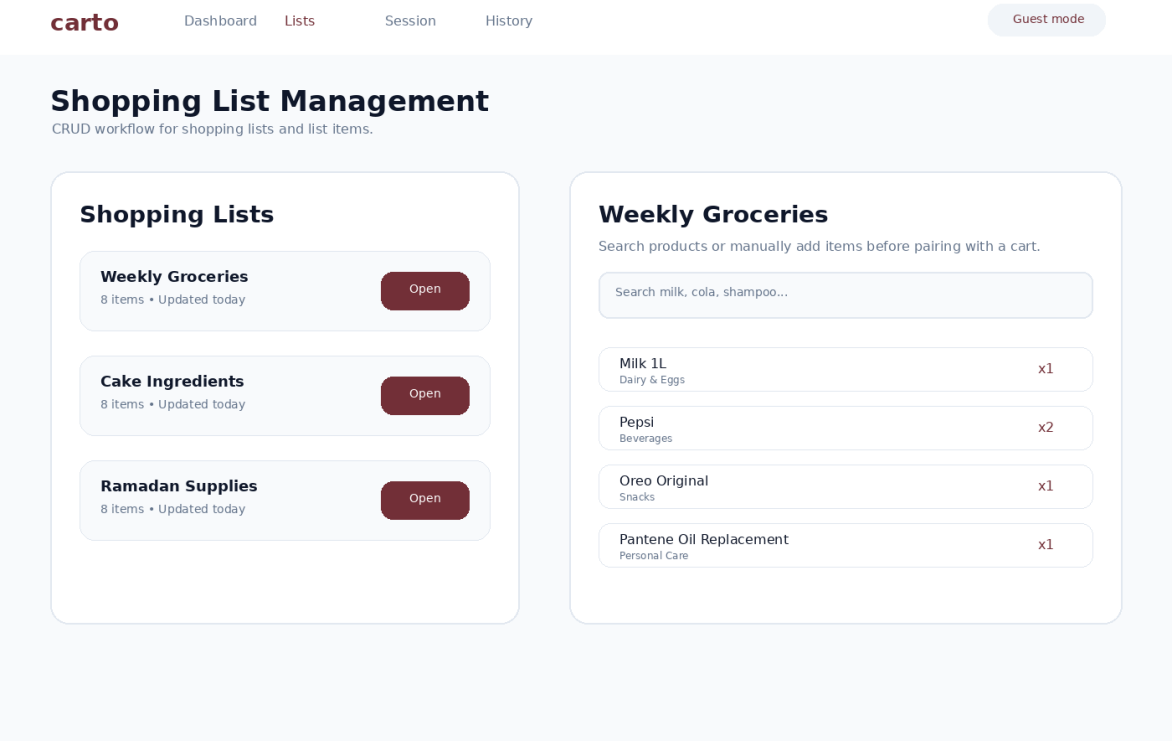


Figure 4.7: Shopping List Management Screen.

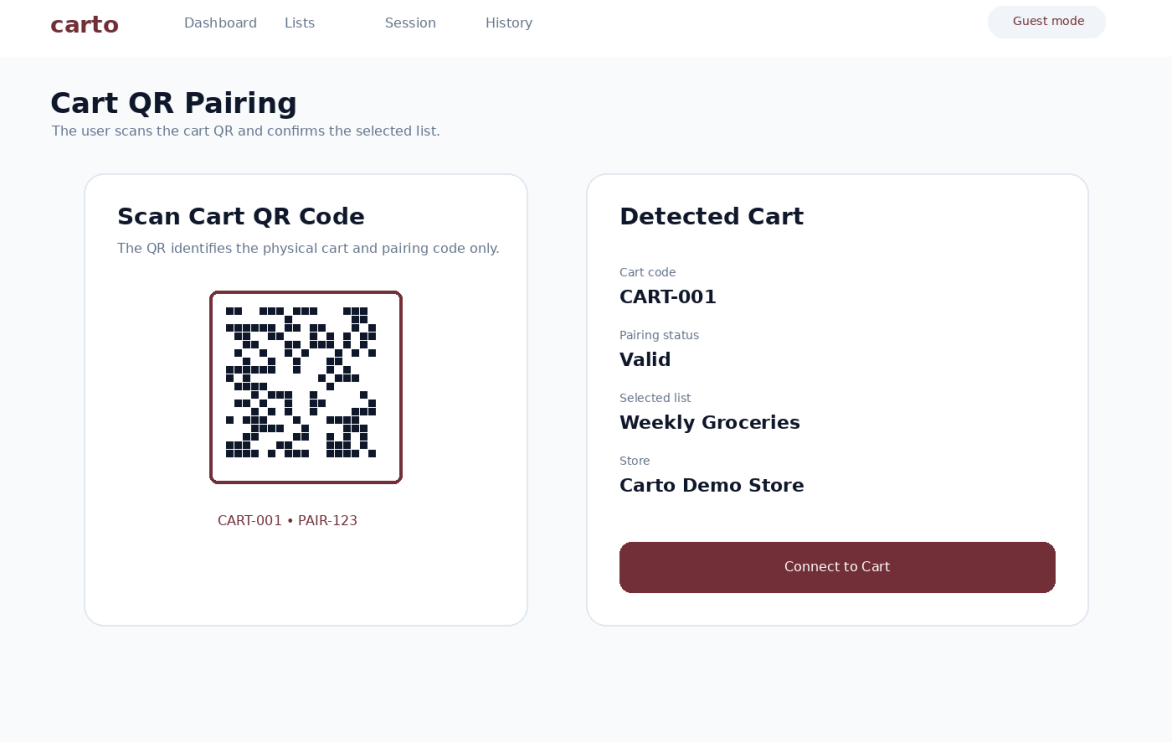


Figure 4.8: Cart QR Pairing Screen.

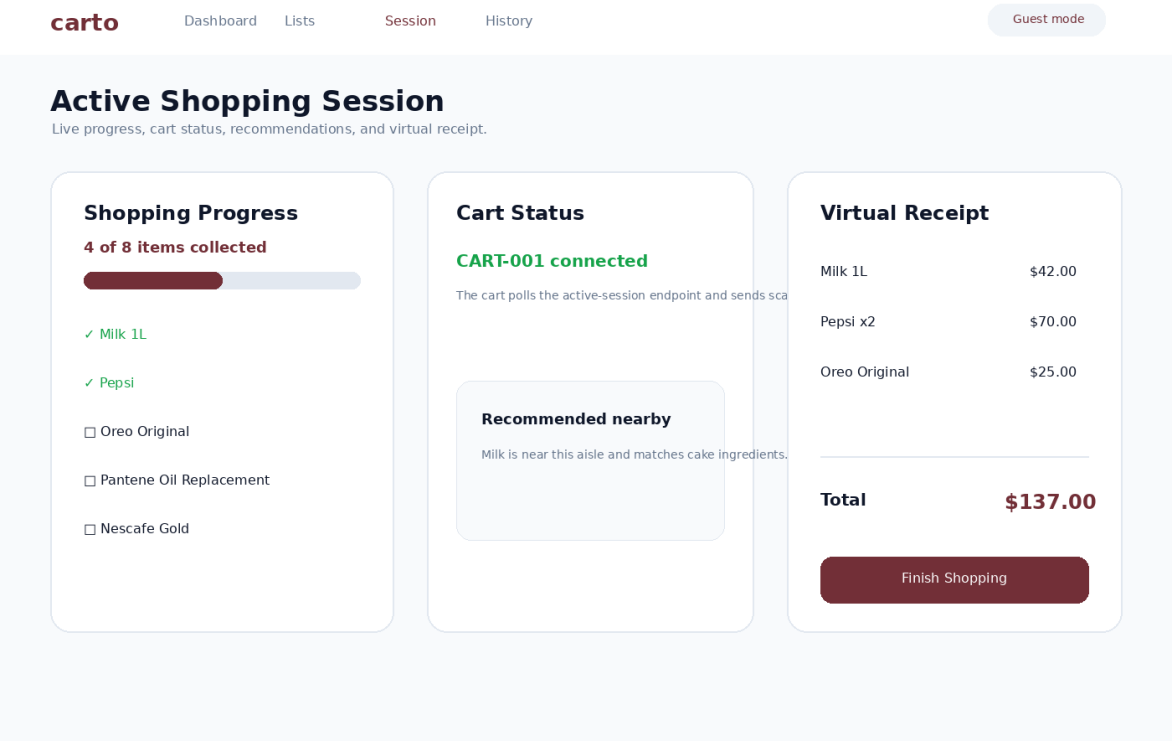


Figure 4.9: Active Shopping Session and Virtual Receipt Screen.

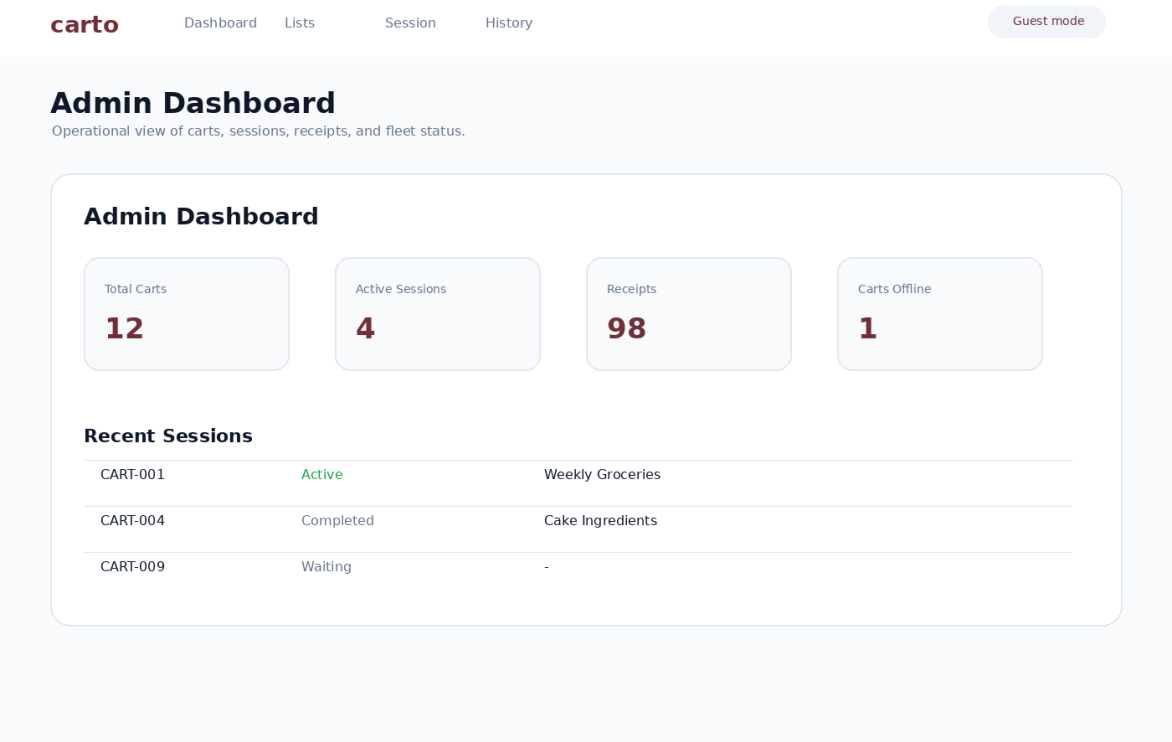


Figure 4.10: Admin Dashboard Screen.

4.4.3 Backend API and Data Model Design

The backend design is implemented through Next.js API routes and Prisma. The database schema defines the project entities required to move from list planning to cart pairing and checkout. User and GuestSession represent account-based and guest-based access. ShoppingList and ListItem hold the planned grocery list. Cart represents a physical smart cart and stores the public cartCode, pairing information, device secret, status, store relation, last-seen timestamp, and pairing expiry. CartSession connects a cart, a selected list, a user or guest session, and a receipt. Receipt and ReceiptItem store the running bill. Product, UserStats, UserFavoriteProduct, Wishlist, WishlistItem, and Notification support product search, personalisation, analytics, wishlists, and system messages.

Table 4.2: Main Database Entities and Responsibilities.

Entity	Responsibility	Main Relationships
User / GuestSession	Represents authenticated shoppers and temporary guest shoppers.	Own shopping lists, cart sessions, receipts, notifications, and statistics.
Store / Cart	Represents supermarket locations and physical carts.	A store owns carts; a cart participates in many cart sessions and has a reusable cart code.
ShoppingList / ListItem	Stores the planned grocery list before the trip.	A list belongs to a user or guest and is attached to one or more cart sessions.
CartSession	Represents the active or completed link between a shopper list and a physical cart.	Connects cart, list, user/guest, and receipt with ACTIVE, DISCONNECTED, COMPLETED, or CHECKED_OUT status.
Receipt / ReceiptItem	Stores the live and final bill.	One receipt belongs to a session and contains receipt items detected by the cart or AI pipeline.
Product / Wishlist / Notification	Supports product catalogue, preferences, wishlist, and user communication.	Products can be searched, favourited, added to wishlists, and used in recommendations.

4.5 Cart Screen, QR Pairing, and Integration Design

4.5.1 Cart Screen State Design

The cart-screen repository is a TypeScript monorepo with apps/cart-edge, apps/cart-screen, and packages/shared. It supports two modes. The edge mode preserves an older local flow in which a local edge backend and WebSocket snapshots drive the screen. The carto mode is the integration mode used by the final project. In carto mode, the cart screen communicates only with the backend device endpoints and never connects directly to the PostgreSQL database. The screen displays a QR code while the cart is idle, polls the active-session endpoint to wait for a paired shopping list, and then renders the active shopping state. The map is static and interactive: pan, zoom, and reset are supported, while robot markers, localization UI, yaw rotation, and live pose display are intentionally disabled.

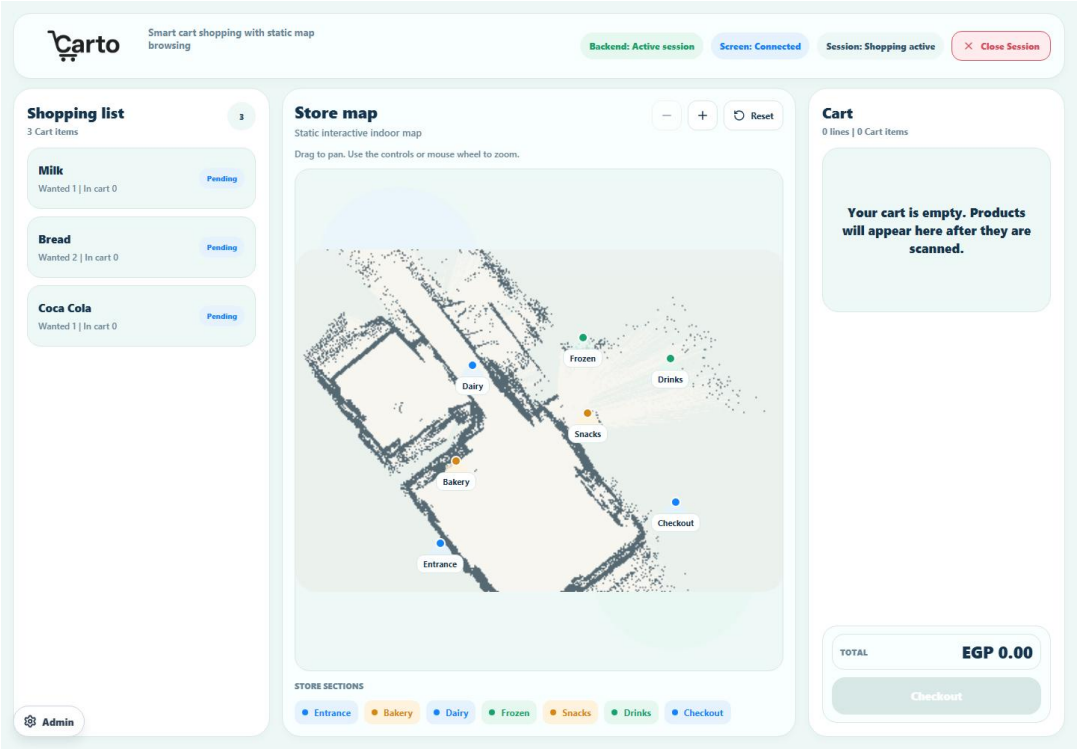


Figure 4.11: Cart Screen Waiting and QR State.

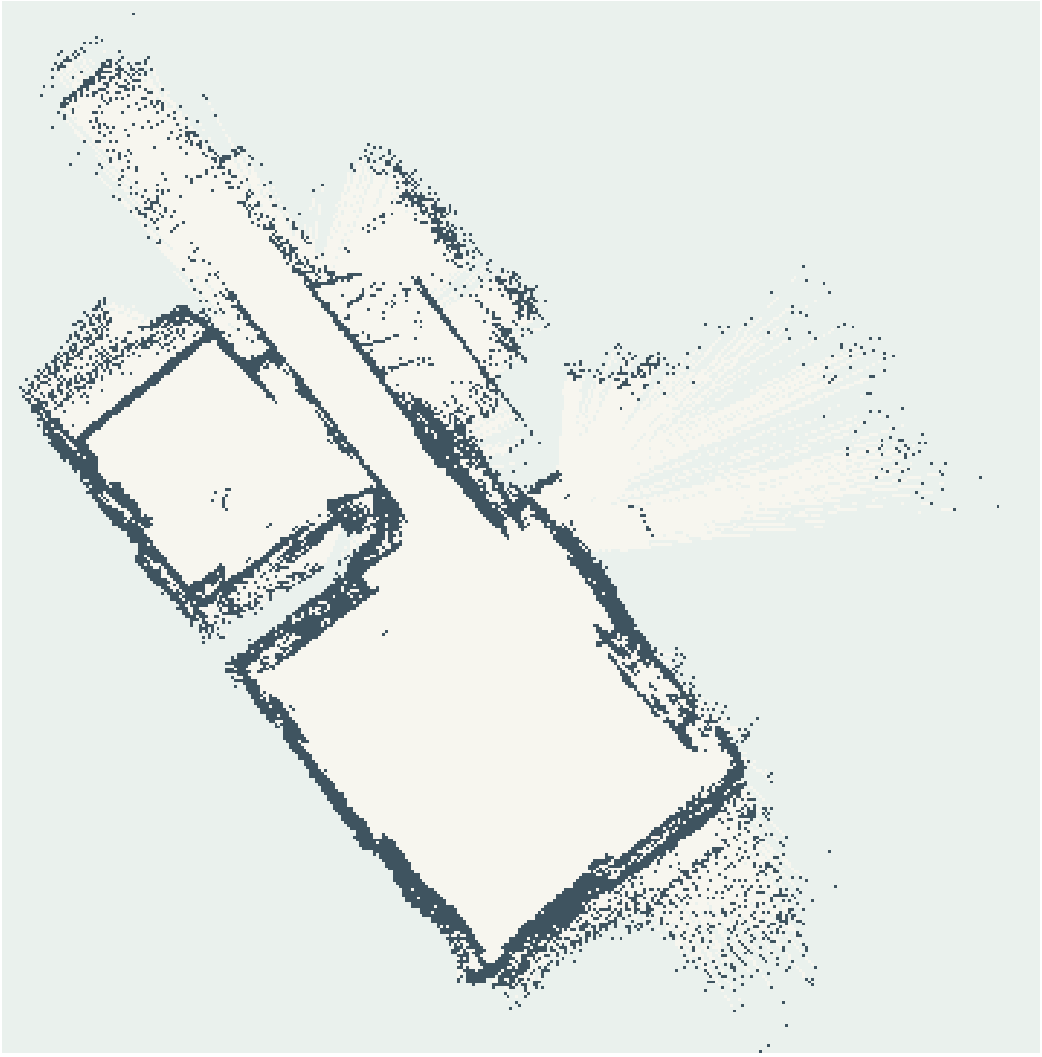


Figure 4.12: Bundled Cart Screen Static Store Map.

4.5.2 QR Pairing Contract

The QR design intentionally separates identity from data transfer. The QR displayed on the cart contains only cart pairing data such as type, cartCode, and pairingCode. It must not include the shopping list, user information, guest information, receipt data, or the device secret. After the phone or web application scans the QR, it sends the selected list ID and the scanned cart credentials to the backend. The backend validates ownership, cart existence, pairing-code correctness, pairing expiry, cart availability, and authenticated or guest identity. If validation succeeds, it creates a CartSession, creates a draft Receipt, and marks the cart as IN_USE.

Table 4.3: QR Pairing Contract and Security Rules.

Design Item	Decision	Reason
QR payload	Contains cartCode and pairingCode only.	Keeps the QR small and prevents exposure of private shopping-list or receipt data.
Device secret	Used only in Authorization bearer headers by the cart device.	Prevents a shopper QR scan from exposing device credentials.
WebApp link call	The WebApp calls POST /api/cart/link after scanning.	The backend remains the source of truth for sessions, receipts, and cart status.
Cart screen polling	The device polls GET /api/carts/[cartCode]/active-session.	The cart learns about the active session without receiving direct data from the phone.
Session reuse	Old completed sessions are allowed, while active sessions are protected.	The same physical cart can be reused after checkout without data leakage.

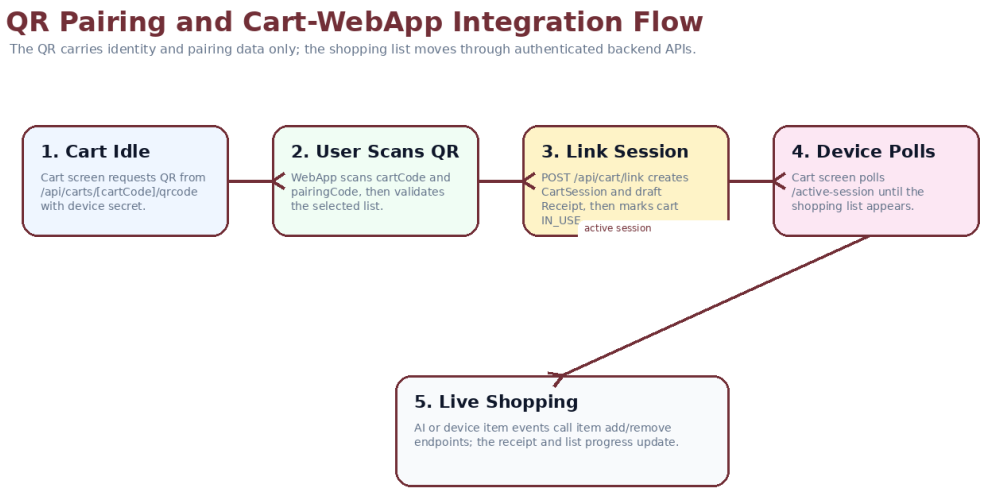


Figure 4.13: QR Pairing and Cart-WebApp Integration Flow.

4.5.3 Device API and Polling Integration

The device API design uses authenticated cart requests. A Raspberry Pi or mounted cart screen calls GET /api/carts/[cartCode]/qrcode to request the current QR, then polls GET /api/carts/[cartCode]/active-session every few seconds. A waiting response keeps the QR screen visible. An active response contains the cart session ID, receipt ID, shopping list, list items, current cart items, total, payment status, and session metadata. Product events are sent to POST /api/carts/[cartCode]/items, removal events are sent to POST/api/carts/[cartCode]/items/remove, and the trip can be finished through checkout or close-session endpoints. This polling model is simple, robust against temporary request failures, and suitable for a cart screen that must remain readable even if the network briefly drops.

4.6 AI Product Recognition Design

4.6.1 AI Problem Definition and Design Challenges

The AI subsystem is designed to solve a domain-specific retail-recognition problem. Generic object detectors trained on public datasets do not reliably recognise Egyptian supermarket products because many local packages and Arabic/English brand variants are absent from COCO, Open Images, and ImageNet. The system therefore builds a custom Egyptian retail dataset and trains a detector specifically for the target products. The design challenge is not only object detection; the system must convert visual predictions into reliable basket events, meaning it must decide whether a product was genuinely added to or removed from the cart.

The AI documentation identified six challenges that drive the design: dataset scarcity and domain specificity, high visual similarity between products in the same category, lighting and orientation variability, real-time throughput requirements, ambiguity caused by products crossing a virtual boundary, and the complexity of fusing YOLO detections with MediaPipe hand landmarks. The final design answers these challenges by combining a YOLO11m detector, ByteTrack multi-object tracking, MediaPipe Hands, EMA smoothing, confirmation frames, debounce windows, and a FastAPI backend that exposes REST and WebSocket interfaces.

Table 4.4: AI Product Recognition Objectives and Success Criteria.

Objective	Success Criterion	Measurement Method
High-accuracy product detection	mAP@50 >= 0.95	Ultralytics validation and held-out test evaluation.
Precise localisation	mAP@50:95 >= 0.70	COCO-style multi-threshold mAP measurement.
Real-time inference	<= 100 ms per frame on GPU	Profiled inference timing.
Low false-positive event rate	Less than one false event per 30 seconds	Manual session review and event inspection.
Production-ready delivery	Containerised and documented API	Docker build, endpoint tests, and API review.

4.6.2 Dataset and Product Class Design

The initial dataset contained approximately 4,000 Roboflow images across 30 labels. A cleaning audit discovered class imbalance, low intra-class diversity, annotation quality variation, missing annotations, and a duplicate class problem in which RedBull was represented by two labels. The duplicate was merged, leaving 29 final product classes. To address diversity limitations, a video collection campaign recorded products from frontal, side, and overhead angles; under fluorescent, natural, and mixed lighting; against shelves, conveyors, baskets, and plain backgrounds; with hand, product, and bag occlusions; and at distances between about 20 cm and 1.5 m. Frames were sampled every third frame, resized to 640 x 640 with letterboxing, semi-automatically annotated with a YOLO model and corrected in Roboflow, then deduplicated into 6,011 images and about 34,000 annotated boxes.

Table 4.5: Complete List of Product Classes Used by the AI Model.

Product Classes
Big Ships, Biskrem, California Garden Beans, Fine, Freska, Hohos, Lifebuoy, Maxtella, Milk, Nescafe Gold, PLYMS Tuna, Pantene Oil Replacement, RedBull, Rhodes Cheese, Herbal Essences Shampoo, Supermi Indomie, Toffifee, V Cola, Zabado, Bless Conditioner, Cadbury Dairy Milk, Herbal Essences Conditioner, Juhayna Mix Chocolate, Nivea Men Deodorant, Oreo Original, Pepsi, Pyrosol, Suntop, Tiger Chilli.

4.6.3 YOLO11m Model Architecture

YOLOv8m was first used as a baseline because modern YOLO variants provide real-time anchor-free detection. The YOLOv8 experiment showed improving losses but insufficient generalisation because the training schedule was short, the original dataset was not diverse enough, there was no explicit attention mechanism to focus on small brand details, and small-scale feature representation was weak for distant products. These root causes motivated the transition to YOLO11m. YOLO11m introduces A2C2f blocks with area-based attention, stronger multi-scale feature fusion, and better parameter efficiency. The attention module allows the backbone to weight discriminative packaging regions such as logos, product names, and small text more strongly than uninformative background regions within the bounding box.

Table 4.6: YOLO11m Model Structure Summary.

Stage	Structure	Purpose
Stem	6 x 6 convolution with stride 2 followed by 3 x 3 convolution with stride 2.	Downsamples 640 x 640 input and extracts low-level image features.
Early stages	C3k2/CSP-style blocks at high spatial resolutions.	Learns edges, colours, texture, and simple shapes.
Attention stages	A2C2f blocks with AAttn at 40 x 40 and 20 x 20 feature scales.	Focuses on discriminative product packaging details.
SPPF	Spatial Pyramid Pooling Fast at deep scale.	Aggregates global context.
PAN-FPN neck	Top-down and bottom-up multi-scale feature fusion.	Combines spatial detail and semantic context for small, medium, and large products.
Detection heads	Three decoupled anchor-free heads.	Outputs class probabilities and bounding-box distances.

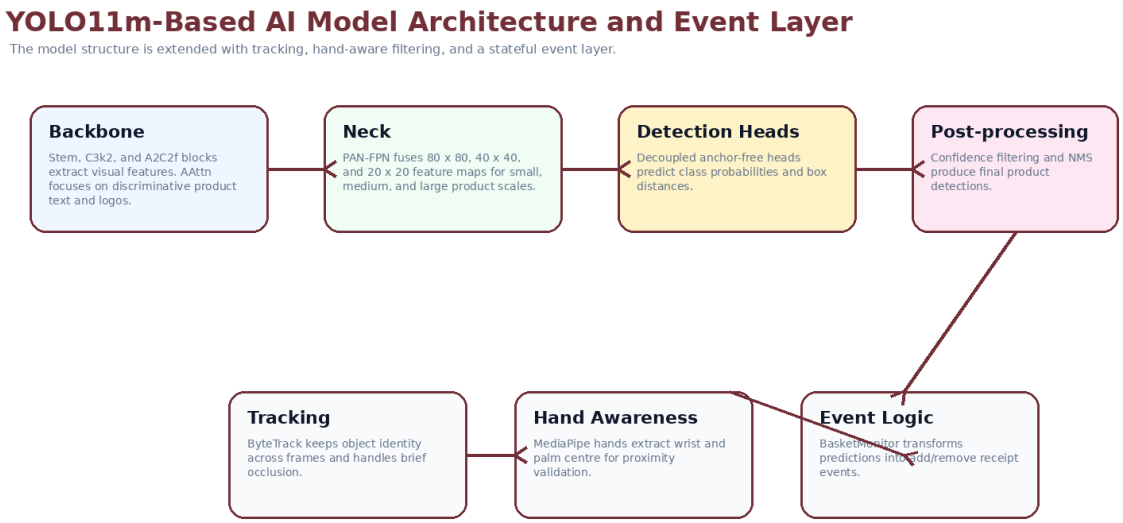


Figure 4.14: YOLO11m-Based AI Model Architecture and Event Layer.

4.6.4 Prediction Workflow and Event Generation

The AI input is a camera frame. YOLO11m returns product bounding boxes, class names, confidence scores, and box centroids. ByteTrack assigns stable track IDs so the system can reason about an object across multiple frames rather than treating each frame independently. MediaPipe Hands detects palms and regresses 21 hand landmarks, from which the wrist and palm centre are extracted. BasketMonitor then smooths centroids with an exponential moving average, determines which side of a virtual crossing line the product lies on, requires a configurable number of consecutive frames to confirm a side change, optionally requires hand proximity within 150 pixels, and applies a debounce window after an event. The result is a structured FrameResult containing tracked products, events, hand positions, basket count, and an optional annotated image.

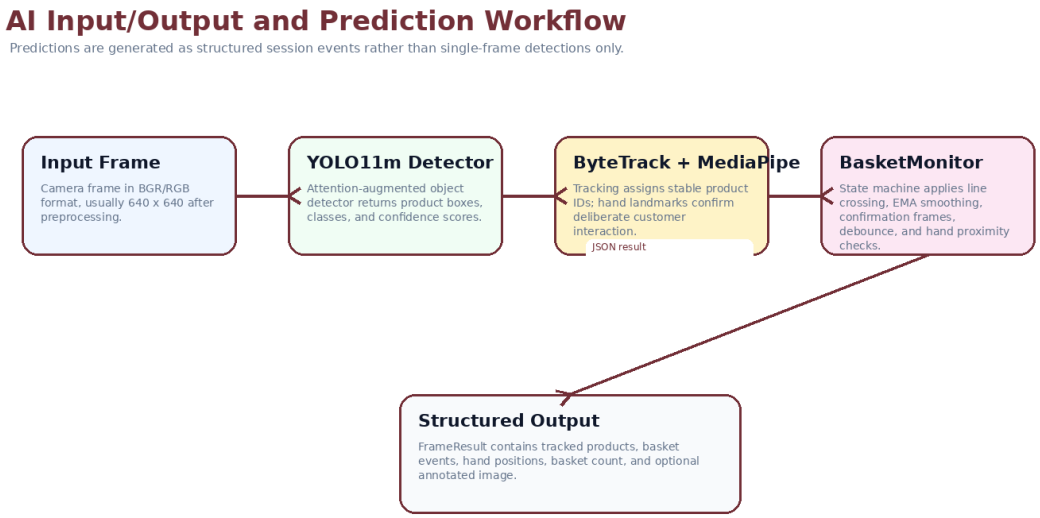


Figure 4.15: AI Input/Output and Prediction Workflow.

4.6.5 AI Backend API Design

The AI backend is designed as a FastAPI service running under Uvicorn. It follows a layered structure: routers parse HTTP and WebSocket requests, the pipeline orchestrates detection/tracking/hand/event services, service modules contain the detector and event logic, the event store keeps recent basket events, and Pydantic schemas validate every request and response. The WebSocket endpoint accepts binary JPEG frames and returns JSON FrameResult messages. A single-slot frame buffer is used so stale frames are dropped when inference is slower than the incoming camera frame rate, preventing latency from accumulating during a live shopping session.

Table 4.7: AI Backend Endpoint Reference.

Endpoint	Type	Design Function
GET /api/v1/health	REST	Reports API and model load status.
POST /api/v1/detect/image	REST	Runs stateless detection on a single image and returns boxes, classes, confidence, and optional annotation.
POST /api/v1/detect/video	REST	Processes a full video and returns annotated output plus frame statistics.
POST /api/v1/basket/configure	REST	Creates or updates the crossing line, thresholds, hand requirement, and session configuration.
POST /api/v1/basket/process	REST	Processes one frame through the stateful basket pipeline.
POST /api/v1/basket/process/video	REST	Runs end-to-end basket processing on a video.
GET /api/v1/basket/events	REST	Retrieves the latest basket events for a session.
GET /api/v1/basket/state	REST	Returns basket count, active tracks, and current configuration.
DELETE /api/v1/basket/reset	REST	Resets counters and basket count while preserving configuration.
WS /api/v1/basket/stream/{id}	WebSocket	Streams live camera frames in and FrameResult JSON out.

4.7 Integrated Database and End-to-End Data Flow

The system data flow begins before the store visit. The user creates a list in the WebApp, where it is stored as ShoppingList and ListItem records. At the store, the cart screen displays a backend-generated QR code. The user scans the QR and confirms the list. The backend creates a CartSession and draft Receipt, marks the Cart as IN_USE, and makes the selected list available to the device active-session endpoint. The cart screen then renders the list, map, receipt, and recommendations. When the cameras identify product events, the AI service produces add/remove events. These events are mapped to cart device item endpoints and then to ReceiptItem records. At checkout, the receipt is locked or marked paid, the cart session is completed, and the cart becomes available for the next shopper.

CHAPTER 5

System Implementation

Chapter 5: System Implementation

5.1 Introduction

This chapter documents how the design described in Chapter 4 was implemented in the project repositories. The implementation is divided into the same four parts used throughout this thesis: map generation, web application, cart screen and QR integration, and AI product recognition. The chapter focuses on source code structure, runtime flow, implementation decisions, and how the subsystems exchange data.

Table 5.1: Source Repository Structure by Component.

Repository	Main Folders	Implementation Responsibility
Carto_ROS_Mapping	config/, maps/, src/simple_lidar/, src/carto_pose_bridge/	ROS 2 workspace for serial LiDAR decoding, /scan publication, SLAM configuration, and saved PGM/YAML maps.
Carto-main WebApp	app/, components/, lib/, prisma/, store/, docs/	Next.js web application, API routes, Prisma schema, authentication, lists, sessions, receipts, checkout, admin, and device documentation.
carto1.1 integration and screen	apps/cart-screen/, apps/cart-edge/, packages/shared/, docs/	Cart screen interface, local edge mode, carto backend mode, shared protocol types, and device integration documentation.
AI service documentation/source	detector, hand tracker, basket monitor, pipeline, schemas, event store, Docker deployment	YOLO11m detection, ByteTrack tracking, MediaPipe hand features, stateful basket events, API schemas, and streaming backend.

5.2 Map Generation Implementation

5.2.1 Custom LiDAR Decoder Implementation

The custom LiDAR decoder is implemented in reader.py inside the simple_lidar package. The implementation uses pyserial to read binary data from /dev/ttyUSB0 at 115200 baud. It maintains a byte buffer, searches for the LD06 packet header 0x54 0x2C, and parses fixed-length 47-byte packets. Each packet contains speed, a start angle, twelve three-byte measurement points, an end angle, timestamp, and CRC byte. The implementation computes the angular difference between start and end, divides it across the twelve measurements, converts raw distance from millimetres to metres, rejects values outside the 0.12 m to 4.0 m sensor range, and maps angles into 360 scan indices corresponding to -pi to pi.

Implementation summary of the custom LiDAR decoder.

```
Packet length: 47 bytes
Header: 0x54 0x2C
Measurements: 12 points per packet
Output topic: /scan
Output type: sensor_msgs/LaserScan
Frame ID: laser
Range limits: 0.12 m to 4.0 m
Publish rate: 10 Hz
```

5.2.2 ROS 2 Mapping Runtime

The mapping runtime is launched through separate terminals to keep each component observable during debugging. The LiDAR decoder publishes /scan. A static transform publisher connects base_link to laser. RF2O laser odometry consumes /scan to estimate odometry from scan motion. SLAM Toolbox runs in online asynchronous mapping mode using slam_clean.yaml. RViz is used to monitor scans, transforms, and the evolving occupancy grid. When the map is satisfactory, nav2_map_server map_saver_cli saves the final map under the maps directory.

Table 5.2: ROS Mapping Implementation Files and Configuration.

File or Command	Main Parameters	Purpose
src/simple_lidar/simple_lidar/reader.py	port=/dev/ttyUSB0, baudrate=115200, frame_id=laser, publish_rate_hz=10	Reads LiDAR bytes and publishes /scan LaserScan messages.
tf2_ros static_transform_publisher	0 0 0.18 0 0 0 base_link laser	Defines the fixed transform between cart base and LiDAR frame.
rf2o_laser_odometry.launch.py	laser_scan_topic:=/scan	Estimates odometry from 2D laser scans.
config/slam_clean.yaml	resolution=0.05, max_laser_range=4.0, do_loop_closing=true, scan_topic=/scan	Configures SLAM Toolbox for online mapping.
nav2_map_server map_saver_cli	-f <map location and name>	Exports the final map as PGM and YAML.

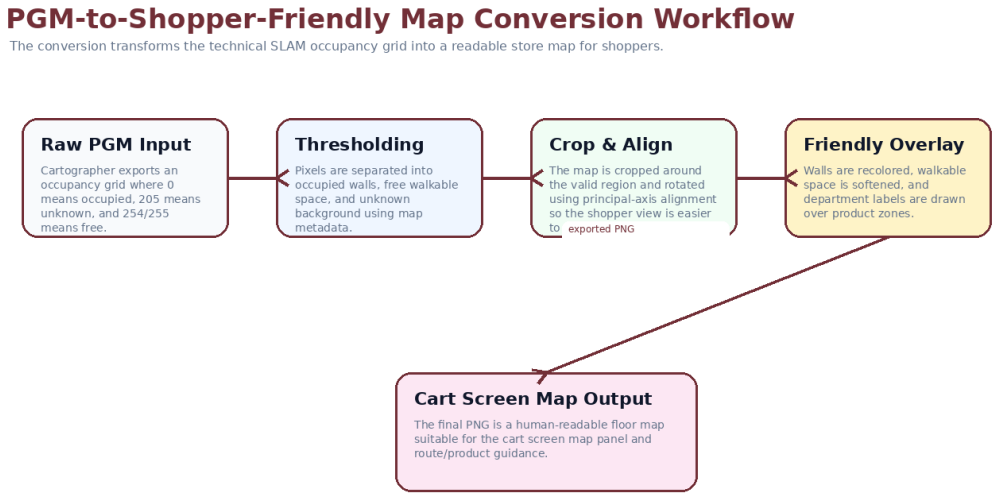


Figure 5.1: LiDAR Decoding and ROS Mapping Runtime Flow.

5.2.3 PGM Conversion Implementation

The PGM conversion implementation was tested using the uploaded khoulyz_house_edited.pgm file. The image has four pixel values: 0 for occupied cells, 205 for unknown cells, 254 for free cells, and 255 for additional free/white pixels. The conversion script loads the PGM as a grayscale image, classifies pixels by value, crops around known map content, estimates the principal direction of occupied pixels, rotates the map into a more readable orientation, thickens wall pixels, recolours walls and floor space, and overlays labelled zones. The final exported PNG is used as the test output and replaces the older map image in the documentation testing section.

5.3 Web Application Implementation

5.3.1 Repository Structure

The WebApp is implemented as a Next.js 14 project. The app directory contains route pages and API routes. The components directory contains reusable UI, layout, list, receipt, cart scanner, and admin components. The lib directory contains authentication configuration, Prisma client access, service logic, validation schemas, cart-session helpers, device API utilities, and product utilities. The prisma directory defines the schema, migrations, and seed scripts. The store directory contains client-side session state. This structure separates view rendering, domain logic, validation, persistence, and shared client state.

Table 5.3: Web Application Pages, APIs, and Responsibilities.

Area	Implemented Routes / Files	Responsibility
Authentication	/auth/signin, /auth/signup, api/auth, NextAuth configuration	Handles credentials, Google/phone readiness, protected pages, and guest access.
Shopping lists	/lists, /lists/new, /lists/[id], api/lists, ListItemsManager, ProductSearch	Creates, edits, deletes, restores, and manages list items.
Cart pairing	/session/start, components/carto/QrScanner, api/cart/link, api/carts/[cartCode]/qrcode	Reads cart QR payloads and creates cart sessions and draft receipts.
Active session	/session, api/sessions, session-store, VirtualReceipt	Polls the active session, displays progress, and refreshes the live receipt.
Checkout and history	/checkout, /checkout/success, /history, payment API routes	Locks/finishes sessions, performs payment, stores receipt history.
Admin	/admin, /admin/carts, /admin/products, /admin/sessions, admin services	Displays cart fleet, products, sessions, users, receipts, revenue, and operational state.

5.3.2 Authentication, Guest Mode, and List CRUD

Authentication is implemented with NextAuth and supporting validation logic. The sign-in page can validate credentials, handle deployment readiness states, and offer guest mode when configured. Guest mode is important for demonstrations because a shopper can use the project without creating a permanent account. Shopping-list CRUD uses API routes backed by Prisma, while client components apply optimistic updates to keep the interface responsive. When a list is locked by an active cart session, editing is restricted to prevent the shopper from changing the list in a way that conflicts with the cart screen.

5.3.3 Session, Receipt, Checkout, and Admin Implementation

The session implementation connects the list to a cart and receipt. The WebApp scans a QR payload, sends cartCode, pairingCode, and listId to the backend, and creates a live session. The active session page polls the backend every few seconds. The VirtualReceipt component polls the receipt endpoint while the receipt remains in draft state, cart recalculates subtotal, tax, and total, and changes the status badge after sending the final total to webapp to handle the rest of the payment process then changes it from live updating to ready for checkout or paid. The admin implementation aggregates operational metrics such as total carts, active sessions, completed sessions, receipts, users, guest sessions, revenue, and cart fleet status.

5.4 Cart Screen and QR Integration Implementation

5.4.1 Cart Screen Repository Structure and Modes

The cart-screen repository contains the mounted UI in apps/cart-screen, the local edge service in apps/cart-edge, and shared protocol models in packages/shared. The implementation supports edge mode for the earlier local backend and carto mode for the integrated WebApp backend. In carto mode, the screen obtains the QR from the backend, displays it to the shopper, polls the active-session endpoint, and maps the backend JSON response into the existing snapshot-driven interface. The bundled store-map-friendly.png is displayed as the static supermarket map inside the cart UI, and the screen keeps map interactivity limited to pan, zoom, and reset.

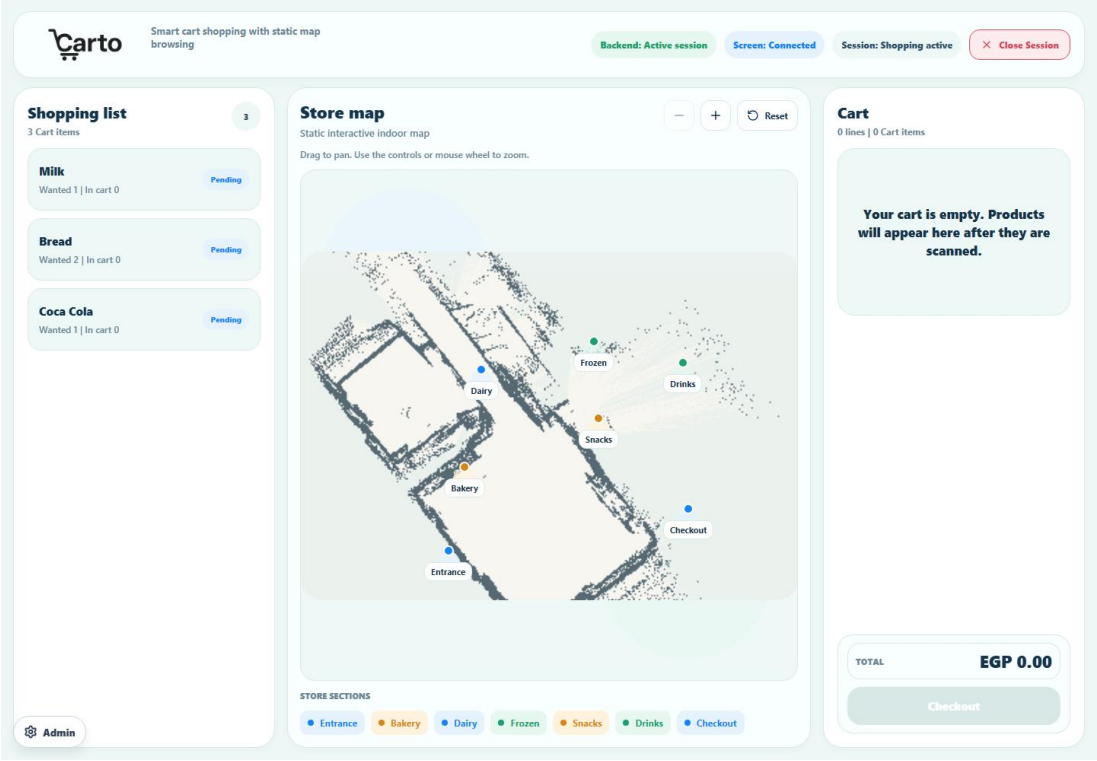


Figure 5.2: Cart Screen Interface with Shopping List, Map, and Receipt Panels.

5.4.2 Device Endpoints and Runtime Synchronisation

Device requests use the Authorization: Bearer <deviceSecret> header. The cart screen calls the QR endpoint while waiting, then calls the active-session endpoint until it receives active session data. Once active, it continues polling at a slower refresh interval so temporary failures do not erase the current session from the screen. Add-item, remove-item, checkout, close-session, and reset requests use device-authenticated endpoints. This implementation keeps the cart app independent from the database and avoids placing sensitive data in the QR code.

Table 5.4: Cart Screen Runtime Modes and Device Endpoints.

Runtime Feature	Implementation Detail	Purpose
edge mode	Uses cart-edge WebSocket snapshots and local development helpers.	Preserves the earlier local testing workflow.
carto mode	Uses backend QR, active-session polling, and authenticated device endpoints.	Integrates the cart screen with the WebApp and database-backed backend.
GET /api/carts/[cartCode]/qrcode	Device-authenticated QR generation endpoint.	Displays QR payload without shopping-list or device-secret data.
GET /api/carts/[cartCode]/active-session	Device-authenticated polling endpoint.	Moves screen from waiting state to active shopping state.
POST /items and /items/remove	Device-authenticated write endpoints.	Synchronises detected product events with receipt items.
POST /checkout and /close-session	Device-authenticated trip completion endpoints.	Completes payment simulation or closes the active session and frees the cart.

5.5 AI Product Recognition Implementation

5.5.1 Data Collection, Cleaning, and Splitting

The AI implementation started from a 4,000-image Roboflow dataset and expanded it through targeted video collection. The final dataset contains 6,011 images and approximately 34,000 annotated boxes. Every image was resized to 640 x 640 with letterboxing so that training and inference use the same scale. Machine-generated labels were produced by a preliminary YOLO model and then corrected by human annotators. Duplicate images were removed with perceptual hashing, and stratified splitting preserved product-class representation across training, validation, and test partitions.

Table 5.5: Dataset Partition Statistics After Consolidation and Cleaning.

Partition	Images	Approx. Boxes	Proportion	Purpose
Training	4,808	27,200	80%	Model parameter optimisation.
Validation	601	3,400	10%	Hyperparameter tuning and early stopping.
Test	602	3,400	10%	Final unbiased performance estimation.
Total	6,011	34,000	100%	Complete consolidated dataset.

5.5.2 Augmentation and Training Configuration

The training pipeline uses online augmentation so that the model sees a new transformed version of each image at each epoch. The augmentation design simulates the main deployment variations: colour temperature changes, saturation changes, exposure changes, small rotations, translations, scale variation, horizontal flips, mosaic composition, MixUp, RandAugment, and random erasing. Mosaic is especially important because it exposes the model to multiple objects per forward pass and improves small-object recall. It is disabled during the final 15 epochs so the model can calibrate on clean single-image inputs that match inference conditions.

Table 5.6: Data Augmentation Configuration and Technical Rationale.

Technique	Setting	Rationale
HSV hue jitter	0.015	Simulates warm/cool lighting changes while preserving product identity.
HSV saturation jitter	0.70	Covers over-saturated and desaturated camera conditions.
HSV value jitter	0.40	Handles dark aisles and reflective over-exposure.
Rotation	+/-10 degrees	Models camera tilt and handled product orientation without overly loose boxes.
Translation	+/-10%	Improves robustness to off-centre products.
Scale	+/-50%	Models distance variation between product and camera.
Horizontal flip	0.5 probability	Doubles effective data for symmetric product views.
Mosaic	1.0 probability, closed in final 15 epochs	Improves multi-object and small-product learning, then calibrates on clean images.
MixUp	0.10 probability	Regularises confidence and reduces overfitting.
RandAugment	Auto policy	Adds colour and geometric diversity.
Random erasing	0.40 probability	Simulates partial occlusion by hands or other products.

YOLO11m was initialised from COCO-pretrained weights and trained for 150 epochs with batch size 16, image size 640, AdamW optimiser, cosine learning-rate schedule, three-epoch warmup, mixed precision, eight data-loading workers, deterministic seed 42, and close_mosaic=15. The loss combines CIoU box loss, binary cross-entropy classification loss, and Distribution Focal Loss. The box loss receives the largest weight because accurate centroids are critical for crossing-line event detection.

Table 5.7: YOLO11m Training Configuration.

Parameter	Value	Purpose
Model	yolo11m.pt	COCO-pretrained YOLO11m checkpoint.
Epochs	150	Long schedule for domain adaptation.
Image size	640 x 640	Matches training and inference resolution.
Batch size	16	Fits GPU memory while preserving stable gradients.
Optimiser	AdamW	Adaptive optimiser with decoupled weight decay.
Learning rate	0.001 peak to 0.00001 final	Cosine decay for stable convergence.
Warmup	3 epochs	Protects pretrained weights during early head adaptation.
Loss weights	box=7.5, cls=0.5, dfl=1.5	Prioritises precise localisation for event logic.
AMP	Enabled	Improves GPU speed and memory efficiency.
Early stopping patience	40 epochs	Stops if mAP@50:95 stops improving.

5.5.3 Tracking, Hand Detection, and Event State Machine

The runtime AI pipeline uses YOLO11m for detection, ByteTrack for identity tracking, MediaPipe Hands for hand context, and BasketMonitor for event confirmation. MediaPipe produces 21 hand landmarks; the implementation extracts the wrist and a palm-centre feature computed from wrist and metacarpophalangeal joints. BasketMonitor receives detections and hand features, smooths each track centroid using EMA with alpha=0.4, evaluates line side by signed cross product, waits for three confirmation frames, applies a 30-frame debounce counter, and optionally requires the nearest hand feature to be within 150 pixels. This converts frame-level visual detections into high-level ADD_TO_BASKET and REMOVE_FROM_BASKET events.

Summary of the AI frame processing pipeline.

```
Frame processing order:
1. YOLO11m + ByteTrack track products in the current frame.
2. Convert BGR to RGB and extract MediaPipe hand positions.
3. BasketMonitor applies EMA smoothing, side assignment, confirmation, debounce, and hand-proximity checks.
4. Events are pushed to the event store.
5. The API returns FrameResult with products, hands, events, basket_count, and optional annotated image.
```

5.5.4 AI Service Deployment

The AI service is packaged as a FastAPI application with Docker support. The model checkpoint is stored as models/best.pt and loaded at application startup. REST endpoints share a stateless model instance, while WebSocket sessions use isolated tracking model instances because ByteTrack stores track state inside the model instance. Inference is offloaded to a thread pool to keep the asyncio event loop responsive. Annotated frames are JPEG-compressed before base64 encoding to reduce WebSocket bandwidth. Tracks that have not been seen for 45 frames are evicted to prevent memory growth in long sessions.

Table 5.8: Model Export Format Comparison.

Format	Relative Latency	Compatibility	Notes
PyTorch .pt	Baseline	Python	Default checkpoint, easiest to update.
TorchScript	about 0.9x	Python and C++	Platform-independent export.
ONNX	about 0.7x	Universal	Compatible with TensorRT, OpenVINO, and Core ML.
TensorRT FP16	about 0.3x	NVIDIA GPU	Uses graph fusion and FP16 quantisation.
TensorRT INT8	about 0.2x	NVIDIA GPU	Requires calibration and offers minimal accuracy loss.
OpenVINO	about 0.5x	Intel CPU/GPU	Optimised for Intel deployment.

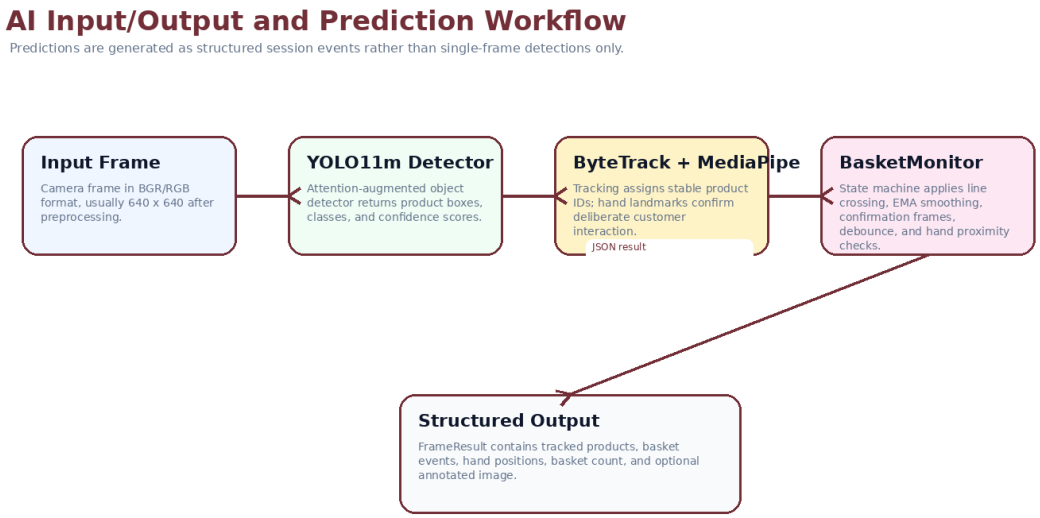


Figure 5.3: AI Backend Layered Architecture.

5.6 End-to-End Integration Implementation

The final implementation connects the four subsystems through backend contracts. The map-generation repository supplies the store map asset and map metadata. The WebApp creates the shopping list and performs the QR scan. The backend creates the active session and draft receipt. The cart screen polls for session data and displays the list, map, and receipt. The AI service produces product events that can be mapped to device item endpoints. When shopping ends, checkout finalises the session and frees the cart for reuse. This implementation is intentionally modular: each component can be tested independently, but the full value appears when they operate as one shopping workflow.

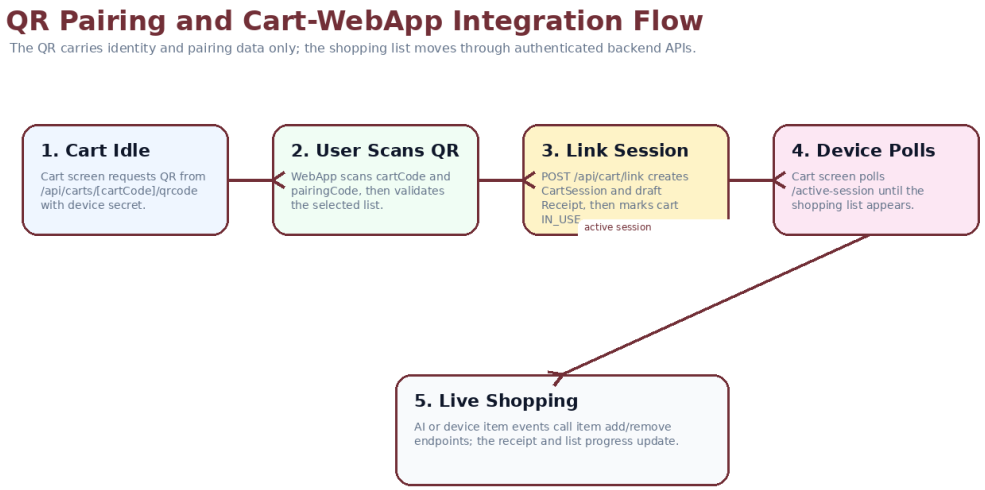


Figure 5.4: End-to-End Integrated Shopping Flow.

Carto

CHAPTER 6

System Testing and Results

Chapter 6: System Testing and Results

6.1 Introduction

Testing was moved into a dedicated chapter to separate implementation details from validation results. The test plan covers the same four subsystem areas: map generation, the WebApp, cart screen and QR integration, and AI product recognition. The purpose of this chapter is to show expected versus actual behaviour, pass/fail status, and discussion of outcomes. The AI section also includes the complete set of charts and graphs from the AI documentation so that the final report documents both numerical performance and visual model behaviour.

6.2 Testing Strategy

Testing combined functional tests, integration tests, map-output inspection, and AI evaluation. Functional tests verify that individual features such as list creation, QR scanning, cart polling, receipt updates, and checkout behave as expected. Integration tests verify that data moves correctly across repositories. Map tests verify that the generated PGM can be converted to a user-friendly map. AI tests use held-out validation and test data, experiment logs, confusion-matrix analysis, precision-recall curves, confidence curves, and qualitative validation samples. The expected result for the whole system is a complete shopping journey from list creation to cart pairing, product recognition, receipt update, and checkout.

6.3 Map Generation Testing and Results

6.3.1 PGM Conversion Test

The map-generation test uses khoulyz_house_edited.pgm as the input map. The raw file has size 358 x 363 pixels and contains the occupancy values 0, 205, 254, and 255. This confirms that the map is a trinary occupancy grid compatible with ROS map-server conventions. The conversion test checks whether the raw robotics map can be read, classified, cropped, aligned, recoloured, labelled, and exported as a shopper-friendly PNG suitable for the cart screen.

Table 6.1: Map Conversion Test Case and Outcome.

Test Step	Expected Result	Actual Result	Status
Load PGM input	The image opens as a grayscale occupancy grid.	The file opened successfully with dimensions 358 x 363 and four pixel values.	Pass
Classify pixels	Occupied, free, and unknown cells are separated.	0 was treated as occupied, 205 as unknown, and 254/255 as free.	Pass
Crop and align	The useful map region is isolated and rotated into a readable orientation.	The map was cropped around valid pixels and aligned using the principal occupied-pixel direction.	Pass
Visual conversion	Walls, floor, and product zones are visible to shoppers.	A labelled shopper-friendly map was generated with aisle and department overlays.	Pass
Cart-screen suitability	The final output can be inserted as a map panel asset.	The output is a PNG with readable labels and clear obstacle/floor separation.	Pass

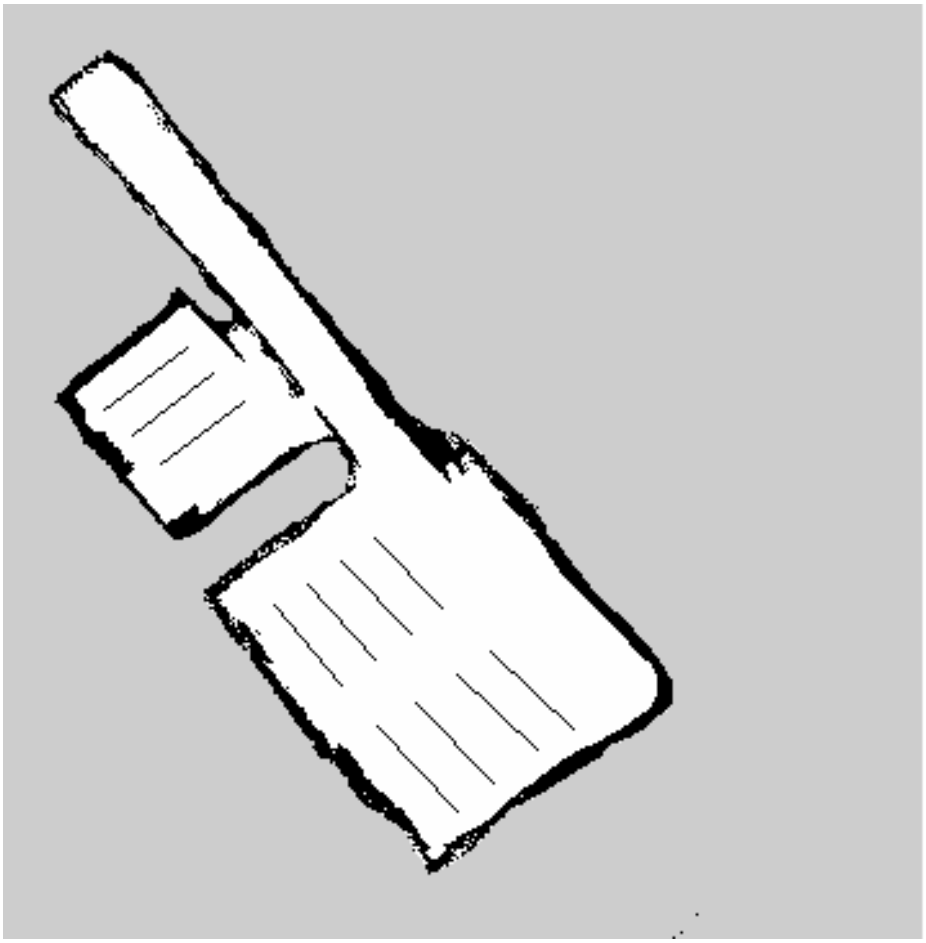


Figure 6.1: Map Generation Test - Raw PGM Occupancy Grid.

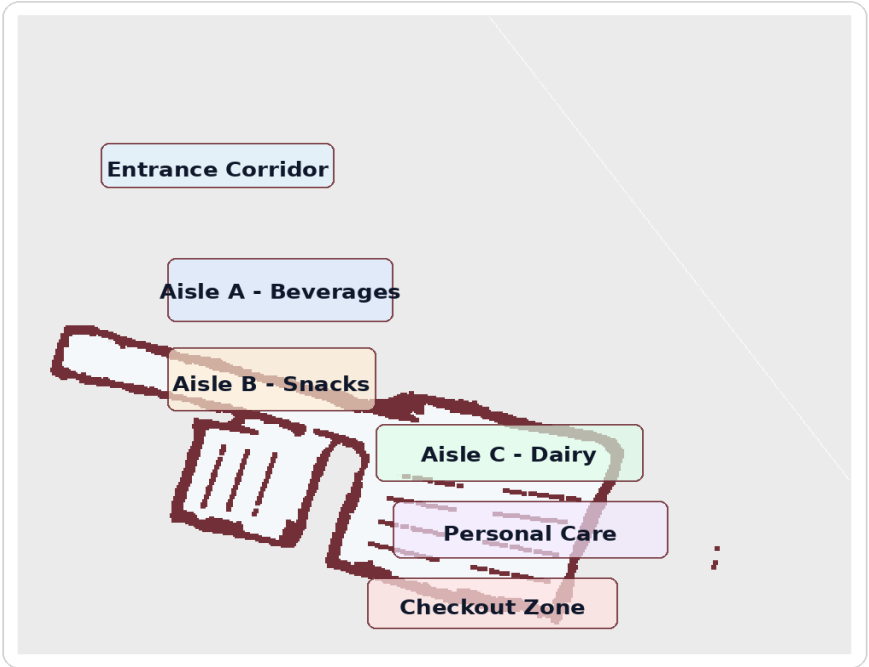


Figure 6.2: Map Generation Test - Shopper-Friendly Output.

6.3.2 Discussion of Mapping Outcomes

The mapping outcome proves that the SLAM-generated PGM can be transformed into a presentation layer for shoppers. The raw map preserves the geometric information required by robotics and route computation, while the friendly output supports human interpretation. The main limitation is that department labels and product zones are currently overlaid as semantic annotations rather than automatically extracted from the map. In a supermarket deployment, these labels should be connected to the store product database so every product has an aisle or zone identifier used for path planning and recommendations.

6.4 Web Application Testing and Results

The WebApp tests validate the user-facing workflow: account or guest entry, list CRUD, QR scan, session creation, virtual receipt display, checkout, and admin monitoring. The tests are based on implemented pages and documented API behaviour. They confirm that the WebApp acts as the shopper control interface and that session state is persisted in the backend rather than transmitted through the QR code.

Table 6.2: Web Application Test Cases.

Test Case	Expected Result	Actual Result	Status
Sign in or continue as guest	User can enter the application through credentials or guest mode.	The auth pages implement credentials handling, readiness checks, and guest bypass flow.	Pass
Create shopping list	A new list is created and displayed in /lists.	ShoppingList and ListItem routes support list creation and display.	Pass
Add and edit list items	Items can be added, quantities changed, and items removed.	ListItemsManager implements item add, update, delete, and optimistic feedback.	Pass
Scan cart QR	The QR scanner reads cart pairing data and validates it before linking.	QrScanner and /session/start parse QR payload and submit /api/cart/link.	Pass
Create active session	Backend creates CartSession and draft Receipt.	/api/cart/link stores or updates cart, creates active session, and creates a receipt.	Pass
Display virtual receipt	Receipt items and totals appear during active shopping.	VirtualReceipt polls receipt data and recalculates subtotal, tax, and total.	Pass
Checkout flow	Finishing shopping locks or completes the receipt and moves to checkout.	Session finish and checkout pages support payment and success navigation.	Pass
Admin dashboard	Admin can view carts, sessions, receipts, users, and fleet status.	Admin pages and services aggregate operational data for dashboard cards and tables.	Pass

6.5 Cart Screen, QR Pairing, and Integration Testing

The cart-screen and QR tests verify that the physical cart can wait for pairing, expose a safe QR code, receive an active shopping list from the backend, display the map and receipt, and send item events. The main integration rule is that the cart screen does not call /api/cart/link and does not access the database directly; it only uses device-authenticated endpoints and polling.

Table 6.3: Cart Screen and QR Integration Test Cases.

Test Case	Expected Result	Actual Result	Status
Idle cart QR display	The cart screen displays a QR code containing pairing data only.	The carto mode requests QR from the backend and displays data.qrValue.	Pass
QR security	QR does not include shopping list, user data, receipt data, or device secret.	Repository documentation explicitly forbids those fields in the QR payload.	Pass
Active-session polling	Waiting response keeps QR visible; active response loads list and receipt.	The screen polls active-session and maps shoppingList/list fallbacks to UI state.	Pass
Temporary backend failure	A temporary poll failure should not clear an already active session.	Carto mode preserves active session data until the backend explicitly returns waiting.	Pass
Add item event	Detected item is sent to POST /api/carts/[cartCode]/items.	Device endpoint supports receipt item addition by name, price, quantity, and category.	Pass
Remove item event	Removed product decreases receipt quantity or removes item line.	Device endpoint supports item removal by name or productId with quantity.	Pass
Close or checkout	The cart can close the session or perform mock device checkout.	Device endpoints support checkout and close-session, freeing the cart for reuse.	Pass

6.6 AI Product Recognition Testing and Results

6.6.1 Controlled Experiments

The AI experiments isolate the effects of architecture, dataset size, and training duration. YOLOv8m trained on the 4,000-image dataset for 15 epochs established the baseline and revealed insufficient convergence. Extending YOLOv8m to 50 epochs improved performance but showed overfitting. Expanding the dataset to 6,000 images created the largest improvement. Switching to YOLO11m improved fine-grained product discrimination, and extending YOLO11m to 150 epochs produced the final model that met all objectives.

Table 6.4: Complete AI Experiment Log.

Exp.	Model	Dataset	Epochs	Precision	Recall	mAP@50	mAP@50:95	Key Finding
E-1	YOLOv8m	4k	15	0.887	0.851	0.893	0.621	Baseline; insufficient epochs.
E-2	YOLOv8m	4k	50	0.912	0.878	0.921	0.648	Longer training helps but still overfits.
E-3	YOLOv8m	6k	100	0.935	0.921	0.947	0.682	Dataset expansion is the dominant gain.
E-4	YOLO11m	6k	50	0.941	0.935	0.952	0.687	Architecture advantage becomes visible.
E-5	YOLO11m	6k	100	0.956	0.961	0.968	0.703	YOLO11m continues improving.
E-6	YOLO11m	6k	150	0.964	0.970	0.973	0.711	Final model; all objectives met.

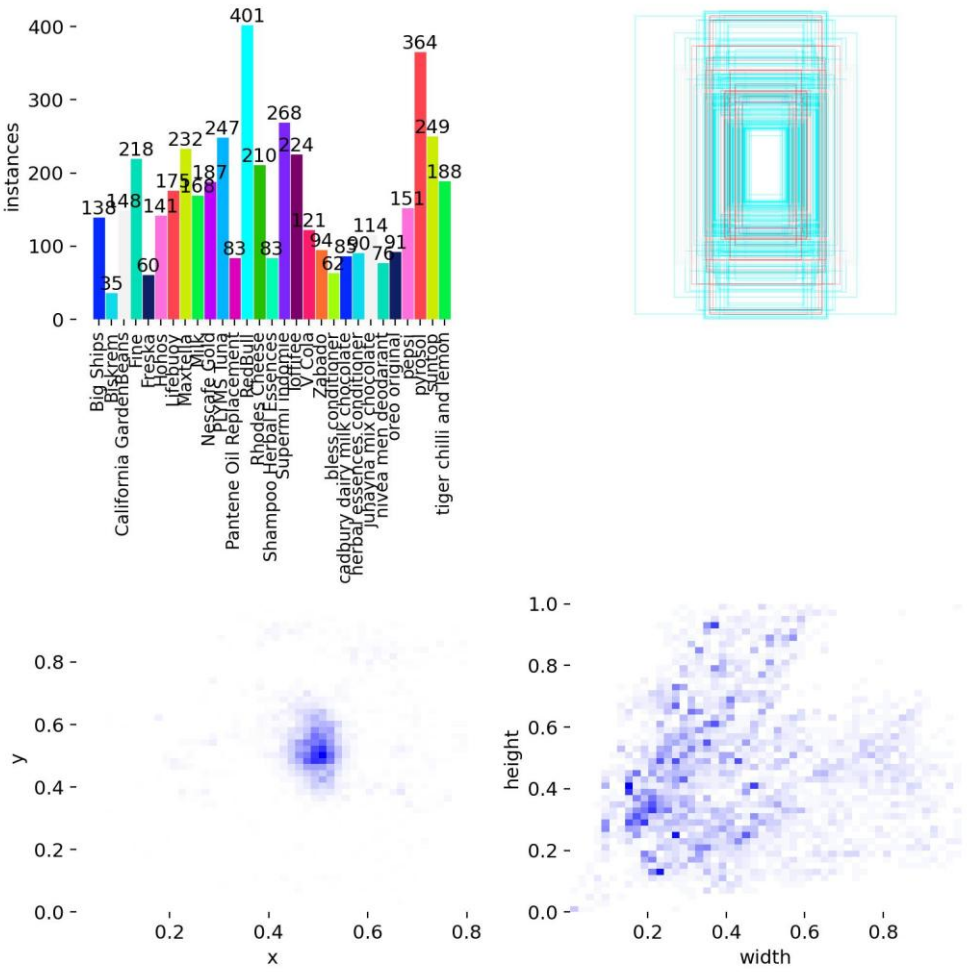


Figure 6.3: Class Distribution Across the Consolidated AI Product Dataset.

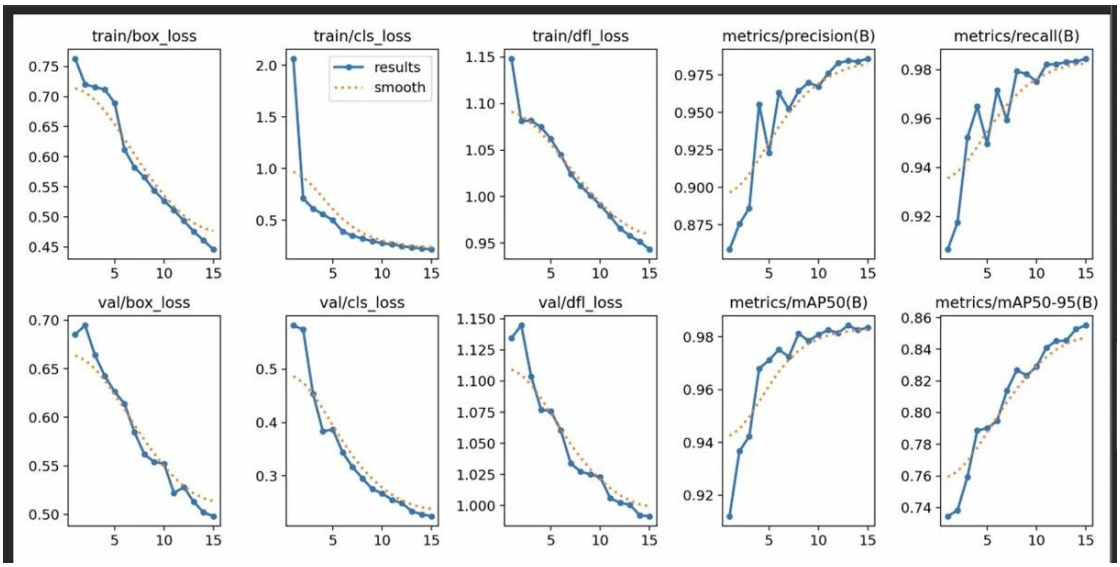


Figure 6.4: YOLOv8m Baseline Training Curves.

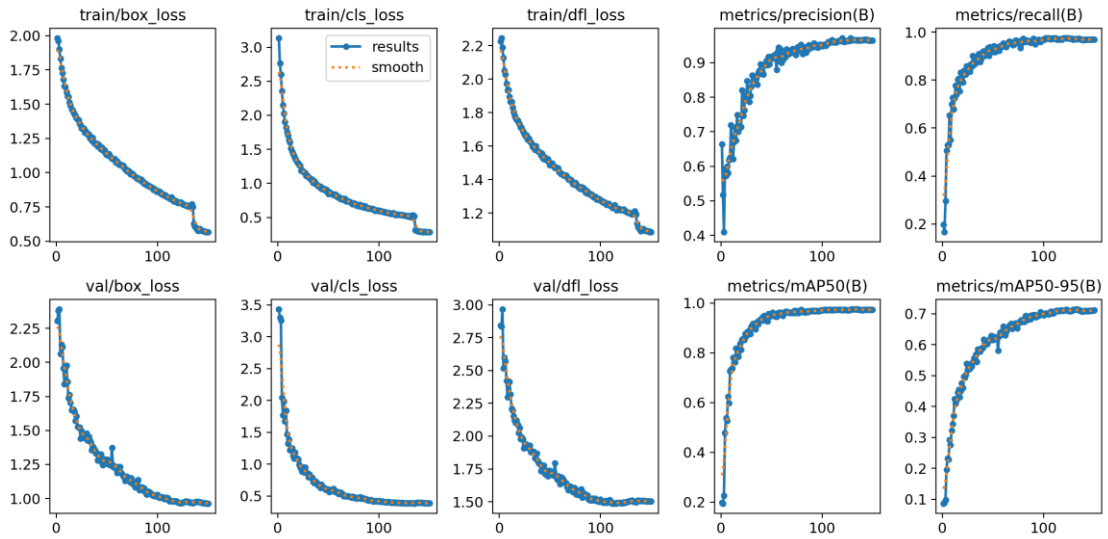


Figure 6.5: YOLO11m Training and Validation Curves.



Figure 6.6: Training Batch Mosaics with Ground-Truth Boxes.

6.6.2 Final Model Evaluation

The final YOLO11m checkpoint was evaluated on a fully held-out test split containing 602 images that were not used in training, validation, or hyperparameter decisions. Precision reached 0.964, recall reached 0.970, the F1 score reached 0.967, mAP@50 reached 0.973, and mAP@50:95 reached 0.711. These results meet the target of high detection accuracy and precise localisation. The mAP@50:95 value is particularly important because strict localisation supports the crossing-line logic used by BasketMonitor.

Table 6.5: Final YOLO11m Evaluation Metrics on the Held-Out Test Set.

Metric	Value	Target	Interpretation
Precision (B)	0.964	>= 0.95	96.4% of detected boxes are correct; low false-positive rate.
Recall (B)	0.970	>= 0.95	97.0% of ground-truth products are detected; low miss rate.
F1 Score	0.967	-	Balanced precision-recall trade-off.
mAP@50 (B)	0.973	>= 0.95	Target exceeded at the standard IoU threshold.
mAP@50:95 (B)	0.711	>= 0.70	Target met across strict IoU thresholds.
Train/box loss	0.567	Converged	Final CIoU box loss.
Train/cls loss	0.284	Converged	Final BCE classification loss.
Train/DFL loss	1.086	Converged	Final distribution focal loss.
Val/box loss	0.964	Low gap	Validation box loss remained stable.
Val/cls loss	0.387	Low gap	Validation classification loss remained stable.
Val/DFL loss	1.503	Low gap	Validation DFL remained stable.

6.6.3 Curve, Confusion Matrix, and Qualitative Results

The precision-recall curve shows a high area under the curve of 0.973, confirming that the model maintains high precision across most recall values. The F1-confidence curve peaks around a confidence threshold of 0.45 and remains broad, meaning that the model does not require extremely precise threshold calibration. The confusion matrix shows strong diagonal dominance across most of the 29 product classes. The main confusions occur between products with similar packaging, such as Pantene Oil Replacement and Herbal Essences Shampoo, V Cola and RedBull, and Bless Conditioner and Herbal Essences Conditioner. These errors are consistent with the visual difficulty of the retail domain rather than random model failure.

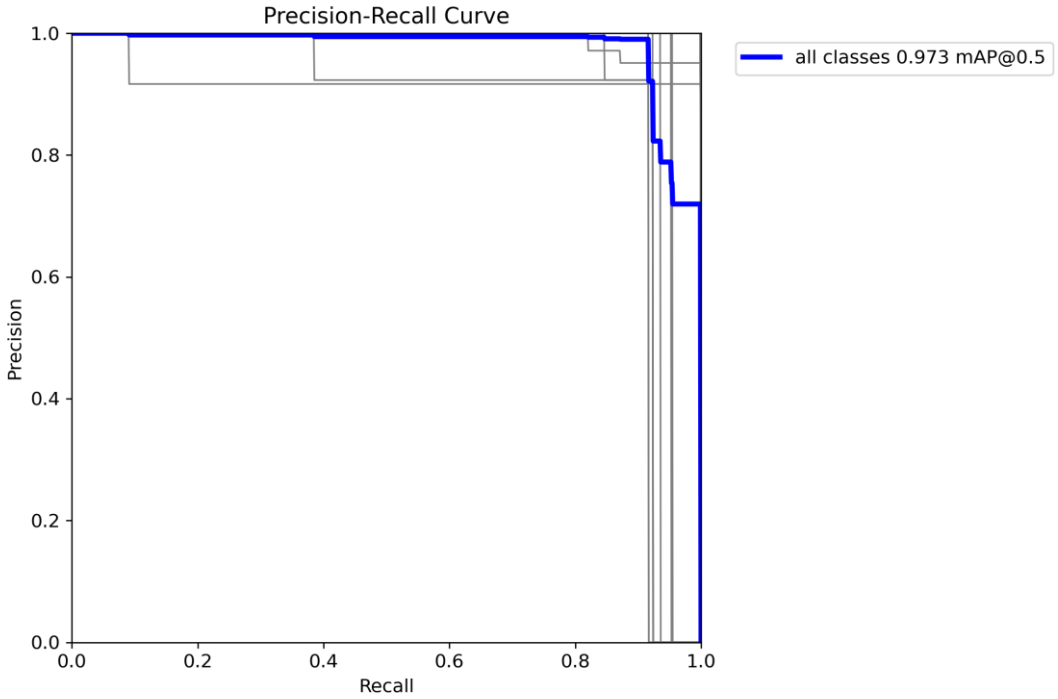


Figure 6.7: Precision-Recall and F1-Confidence Curves.

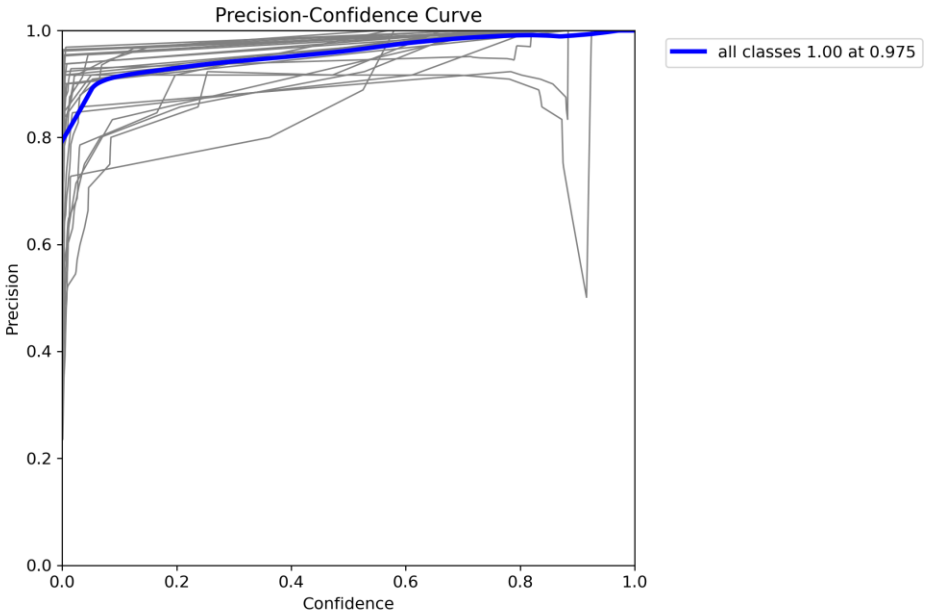


Figure 6.8: Precision and Recall Confidence Curves.

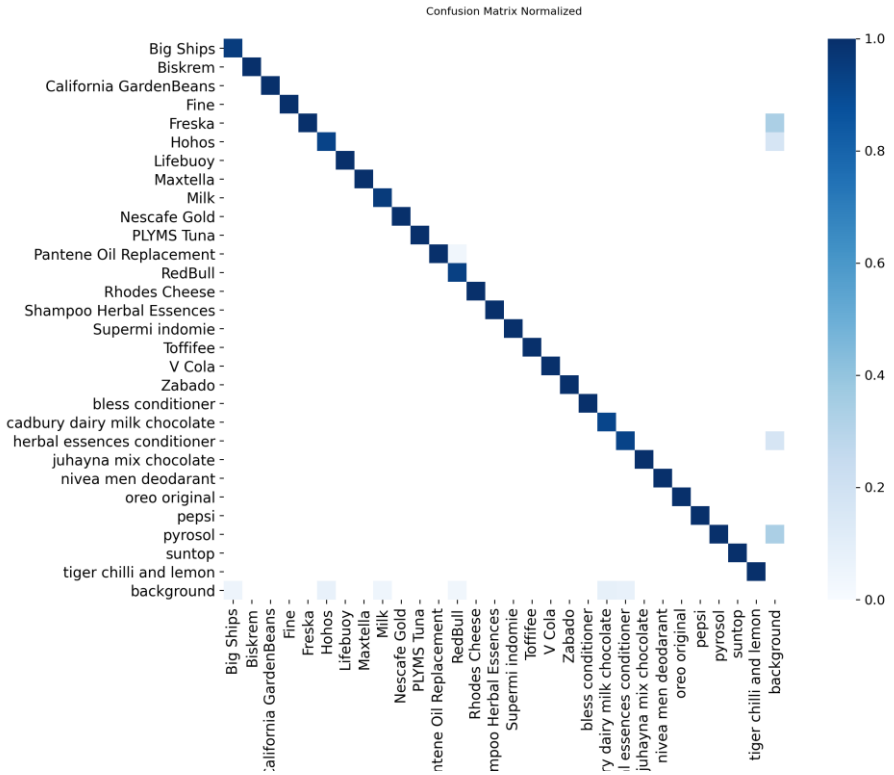


Figure 6.9: AI Product Recognition Confusion Matrix.



Figure 6.10: Ground Truth and YOLO11m Predictions on Validation Batch 0.



Figure 6.11: Ground Truth and YOLO11m Predictions on Validation Batch 1.

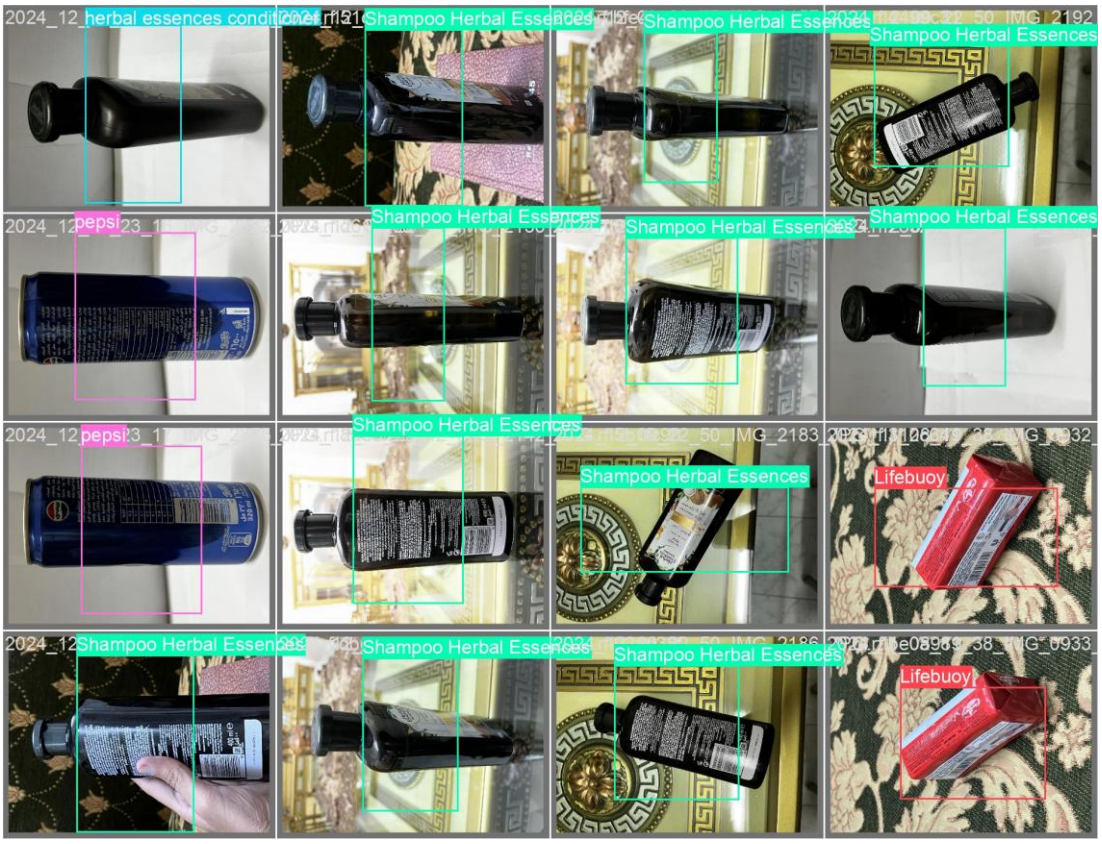


Figure 6.12: Ground Truth and YOLO11m Predictions on Validation Batch 2.

Failure-case review showed four main limitations: heavy occlusion covering more than 70% of the product surface, fast motion blur during add/remove events, extreme lighting reflections on metallic packaging, and severe truncation at image boundaries. The event pipeline mitigates some of these errors because a single missed or blurred frame does not immediately cancel event detection; confirmation frames and tracking history allow adjacent frames to carry the event.

6.6.4 Performance and Deployment Results

Deployment profiling shows that CPU-only inference is not sufficient for the real-time target. CPU deployment reaches approximately 5 FPS, while GPU deployment reaches approximately 35 FPS and therefore exceeds the 20 FPS real-time requirement. MediaPipe adds a smaller latency overhead than YOLO, and the model checkpoint size remains 39 MB in both CPU and GPU deployments. The default Docker image is CPU oriented for compatibility, but production use should use a CUDA-capable base image.

Table 6.6: AI Resource Utilisation Under Sustained 30-FPS Input Load.

Metric	CPU Deploy	GPU Deploy	Notes
Inference latency (YOLO)	about 160 ms	about 18 ms	Per frame, single product.
Inference latency (MediaPipe)	about 18 ms	about 8 ms	Per frame, two hands.
Total pipeline latency	about 185 ms	about 28 ms	Includes encode/decode overhead.
Sustainable frame rate	about 5 FPS	about 35 FPS	Single session.
Peak RAM (API process)	about 1.2 GB	about 800 MB	Model and runtime memory.
VRAM (model)	N/A	about 1.2 GB	YOLO11m weights.
Container image size	4.1 GB	6.8 GB	CUDA layers increase image size.
Model checkpoint size	39 MB	39 MB	Same weights on both devices.

6.7 End-to-End System Testing Discussion

End-to-end testing confirms that Carto works as an integrated prototype. The WebApp can create a list, the cart screen can display a safe QR code, the WebApp can pair the list to the cart through the backend, the cart screen can load the list and show a map, and product events can update the receipt. The prototype still contains demonstration choices, such as mock checkout and static semantic product zones, but the major architectural contracts are validated. The most important outcome is that the system separates responsibilities cleanly: the WebApp owns shopper planning, the cart screen owns the cart display, the mapping subsystem owns the store map, and the AI service owns product event generation.

Table 6.7: End-to-End System Integration Test Results.

Scenario	Expected Result	Actual Result	Status
Prepare list and enter store	A shopping list is saved and selected for pairing.	The WebApp supports list creation, item editing, and activation.	Pass
Pair cart	Scanning QR creates a cart session and receipt.	QR scan and /api/cart/link create active backend state.	Pass
Display map and list on cart	Cart screen changes from waiting to active session state.	Polling active-session returns shopping list, cart items, total, and status.	Pass
Recognise products	AI produces product detections and add/remove events.	YOLO11m meets detection targets and BasketMonitor creates event-level output.	Pass
Update receipt	Added/removed items affect receipt totals.	Device item endpoints and VirtualReceipt support live receipt updates.	Pass
Finish shopping	Checkout finalises or closes the session and frees cart.	Checkout and close-session endpoints complete the trip.	Pass

CHAPTER 7

Conclusions and Future Work

Chapter 7: Conclusions and Future Work

7.1 Introduction

This chapter concludes the thesis after separating testing into Chapter 6. It summarises the project, identifies the contributions of each subsystem, states the limitations that remain in the current prototype, and proposes a future work roadmap for converting the academic prototype into a deployable supermarket solution.

7.2 Project Summary

Carto is a smart shopping cart project that combines web planning, QR-based cart pairing, a cart-mounted screen, LiDAR-generated store mapping, AI product recognition, live receipt management, recommendations, and checkout. The user creates a list before shopping, scans the cart QR at the supermarket, receives the list on the smart cart screen, follows the map and product guidance, adds products that can be detected by the AI system, and finishes shopping through checkout. The final implementation uses a backend-mediated architecture so the QR code remains safe and the database remains the source of truth for sessions, lists, receipts, carts, and products.

7.3 Project Contributions

7.3.1 Map Generation Contribution

The mapping contribution is the custom LiDAR-to-LaserScan decoder and ROS 2 mapping pipeline. It allows LiDAR data that is not originally available as a standard ROS LaserScan to be decoded, cleaned, and published for SLAM Toolbox. The generated PGM map can then be converted into a shopper-friendly representation. This provides the store layout foundation without depending on a localization module in the final prototype.

7.3.2 Web Application Contribution

The WebApp contribution is a full-stack list, session, receipt, checkout, and admin platform. It supports authentication, guest mode, shopping-list CRUD, QR scanning, cart linking, active session monitoring, virtual receipts, mock payment, history, products, carts, sessions, and admin analytics. It demonstrates how a shopper can control the shopping journey before and during the physical supermarket visit.

7.3.3 Cart Screen and QR Integration Contribution

The cart-screen contribution is the mounted display interface and safe pairing model. The screen presents a QR while idle, receives session state through device-authenticated polling, displays the assigned list and map, and sends product events to backend item endpoints. The QR contract is especially important because it avoids transferring sensitive list or device data through the QR itself.

7.3.4 AI Product Recognition Contribution

The AI contribution is a domain-specific Egyptian retail product-recognition and basket-event pipeline. The system uses a custom 6,011-image dataset, YOLO11m object detection, ByteTrack tracking, MediaPipe hand landmarks, and a BasketMonitor state machine. It achieves Precision = 0.964, Recall = 0.970, mAP@50 = 0.973, and mAP@50:95 = 0.711, meeting the target detection and localisation criteria. More importantly, it converts model outputs into meaningful add/remove events suitable for receipt updates.

Table 7.1: YOLOv8m and YOLO11m Comparison.

Criterion	YOLOv8m Best Result	YOLO11m Final Model
Precision (B)	0.935	0.964
Recall (B)	0.921	0.970
mAP@50 (B)	0.947	0.973
mAP@50:95 (B)	0.682	0.711
Training epochs	100	150
Attention mechanism	None	A2C2f with AAttn
Parameter count	about 26M	about 20M
GFLOPs at 640px	about 79	about 68
Fine-grained class confusion	Moderate	Reduced
Checkpoint size	about 52 MB	about 39 MB

7.4 Current Limitations

Several limitations remain. The shopper-friendly map currently uses manually assigned semantic zones, so product-zone data should be connected to a real supermarket database. The checkout flow contains mock/demo payment behaviour and requires production-grade payment confirmation before deployment. The AI service uses in-memory event storage by default, which means events can be lost on restart unless Redis or database persistence is added. CPU-only inference does not meet real-time performance targets. The product catalogue is static, so new products require additional data collection and retraining. The cart screen map is static, with live localization and route overlays intentionally removed from the final scope.

7.5 Future Work Roadmap

Future work should focus on turning the prototype into a scalable retail platform. The map layer should connect the friendly map to a structured product-location database and generate shortest paths between product zones and tell the user their exact real time location in the store. The WebApp should have stronger role-based access, and production monitoring. The cart screen should add stronger offline caching, clearer error recovery, and a polished Raspberry Pi deployment. The AI layer should adopt active learning, TensorRT acceleration, persistent event storage, multi-camera support, and possibly depth sensing to reduce ambiguity in add/remove events.

Table 7.2: Current Limitations and Future Work Roadmap.

Area	Current Limitation	Future Work
Map generation	Semantic zones are manually overlaid after PGM conversion.	Store product database should define aisle, shelf, and zone coordinates automatically.
WebApp	Payment is currently mock/demo oriented.	Integrate production payment gateway flow and audit-ready receipts.
Cart screen	Static map and no localization marker.	Add route overlays and optional future position markers if localization returns to scope.
AI events	In-memory event store can be lost on restart.	Use Redis Streams or database event persistence.
AI performance	CPU-only deployment reaches about 5 FPS.	Use CUDA/TensorRT and shared inference worker pool.
Product catalogue	Adding new products requires retraining.	Adopt active learning and investigate open-vocabulary or CLIP-based detection.
Scalability	Single worker/pilot deployment is limited.	Use load-balanced API replicas, sticky WebSocket routing, and GPU inference workers.

7.6 Final Conclusion

This project successfully presented the design and development of Carto, a smart shopping cart system that aims to improve the supermarket shopping experience through the integration of web technologies, QR-based pairing, indoor map generation, cart-screen interaction, artificial intelligence product recognition, and automated receipt updating. The system was developed to reduce the time and effort required by shoppers to prepare lists, locate products, track selected items, and complete the checkout process. By combining these components into one connected workflow, Carto demonstrates how intelligent retail systems can provide a more organized, interactive, and efficient shopping journey.

The project began with the development of a web application that allows users to create and manage shopping lists before entering the supermarket. This list is then transferred to the smart cart through a QR pairing process, allowing the cart screen to display the active shopping session, selected products, receipt details, and store map. The cart screen acts as the main interaction point during shopping, connecting the user's prepared list with the supermarket environment and supporting the user throughout the shopping process.

A major part of the project was the map generation module, where LiDAR data was decoded and processed to generate a usable store map. The generated PGM occupancy grid was converted into a shopper-friendly representation that can be displayed clearly on the cart interface. This conversion is important because raw robotic maps are not suitable for normal shoppers, while the processed version provides a clearer visual layout that can support route planning and product-location guidance inside the supermarket.

The AI product recognition component was also a key contribution of the system. A YOLO-based object detection model was trained and evaluated using supermarket product images, allowing the system to recognize products placed into the cart. The AI workflow was designed to receive camera input, detect product classes, generate predictions with confidence values, and send the recognized results to the backend and cart interface. This enabled the system to update the receipt automatically and mark detected products from the user's shopping list, reducing manual scanning and improving the automation level of the shopping process.

Testing showed that the main system components were able to perform their intended functions, including shopping-list creation, QR pairing, cart-session display, map processing, product detection, receipt updates, and end-to-end communication between modules. The results confirmed the feasibility of the proposed smart cart concept and showed that the different subsystems can work together as one integrated solution. Although the prototype still has limitations, such as the need for larger product datasets, stronger real-world deployment testing, improved recommendation logic, and further optimization for supermarket-scale operation, the implemented system provides a strong foundation for future development.

In conclusion, Carto achieved the main objective of creating an intelligent shopping cart prototype that combines software, mapping, AI recognition, and user-interface design to enhance the shopping experience. The project proves that smart retail systems can reduce shopping complexity, improve product tracking, and support a more seamless checkout process. With further improvements in hardware integration, product database expansion, real-time recommendation accuracy, and payment system deployment, Carto can be developed into a practical and scalable solution for modern supermarkets.

References:

Li, Y., An, J., He, N., Li, Y., Han, Z., Chen, Z., & Qu, Y. (2025). A Review of Simultaneous Localization and Mapping Algorithms Based on Lidar. World Electric Vehicle Journal, *16*(1), 56. https://doi.org/10.3390/wevj16010056 [1]

Qiao, J., Guo, J., & Li, Y. (2024). Simultaneous localization and mapping (SLAM)-based robot localization and navigation algorithm. Applied Water Science, 14, 151. https://doi.org/10.1007/s13201-024-02183-6 [2]

Fan, X., & Pan, W. (2025). Elevated high-precision mapping and localization technology for periodic inspections. Scientific Reports, *15*(1), 4810. https://doi.org/10.1038/s41598-025-86133-5 [3]

Dusmanu, M., Miksik, O., Schönberger, J. L., & Pollefeys, M. (2021). Cross-descriptor visual localization and mapping. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 6058-6067). https://openaccess.thecvf.com/content/ICCV2021/html/Dusmanu_Cross-Descriptor_Visual_Localization_and_Mapping_ICCV_2021_paper.html [4]

Mod, K., Goswami, A., & Gautam, P. (2017). Mapping and localization using robot operating system. In Proceedings of the International Conference on Computing, Communication and Automation (ICCCA) (pp. 1–4). https://doi.org/10.1109/CCAA.2017.8229840 [5]

Yue, X., He, M., & Zhang, Y. (2023). LiDAR-based SLAM for robotic mapping: State of the art and new frontiers. arXiv. https://arxiv.org/abs/2311.00276 [6]

Zhen, W., & Scherer, S. (2017). A unified 3D mapping framework using a 3D or 2D LiDAR. In *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 4562–4569). https://www.researchgate.net/publication/328628317_A_Unified_3D_Mapping_Framework_using_a_3D_or_2D_LiDAR [7]

Suleymanoglu, B., Soycan, M., & Toth, C. (2022). Indoor mapping: Experiences with LiDAR SLAM. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, XLIII-B1-2022*, 279–285. https://doi.org/10.5194/isprs-archives-XLIII-B1-2022-279-2022 [8]

Shibata, L. R. de S., Ravankar, A. A., Luces, J. V. S., & Hirata, Y. (2025). An AI-based shopping assistant system to support the visually impaired. [9]

Suryawanshi, R., Shirsath, B., Bhande, B., Todkar, A., Awate, A., & Tajmohammed, K. (2024). TechCart: A smart shopping experience. TechRxiv. https://www.techrxiv.org/users/843563/articles/1234753-techcart-a-smart-shopping-experience [10]

Zulfiqar, M. I., Khalid, A., Siddig, A., Nawaz, M. J., & Saay, S. (2025). AI-driven smart shopping carts with real-time tracking and inventory forecasting for enhanced retail efficiency. IEEE Access, 13, 55576–55584. [11]

Mariam, F., Nandi, S. P. N., Ganavi, B. S., & Gowrishankar, B. S. (2022). Design and implementation of smart shopping trolley with mobile cart application. International Journal of Engineering Research in Computer Science and Engineering (IJERCSE), 9(7), 8–10. https://www.researchgate.net/publication/364966608_Design_and_Implementation_of_Smart_Shopping_Trolley_with_Mobile_Cart_Application [12]

Wolniak, R., Stecula, K., & Aydın, B. (2024). Digital transformation of grocery in-store shopping—scanners, artificial intelligence, augmented reality and beyond: A review. Foods, *13*(18), 2948. https://doi.org/10.3390/foods13182948 [13]

Pathan, Z. H., Sikandar, A., Aziz, A., Asghar, S. A., & Saleem, M. (2025). User-centric approach to design a digital shopping cart: Enhancing user experience in super stores. Journal of Independent Studies and Research Computing (JISR-C), *23*(1), 58–63. https://doi.org/10.31645/JISRC.25.23.1.7 [14]

Shetty, R., Pawar, D., Poojari, R., Patel, R., & Mal, J. (2021). Smart shopping cart using machine vision along with machine learning. International Research Journal of Engineering and Technology (IRJET), *8*(10), 1731–1735. https://www.researchgate.net/publication/359352285_Smart_Shopping_Cart_using_Machine_Vision_along_with_Machine_Learning [15]

Schultz, C. D., & Zacheus, P. (2025). Smart shopping carts in food retailing: Innovative technology and shopping experience in stationary retail. Journal of Consumer Behaviour, 24(1), 436–454. https://doi.org/10.1002/cb.2426 [16]

Nguyen, H.-N. (2025). A novel smart shopping system with new payment solution at mini-mart. Journal of Technical Education Science, xx(xx). https://doi.org/10.54644/jte.2025.1793 [17]

Shakir, A. M., Mudhafar, Y., & Al-muqarm, A. M. A. (2023). Modernizing the shopping experience: The smart shopping cart. In Proceedings of the 6th International Conference on Engineering Technology and its Applications (IICTA) (pp. 636–640). IEEE. Retrieved from https://www.researchgate.net/publication/376722549_Modernizing_The_Shopping_Experience_The_Smart_Shopping_Cart [18]

Ragesh, N., Giridhar, B., Lingeshwaran, D., Siddharth, P., & Peeyush, K. P. (2019, April). Deep learning based automated billing cart. In 2019 International Conference on Communication and Signal Processing (ICCSP) (pp. 0779–0782). IEEE. https://doi.org/10.1109/ICCSP.2019.8697978 [19]

Liang, X., Qi, Y., Xi, S., Meng, G., & Wang, Z. (2025). Design and development of full self-service smart shopping system for unmanned supermarket. *Journal of King Saud University - Computer and Information Sciences, 37*(1), 152. https://doi.org/10.1007/s44443-025-00182-4 [20]

Aathira, P., Chandran, A., Rajendran, D. P., & Mohanan, H. (2019). Smart shopping trolley system using IoT. International Journal of Computer Science and Information Technology Research, 7(2), 70–78. https://www.researchpublish.com/upload/book/Smart%20Shopping%20Trolley-7435.pdf [21]

Prasad, K. V. S., Karthik, K., Kumar, O. C., Prathiksha, B. S., & Salimath, K. (2023). Smart shopping trolley with automated billing. IOP Conference Series: Materials Science and Engineering, 1295(1), 012008. https://doi.org/10.1088/1757-899X/1295/1/012008 [22]

Jacob, M. R., Navaneeta, M., Devinanda, R., & Deebalakshmi, R. (2023). Smart cart-assisted in-store shopping. SSRN Electronic Journal. https://doi.org/10.2139/ssrn.4927047 [23]

Venu, S., & Gurusamy, M. (2025). A comprehensive review of path planning algorithms for autonomous navigation. Results in Engineering. https://doi.org/10.1016/j.rineng.2025.107750 [24]

Hou, P., & Huang, S. (2025). BCSM-YOLO: An improved product package recognition algorithm for automated retail stores based on YOLOv11. IEEE Access, *13*, 139665–139678. https://doi.org/10.1109/ACCESS.2025.3595175 [26]

Johnson, P., Nguyen, Q. H., & Latham, D. (2022). Performance evaluation of ROS-based SLAM algorithms for handheld indoor mapping and tracking systems [Preprint]. LJMU Research Online. http://researchonline.ljmu.ac.uk/id/eprint/18245/ [27]

Thale, S. P., Prabhu, M. M., Thakur, P. V., & Kadam, P. (2020). ROS based SLAM implementation for autonomous navigation using Turtlebot. In ITM Web of Conferences (Vol. 32, p. 01011). EDP Sciences. https://doi.org/10.1051/itmconf/20203201011 [28]

Oh, J.-S., & Chun, I.-G. (2020). Implementation of smart shopping cart using object detection method based on deep learning. Journal of the Korea Academia-Industrial Cooperation Society, *21*(7), 262–269. https://doi.org/10.5762/KAIS.2020.21.7.262 [29]

Chandrasekar, P., & Sangeetha, T. (2014). Smart shopping cart with automatic billing system through RFID and ZigBee. In 2014 International Conference on Information Communication and Embedded Systems (ICICES) (pp. 1–4). IEEE. https://doi.org/10.1109/ICICES.2014.7033877 [30]

Al-Hakmani, A. K. S., & Sajan, A. (2020). Design and implementation of a smart shopping trolley using RFID technology. Journal of Student Research, 4(1), [article pages]. https://www.jsr.org/index.php/path/article/view/978 [31]

Carto project source repositories: Carto_ROS_Mapping, carto1.1 integration and screen, and Carto web application. Provided project repositories, 2026. [32]

Open Robotics. (2024). ROS 2 Humble Documentation. https://docs.ros.org/en/humble/ [33]

Macenski, S. (2024). SLAM Toolbox for lifelong mapping and lifelong navigation in ROS. https://github.com/SteveMacenski/slam_toolbox [34]

Vercel. (2024). Next.js Documentation. https://nextjs.org/docs [35]

Meta Open Source. (2024). React Native Documentation. https://reactnative.dev/docs/getting-started [36]

Expo. (2024). Expo Documentation. https://docs.expo.dev/ [37]

Prisma. (2024). Prisma ORM Documentation. https://www.prisma.io/docs [38]

PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. https://www.postgresql.org/docs/ [39]

NextAuth.js. (2024). NextAuth.js Documentation. https://next-auth.js.org/ [40]

Stripe. (2024). Stripe API and Payment Documentation. https://docs.stripe.com/ [41]

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788. [42]

Ultralytics. (2024). YOLO model documentation and training framework. https://docs.ultralytics.com/ [43]

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. Proceedings of the European Conference on Computer Vision (ECCV). [44]

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C. L., Yong, M. G., Lee, J., Chang, W. T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172. [45]

Roboflow. (2024). Roboflow computer vision dataset management and annotation platform documentation. https://docs.roboflow.com/ [46]

FastAPI. (2024). FastAPI framework documentation. https://fastapi.tiangolo.com/ [47]

Docker Inc. (2024). Docker documentation. https://docs.docker.com/ [48]

Streamlit. (2024). Streamlit documentation. https://docs.streamlit.io/ [49]

Carto AI product recognition documentation. Internal project documentation provided by the project team, 2026. [50]